SAP HANA Lab Exam — Merged Solutions (Module 1 to Module 3 Task 4)

Course Code: 25NHOP715 / 26NHOP515 • Tasks 1 to 12 Stitched in Exact Lab Manual Order with Complete Zip & PDF Source References

1. Master Task-to-Zip Reference & Page Directory (Tasks 1–12)

Every stitched page in this document carries a top banner showing the exact **Module & Task**, **Lab Manual pages**, **Zip archive**, and **Original SAP PDF filename & page number**, plus a bottom footer showing the **Merged PDF page number (Pages 4–87)**.

Module / Task	Lab Manual	Task Description	Source Zip File	Exact Source PDF & Pages	Merged Pages
Mod 1, Task 1	**Task 1** (pp. 13–16)	Web-based Development Workbench basics (`gbi-demo`, `UI5App`, `index.html`, `.xsaccess`)	`curriculum_2_pdf (2).zip`	`03_Native_Development_Chapter_01_Web-based_Develeopment_Workbench_Exercise_en.pdf` (pp. 1–5)	**pp. 4–8**
Mod 1, Task 2	**Task 2** (p. 17)	Display tables in Catalog perspective & inspect `GBI_DEMO_CUSTOMER` structure & content	`curriculum_1_pdf (3).zip`	`03_Introduction_to_SAP_HANA_Chapter_3_Data_Modeling_Exercise_en.pdf` (pp. 1–2, Step 1)	**pp. 9–10**
Mod 1, Task 3	**Task 3** (p. 18)	Display calculation view in Editor perspective & inspect Projection mapping (`GBI_DEMO_PROD_DIM_CV`)	`curriculum_1_pdf (3).zip`	`03_Introduction_to_SAP_HANA_Chapter_3_Data_Modeling_Exercise_en.pdf` (p. 3, Step 2)	**p. 11**
Mod 1, Task 4	**Task 4** (pp. 19–20) *[Cut off]*	Create Customer, Product, and Sales `.hdbtable` definitions; populate & view sample data	`curriculum_1_pdf (3).zip`	`06_Introduction_to_SAP_HANA_Chapter_5_Case_Study_en.pdf` (pp. 2–4: Specs) `07_Introduction_to_SAP_HANA_Chapter_5_Case_Study_Solution_en.pdf` (pp. 1–6: Tables; pp. 7–13: Data)	**pp. 12–14** (Specs) **pp. 15–20** (Tables) **pp. 21–27** (Data)
Mod 2, Task 1	**Task 5** (pp. 21–24)	Graph Viewer overview, vertex/edge inspection & filter (`TYPE = Supplier`) on `GBI_GRAPH_DEMO`	`curriculum_6_pdf (2).zip`	`01_Introduction_to_Graph_Processing_Chapter_01_Introduction_to_Graphs_Exercise_en.pdf` (pp. 4–8, Ex. 2)	**pp. 28–32**
Mod 2, Task 2	**Task 6** (pp. 25–26)	Single Source Shortest Path algorithm from Rubber (`RBR00`) to Finished Tire (`TYRE1000`)	`curriculum_6_pdf (2).zip`	`03_Introduction_to_Graph_Processing_Chapter_02_Graph_Processing_on_SAP_HANA_Exercise_en.pdf` (pp. 1–3, Ex. 3)	**pp. 33–35**
Mod 2, Task 3	**Task 7** (pp. 27–29)	Neighborhood Search for Touring Seat Kit (Start Vertex: `TRSK1000`, Direction: `incoming`)	`curriculum_6_pdf (2).zip`	`03_Introduction_to_Graph_Processing_Chapter_02_Graph_Processing_on_SAP_HANA_Exercise_en.pdf` (pp. 4–6, Ex. 4)	**pp. 36–38**
Mod 2, Task 4	**Task 8** (pp. 30–32)	Pattern-Matching algorithm for volcano eruption in China (`V1: Supplier`, `V2: China`)	`curriculum_6_pdf (2).zip`	`03_Introduction_to_Graph_Processing_Chapter_02_Graph_Processing_on_SAP_HANA_Exercise_en.pdf` (pp. 7–9, Ex. 5)	**pp. 39–41**

Module / Task	Lab Manual	Task Description	Source Zip File	Exact Source PDF & Pages	Merged Pages
Mod 3, Task 1	**Task 9** (pp. 33–39)	Create & consume OData service (`GBI.xsodata`) to expose table `GBI_DEMO_PRODUCT` as `Products`	`curriculum_2_pdf (2).zip`	`06_Native_Development_Chapter_02_OData_in_SAP_HANA_Case_Study_en.pdf` (pp. 2–8)	**pp. 42–48**
Mod 3, Task 2	**Task 10** (pp. 40–48)	Create SAPUI5 MVC Hello World App (`MVCHelloWorld`: `index.html`, `Main.view.xml`, `Main.controller.js`)	`curriculum_2_pdf (2).zip`	`10_Native_Development_Chapter_03_SAPUI5_Exercise_#2_en.pdf` (pp. 1–9)	**pp. 49–57**
Mod 3, Task 3	**Task 11** (pp. 49–53)	Create Calculation Views for Customer (`CUSTOMER_DIM_CV_XXX`), Product (`PRODUCT_DIM_CV_XXX`), and Sales Cube (`SALES_CUBE_CV_XXX`)	`curriculum_1_pdf (3).zip`	`06_Introduction_to_SAP_HANA_Chapter_5_Case_Study_en.pdf` (pp. 9–16: Steps) `07_Introduction_to_SAP_HANA_Chapter_5_Case_Study_Solution_en.pdf` (pp. 14–31: Solution)	**pp. 58–65** (Steps) **pp. 66–83** (Solution)
Mod 3, Task 4	**Task 12** (pp. 53–57)	Create OData service using Calculation Views (`Sales` & `SalesOrders` endpoints)	`curriculum_2_pdf (2).zip`	`06_Native_Development_Chapter_02_OData_in_SAP_HANA_Case_Study_en.pdf` (pp. 9–12)	**pp. 84–87**

2. Written Answers & Copy-Paste Code for Task 2, Task 3 & Missing Task 4 Pages

In the source PDFs, **Tasks 2 & 3** ask worksheet questions and **Task 4** has its `.hdbtable` code inside screenshots on pages 15–20 (and is cut off in your lab manual after Customer). Here are the exact answers and copy-pasteable code blocks from `curriculum_1_pdf (3).zip`:

Task 2 & Task 3 Worksheet Answers (from Catalog & Editor Perspectives)

- **Task 2 (a) Data type and length of field `COUNTRY` in `GBI_DEMO.GBI_DEMO_CUSTOMER`:** SQL Data Type is `NVARCHAR` with Dimension/Length `2` (Column Store Data Type: `STRING`).
- **Task 2 (b) Is `GBI_DEMO_CUSTOMER` a Column Store or Row Store table?** It is a `COLUMN` store table (shown in the top-right *Type: COLUMN* field of the table definition).
- **Task 2 (c) Number of records & customer number for Silicon Valley Bikes:** The table has **25 rows** total (23 distinct customers; Customer `6000` Windy City Bikes and one other customer have 2 validity rows). Silicon Valley Bikes (Palo Alto) has Customer Number **`10000`**.
- **Task 3 — Database table & schema used in `GBI_DEMO_PROD_DIM_CV.calculationview`:** Clicking the `Projection` node reveals the underlying database table **`GBI_DEMO.GBI_DEMO_PRODUCT`** (Schema: `GBI_DEMO`, Table: `GBI_DEMO_PRODUCT`), with 7 mapped output columns (`PRODUCT` [Key], `PRODUCT_NAME`, `PRODUCT_CATEGORY`, `DIVISION`, `COLOR`, `PRODUCT_GROUP`, `INTERNAL_PRICE`).

Task 4: Complete `.hdbtable` Definitions (Cut Off in Lab Manual on p. 20)

Create all three files inside package `gbi-student-XXX` (replace `XXX` with your assigned 3-digit student ID). The screenshot solutions for these three files appear on **Merged Pages 15–20** (from `07_Introduction_to_SAP_HANA_Chapter_5_Case_Study_Solution_en.pdf`, pp. 1–6):

1. Customer Master Data Table — File name: `CUSTOMER_ATTR_XXX.hdbtable`

```
table.schemaName = "GBI_XXX";
table.tableType = COLUMNSTORE;
table.columns = [
    {name = "CUSTOMER_NUMBER"; sqlType = NVARCHAR; length = 10; },
    {name = "CUSTOMER_NAME"; sqlType = NVARCHAR; length = 20; },
    {name = "CITY"; sqlType = NVARCHAR; length = 35; },
```

```
    {name = "VALID_TO"; sqlType = DATE; },
    {name = "VALID_FROM"; sqlType = DATE; },
    {name = "SALES_ORGANIZATION"; sqlType = NVARCHAR; length = 4; },
    {name = "COUNTRY"; sqlType = NVARCHAR; length = 2; }
];
table.primaryKey.pkcolumns = ["CUSTOMER_NUMBER", "VALID_TO"];
```

2. Product Master Data Table — File name: `PRODUCT_ATTR_XXX.hdbtable` (Missing from Lab Manual)

```
table.schemaName = "GBI_XXX";
table.tableType = COLUMNSTORE;
table.columns = [
    {name = "PRODUCT"; sqlType = NVARCHAR; length = 8; },
    {name = "PRODUCT_NAME"; sqlType = NVARCHAR; length = 40; },
    {name = "PRODUCT_CATEGORY"; sqlType = NVARCHAR; length = 3; },
    {name = "DIVISION"; sqlType = NVARCHAR; length = 2; },
    {name = "INTERNAL_PRICE"; sqlType = DECIMAL; precision = 17; scale = 2; },
    {name = "PRICE"; sqlType = DECIMAL; precision = 17; scale = 2; },
    {name = "COLOR"; sqlType = NVARCHAR; length = 10; },
    {name = "PRODUCT_GROUP"; sqlType = NVARCHAR; length = 20; }
];
table.primaryKey.pkcolumns = ["PRODUCT"];
```

3. Sales Transaction Data Table — File name: `SALES_XXX.hdbtable` (Missing from Lab Manual)

```
table.schemaName = "GBI_XXX";
table.tableType = COLUMNSTORE;
table.columns = [
    {name = "YEAR"; sqlType = INTEGER; },
    {name = "MONTH"; sqlType = INTEGER; },
    {name = "DAY"; sqlType = INTEGER; },
    {name = "CUSTOMER_NUMBER"; sqlType = NVARCHAR; length = 10; },
    {name = "ORDER_NUMBER"; sqlType = NVARCHAR; length = 10; },
    {name = "ORDER_ITEM"; sqlType = NVARCHAR; length = 3; },
    {name = "PRODUCT"; sqlType = NVARCHAR; length = 8; },
    {name = "SALES_QUANTITY"; sqlType = INTEGER; },
    {name = "UNIT_OF_MEASURE"; sqlType = NVARCHAR; length = 3; },
    {name = "REVENUE"; sqlType = DECIMAL; precision = 17; scale = 2; },
    {name = "CURRENCY"; sqlType = NVARCHAR; length = 3; },
    {name = "DISCOUNT"; sqlType = DECIMAL; precision = 17; scale = 2; }
];
table.primaryKey.pkcolumns = ["ORDER_NUMBER", "ORDER_ITEM"];
```

4. Inserting & Viewing Sample Data in Task 4 (SQL Console or SDI Flowgraphs on Merged Pages 21–27):

```
-- Option A: Quick SQL Console Insert (copy demo records or insert sample rows, then right-click table ->
Open Content):
INSERT INTO "GBI_XXX"."CUSTOMER_ATTR_XXX" SELECT * FROM "GBI_DEMO"."GBI_DEMO_CUSTOMER";
INSERT INTO "GBI_XXX"."PRODUCT_ATTR_XXX"  SELECT * FROM "GBI_DEMO"."GBI_DEMO_PRODUCT";
INSERT INTO "GBI_XXX"."SALES_XXX"         SELECT * FROM "GBI_DEMO"."GBI_DEMO_SALES";
-- Or single-row manual INSERT:
INSERT INTO "GBI_XXX"."CUSTOMER_ATTR_XXX" VALUES ('1000','Rocky Mountain Bikes','Denver','9999-12-
31','1900-01-01','UW00','US');
INSERT INTO "GBI_XXX"."PRODUCT_ATTR_XXX"  VALUES ('BOTL1000','Water
Bottle','AS','AS',10.00,20.00,'Black','Accessories');
INSERT INTO "GBI_XXX"."SALES_XXX"         VALUES
(2011,5,15,'1000','100001','010','BOTL1000',10,'ST',200.00,'USD',10.00);
```

EXERCISE

Explore the SAP HANA Web-based Development Workbench

Exercise Before you start with the development, you need to learn how to use the SAP HANA Web-based Development Workbench, the IDE of choice for SAPUI5 development**.**

Time 15 minutes

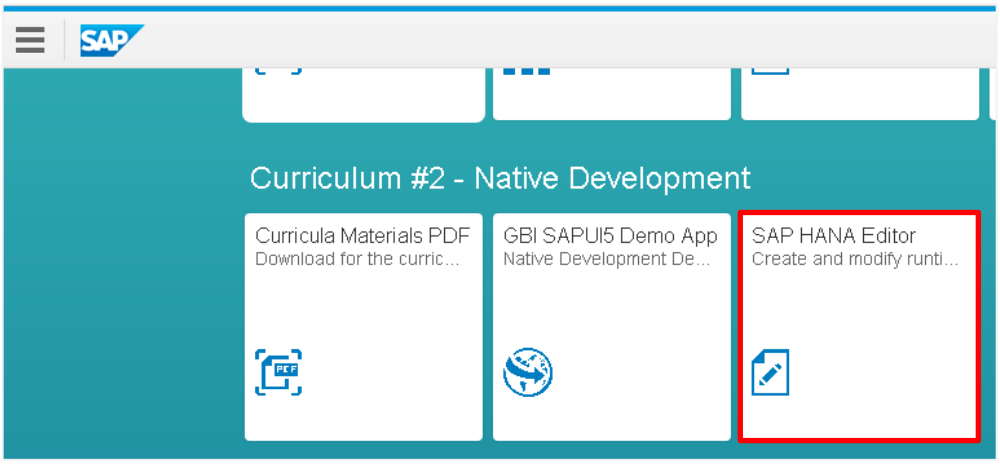
Task Learn the basics of using SAP HANA Web-based Development environment.

Name (Position) Mona Falco (Development Specialist)

Menu path

To open the Web-based Development Workbench Editor, logon to the Fiori Launchpad of the SAP HANA system and follow the menu path:

Fiori Launchpad ► Curriculum #2 - Native Development ► SAP HANA Editor



Open Fiori Launchpad

Open the SAP Fiori Launchpad which is available under

`http://[fqdn]:[http port]/`

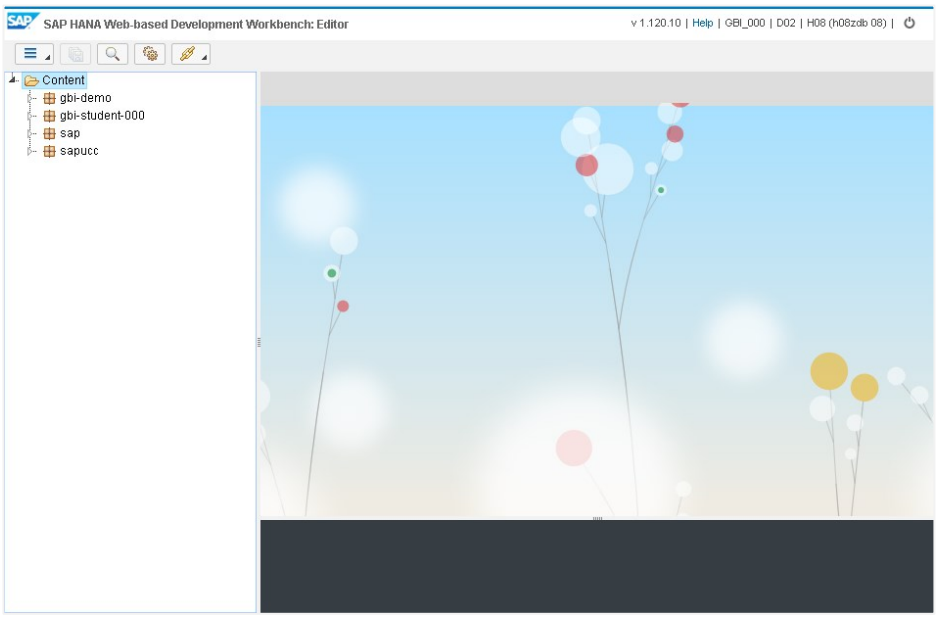
Logon

Logon with your username and your password. All required information will be provided by your instructor.

Open Editor

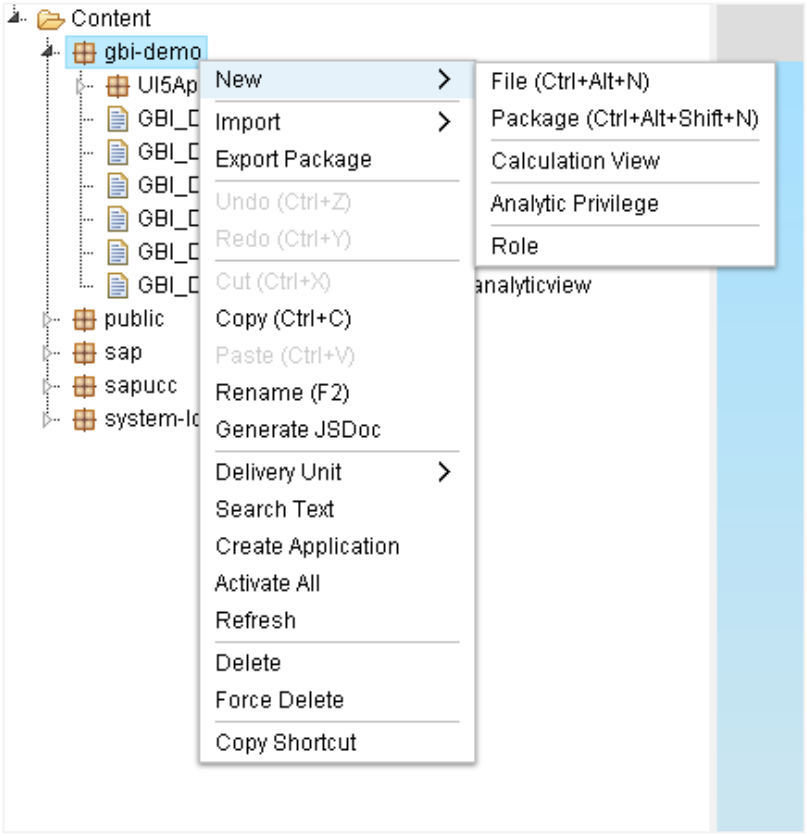
Click on the SAP HANA Editor tile to open the Web-based Development Workbench Editor. The following screen will appear:

EXERCISE



First, right-click on the package *gbi-demo* to open the context menu.

Package gbi-demo



As you can see, the context menu offers a variety of functions, from the creation of new packages and files to the possibility to refresh the package content or delete the whole package with its content.

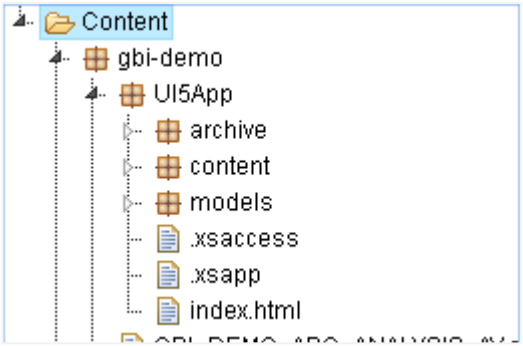
Remember that the functions shown in the context menu depend on the type of package or file for which you open it. An OData service definition will display other options then a XML-based view for a MVC application.

Content-depending context menu

EXERCISE

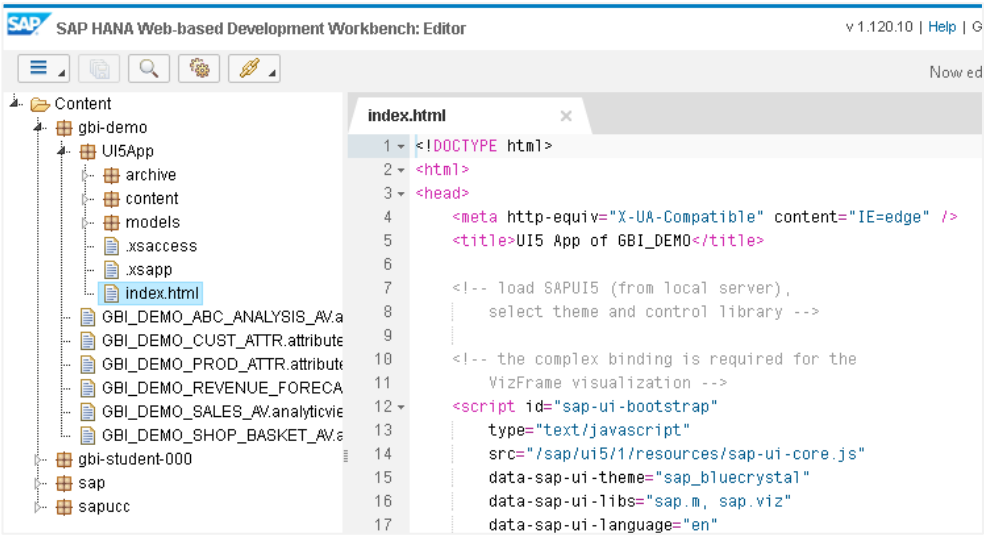
Next, we're going to take a look at how the editor works. Expand the package *gbi-demo* ► *UI5App*.

Package UI5App



Click on the file *index.html* once to open its source code in the editor.

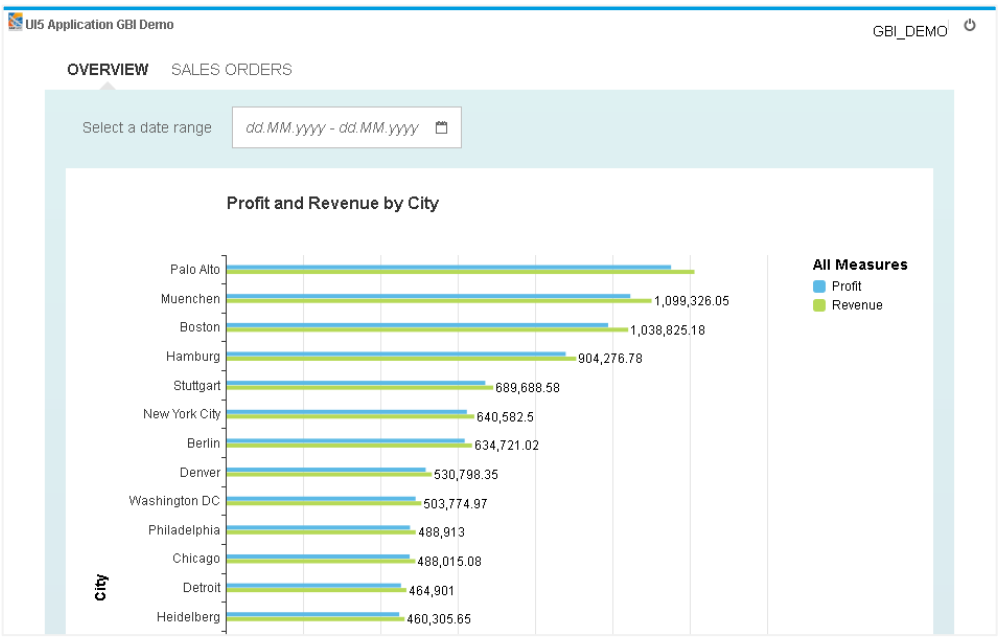
index.html



When opening files of different types, you will see that, depending on the file type, the editor will apply a different syntax high-lighting. For the *index.html*, the syntax highlighting is automatically set to HTML.

You will also notice that the toolbar has changed after opening the file. The toolbar will show buttons for different functions, depending on the selected file. In this case, for example, the Run button was added to the toolbar.

Click on the run button to open the *index.html* in a new browser-window.

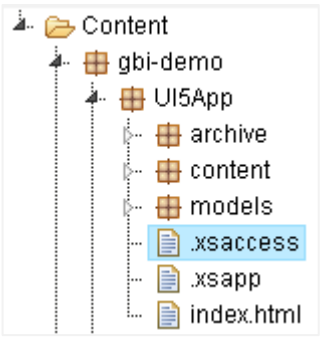


Now we want to take a look at how the editor works when editing different files.

.xsaccess

Close the application window and go back to the SAP HANA Web-based Development Workbench.

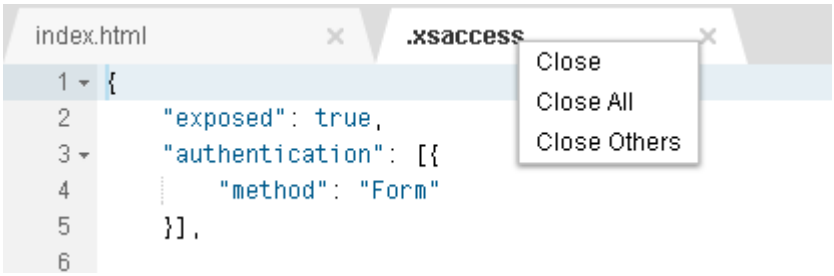
In addition to the index.html, open the *.xsaccess* file by clicking once on it.



Now we have two different files opened in the editor, both visible in different tabs.



You can switch between the files by clicking on the tabs. If you opened many files in different tabs and you want to close all tabs or all but the one, you can right click on a tab to bring up the tab context menu.



Remember to close all files that you do not need any more. It helps you to stay organized and increases the editor's performance.

Close all tabs except the one that contains the *index.html* by right-clicking on the *index.html* and selecting *Close Others*.

If you need more horizontal space for the editor, you can double-click on the tab to collapse the content section on the left side. A double-click on the tab will bring it back.

EXERCISE

SAP University Alliances

SAP HANA Data Modeling

Duration:	30 minutes
Scenario:	The Global Bike Company has acquired a new data warehouse solution based on SAP HANA. You use the SAP HANA Web-based Development Workbench to explore the basic functionality provided by this solution.
Objective:	The objective is to gain an overview of the Web Workbench and the different data structures used by SAP HANA. You will learn how to connect to a SAP HANA database. Furthermore you will learn how to navigate through the catalog and editor perspective of the Web Workbench, and how to display different objects in these perspectives. Finally, you will learn how to access the HANA Launchpad including the HANA curriculum material.

Task 1: Logon to the Web-based Development Workbench

Exercise Logon to the Web-based Development Workbench **Time** 10 min

Task You logon to the SAP HANA Web Workbench

Start the Web Workbench using the URL
http://<**Full Hostname**>:<**http port**>/sap/hana/ide/

The hostname and http port will be provided by your instructor

Enter the user name *GBI-###* and your password.

User name and password will be provided by your instructor

If you logon the first time, you will be asked to change your password. Please do so.

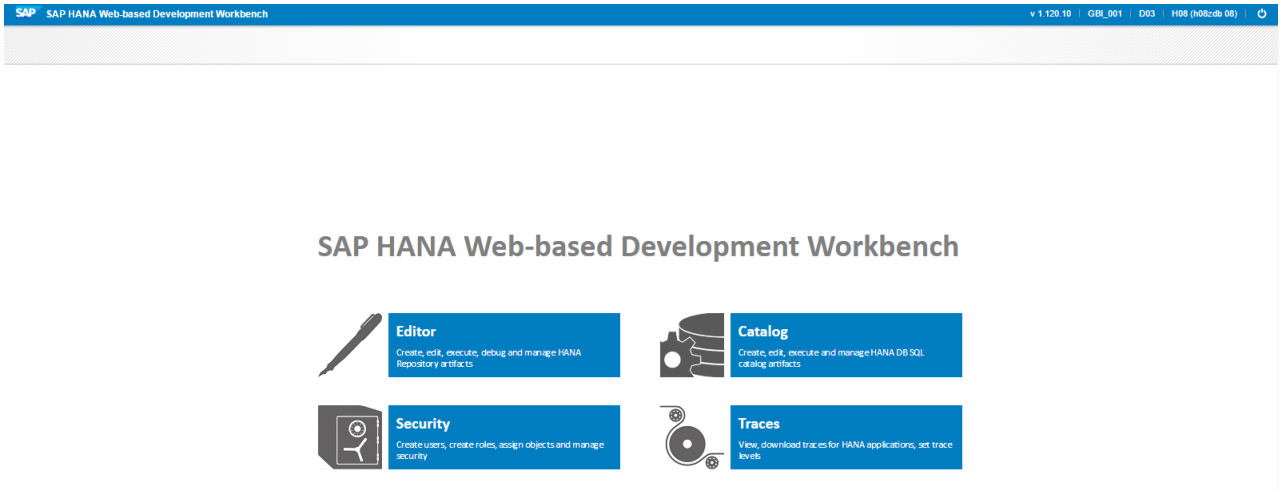
The password rules are as following:

- Minimum 8 characters
- Password must contain capital and small letters
- Password must contain numbers

After logon, you are directed to the welcome screen of the Web Workbench.

© SAP SE

SAP University Alliances



Task 2: Display a table in the catalog perspective

Exercise Display a table in the catalog perspective **Time** 10 min

Task Navigate through the catalog perspective and display a table.

On the welcome screen, click on *Catalog* to open the catalog perspective.

Open the table GBI_DEMO.GBI_DEMO_CUSTOMER

Hint: In the following, tables are identified using the naming convention <database schema.table>.

Navigator View → Catalog → GBI_DEMO → Tables → GBI_DEMO_CUSTOMER

Open the table definition

Right click on table name → Open Definition

Answer the following questions using the table definition:

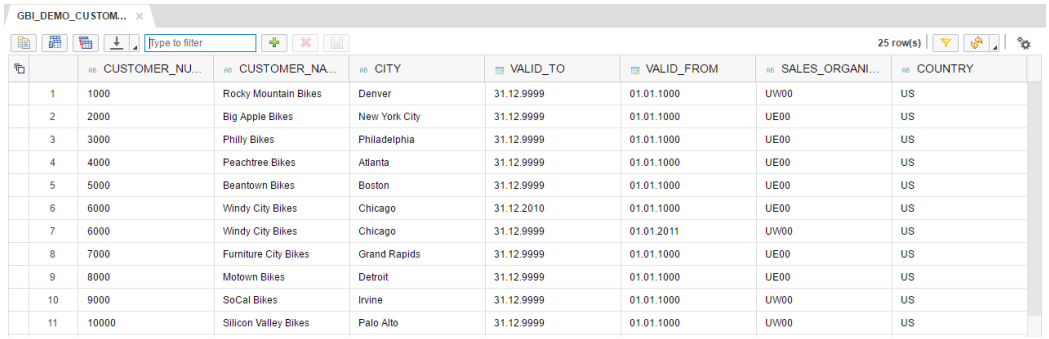
a) What is the data type and length of field COUNTRY?

b) Is the table of type *Column Store* or *Row Store*?

Display the contents of the table

Right click on table name → Open Content

SAP University Alliances



GBI_DEMO_CUSTOM...

Type to filter — 25 row(s)

	CUSTOMER_NU...	CUSTOMER_NA...	CITY	VALID_TO	VALID_FROM	SALES_ORGANI...	COUNTRY
1	1000	Rocky Mountain Bikes	Denver	31.12.9999	01.01.1000	UW00	US
2	2000	Big Apple Bikes	New York City	31.12.9999	01.01.1000	UE00	US
3	3000	Philly Bikes	Philadelphia	31.12.9999	01.01.1000	UE00	US
4	4000	Peachtree Bikes	Atlanta	31.12.9999	01.01.1000	UE00	US
5	5000	Beantown Bikes	Boston	31.12.9999	01.01.1000	UE00	US
6	6000	Windy City Bikes	Chicago	31.12.2010	01.01.1000	UE00	US
7	6000	Windy City Bikes	Chicago	31.12.9999	01.01.2011	UW00	US
8	7000	Furniture City Bikes	Grand Rapids	31.12.9999	01.01.1000	UE00	US
9	8000	Motown Bikes	Detroit	31.12.9999	01.01.1000	UE00	US
10	9000	SoCal Bikes	Irvine	31.12.9999	01.01.1000	UW00	US
11	10000	Silicon Valley Bikes	Palo Alto	31.12.9999	01.01.1000	UW00	US

Task 3: Display a calculation view in the editor perspective

Exercise Display a calculation view in the editor perspective **Time** 5 min

Task Navigate through the editor perspective and display a calculation view.

Switch back to the welcome screen of the Web Workbench

On the welcome screen, click on *Editor*

Open the calculation view GBI_DEMO_PROD_DIM_CV located in package **gbi-demo.WebIDE**

Navigator View → Content → gbi-demo → WebIDE → GBI_DEMO_PROD_DIM_CV.calculationview

Double click on the view name.

What is the name of the database tables used in this view?

Hint: Click on the *Projection* node. The name of the schema and the table is listed in the *Mapping* section.

SAP University Alliances

Step 1: Create the database tables

At first you create the database tables in SAP HANA. These tables will serve as data foundation for the information models (views) that will be created in a later step.

As indicated in the business scenario, you need to create three tables, one for sales transaction data, one for the customer master data, and one table for product master data.

Remark: You create all database tables in the database schema ***GBI_XXX***, whereas ***XXX*** stand for your individual ID. The namespace and your individual ID will be provided by the instructor.

1. Logon to the SAP HANA Web-based Development Workbench

Start the Web Workbench using the URL
http://<**Full Hostname**>:<**http port**>/sap/hana/ide/

Select the *Editor* perspective

2. Create a table definition for the customer master data

Navigate to your package gbi-student-xxx and create a new table definition using the context menu.

Package: gbi-student-XXX

File (table) name: CUSTOMER_ATTR_***XXX***.*hdbtable*

Use following code to create the table. Replace the name of the schema with your own schema.

```
table.schemaName = "GBI_XXX";
table.tableType = COLUMNSTORE;
table.columns = [
    {name = "CUSTOMER_NUMBER"; sqlType = NVARCHAR; length = 10; },
    {name = "CUSTOMER_NAME"; sqlType = NVARCHAR; length = 20; },
    {name = "CITY"; sqlType = NVARCHAR; length = 35; },
    {name = "VALID_TO"; sqlType = DATE; },
    {name = "VALID_FROM"; sqlType = DATE; },
```

© SAP SE

SAP University Alliances

```
{name = "SALES_ORGANIZATION"; sqlType = NVARCHAR; length = 4; },
{name = "COUNTRY"; sqlType = NVARCHAR; length = 2; }
];
table.primaryKey.pkcolumns = ["CUSTOMER_NUMBER", "VALID_TO"];
```

Save the file.

Switch to the *Catalog* perspective and check if the table has been created in your schema GBI_XXX.

3. Create a table definition for the product master data

Use following details to create the table and proceed as described in the previous step:

Package: gbi-student-XXX

Schema: GBI_XXX

Table definition: PRODUCT_ATTR_*XXX.hdbtable*

Table type: Column store

Add following columns to you table:

Name	Key	Data type	Length
PRODUCT	X	NVARCHAR	8
PRODUCT_NAME		NVARCHAR	40
PRODUCT_CATEGORY		NVARCHAR	3
DIVISION		NVARCHAR	2
INTERNAL_PRICE		DECIMAL	17,2
PRICE		DECIMAL	17,2
COLOR		NVARCHAR	10
PRODUCT_GROUP		NVARCHAR	20

Hint: Fields of type DECIMAL are created using following syntax:

{name = "fieldname"; sqlType = DECIMAL; precision = 17; scale = 2; }.

© SAP SE

SAP University Alliances

Switch to the *Catalog* perspective and check if the table has been created in your schema GBI_XXX.

4. Create a table definition for the sales transaction data

Use following details to create the table:

Package: gbi-student-XXX

Schema: GBI_XXX

Table definition: SALES_*XXX.hdbtable*

Table type: Column store

Add following columns to your table:

Name	Key	Data Type	Length
YEAR		INTEGER	
MONTH		INTEGER	
DAY		INTEGER	
CUSTOMER_NUMBER		NVARCHAR	10
ORDER_NUMBER	X	NVARCHAR	10
ORDER_ITEM	X	NVARCHAR	3
PRODUCT		NVARCHAR	8
SALES_QUANTITY		INTEGER	
UNIT_OF_MEASURE		NVARCHAR	3
REVENUE		DECIMAL	17,2
CURRENCY		NVARCHAR	3
DISCOUNT		DECIMAL	17,2

Switch to the *Catalog* perspective and check if the table has been created in your schema GBI_XXX.

© SAP SE

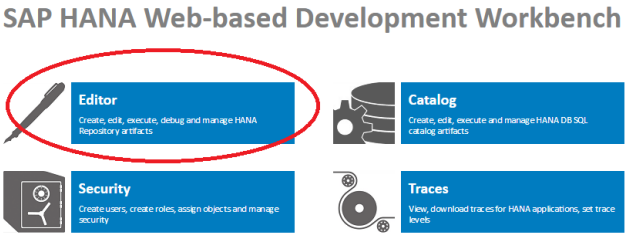
SAP University Alliances

SAP HANA Data Modeling Case Study Solution

Step 1: Create the database tables

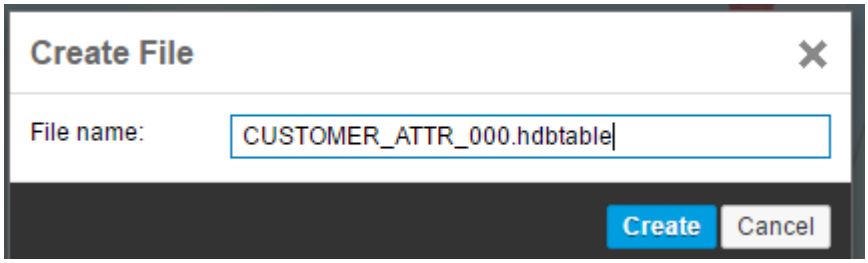
1. Logon to the SAP HANA Web-based Development Workbench





2. Create a table definition for the customer master data

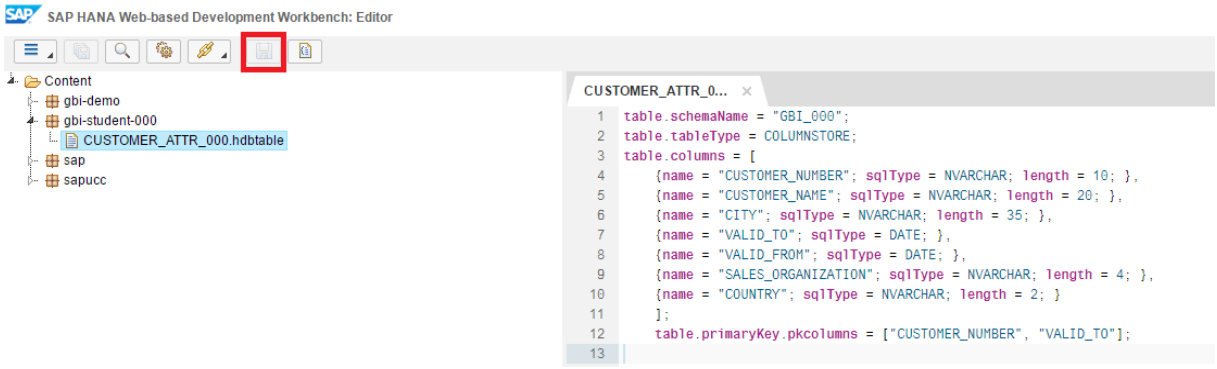
Editor perspective → Content → gbi-student-xxx → Right click → New →File



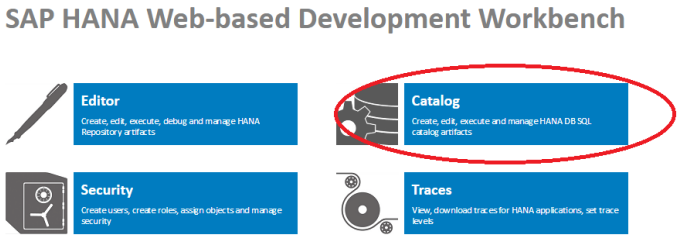
The code for the table definition should look as following:

© SAP SE

SAP University Alliances

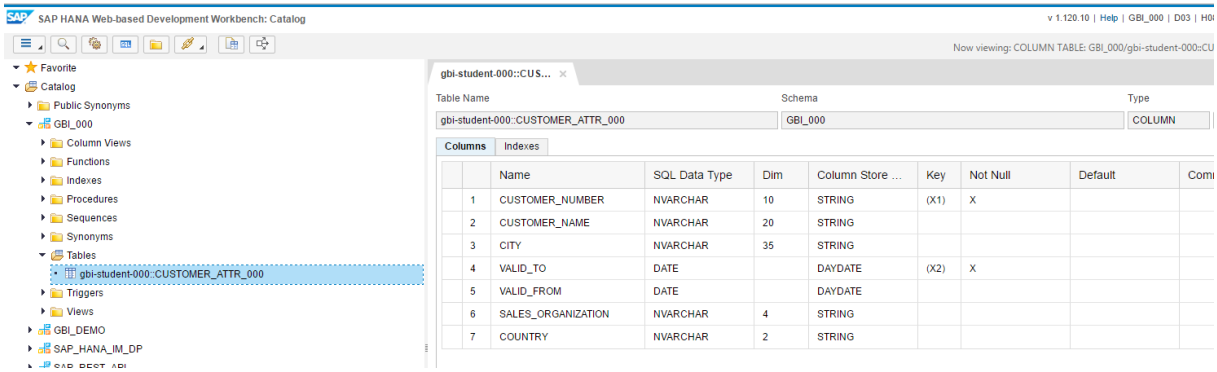






Catalog perspective → Catalog → GBI_XXX → Tables → CUSTOMER_ATTR_XXX

The created table should look as following:

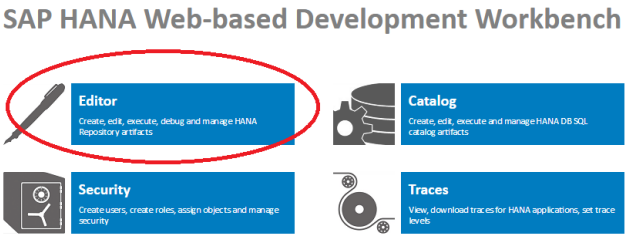


© SAP SE

SAP University Alliances

3. Create a table definition for the product master data

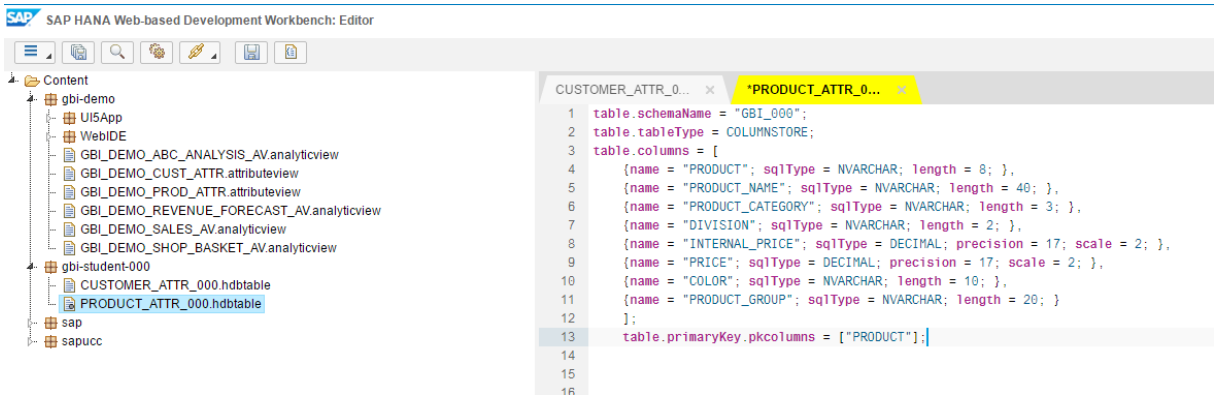




Editor perspective → Content → gbi-student-xxx → Right click → New →File

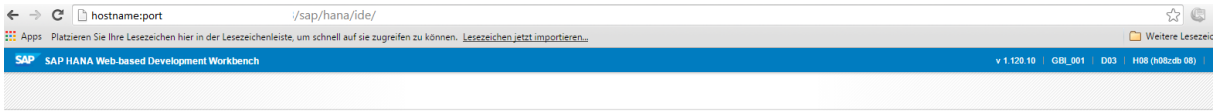


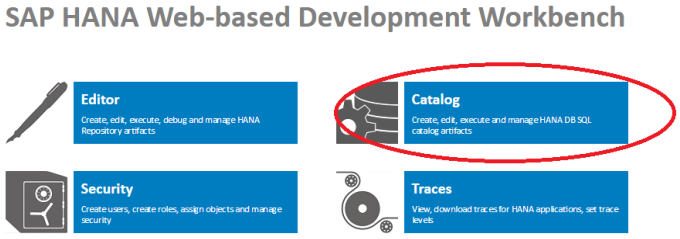
The code for the table definition should look as following:



© SAP SE

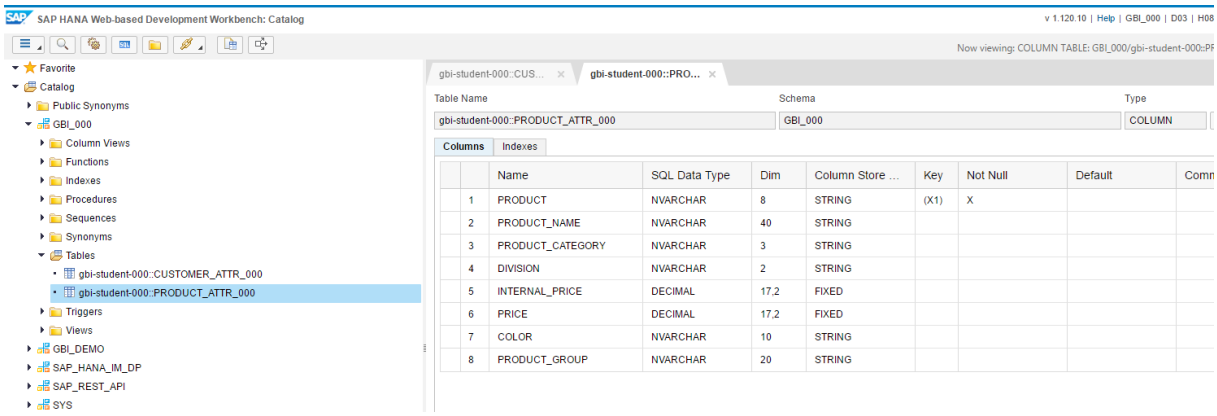
SAP University Alliances





Catalog perspective → Catalog → GBI_XXX → Tables → PRODUCT_ATTR_XXX

The created table should look as following:

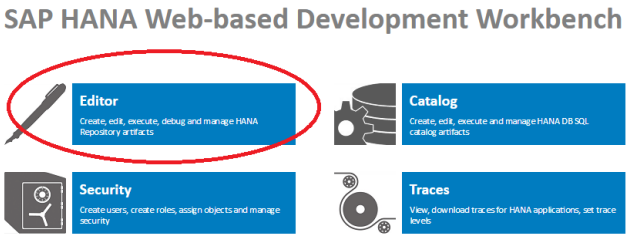


© SAP SE

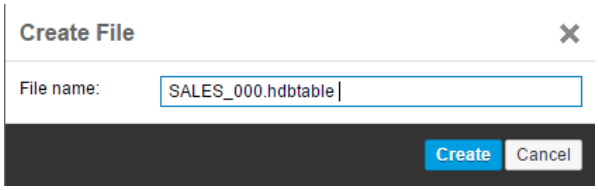
SAP University Alliances

4. Create a table definition for the sales transaction data

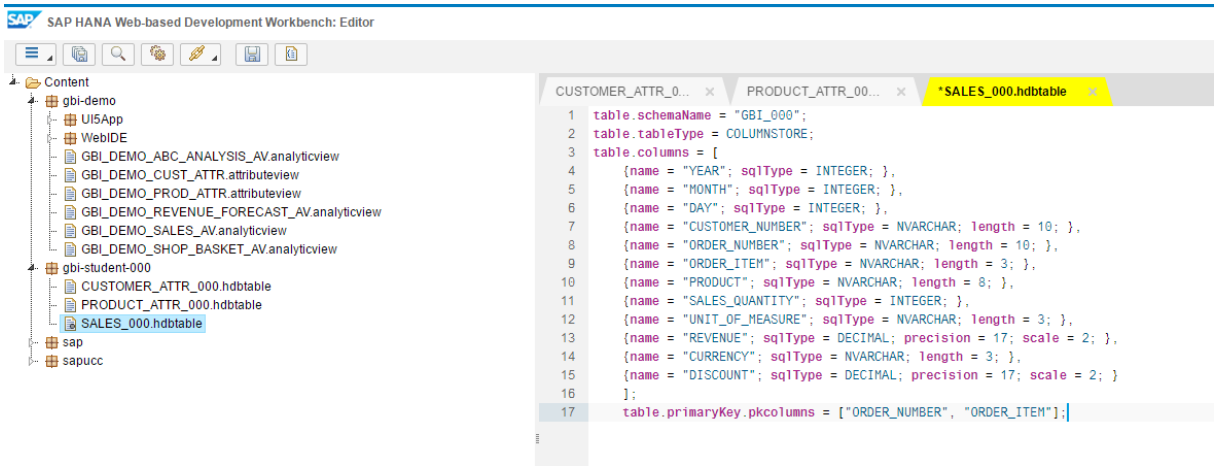




Editor perspective → Content → gbi-student-xxx → Right click → New →File

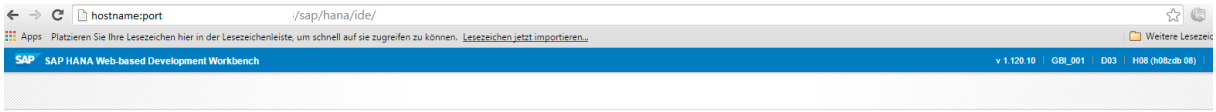


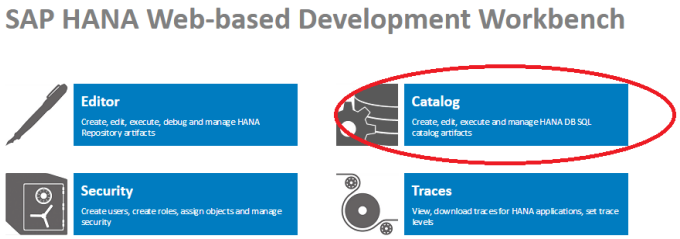
The code for the table definition should look as following:



© SAP SE

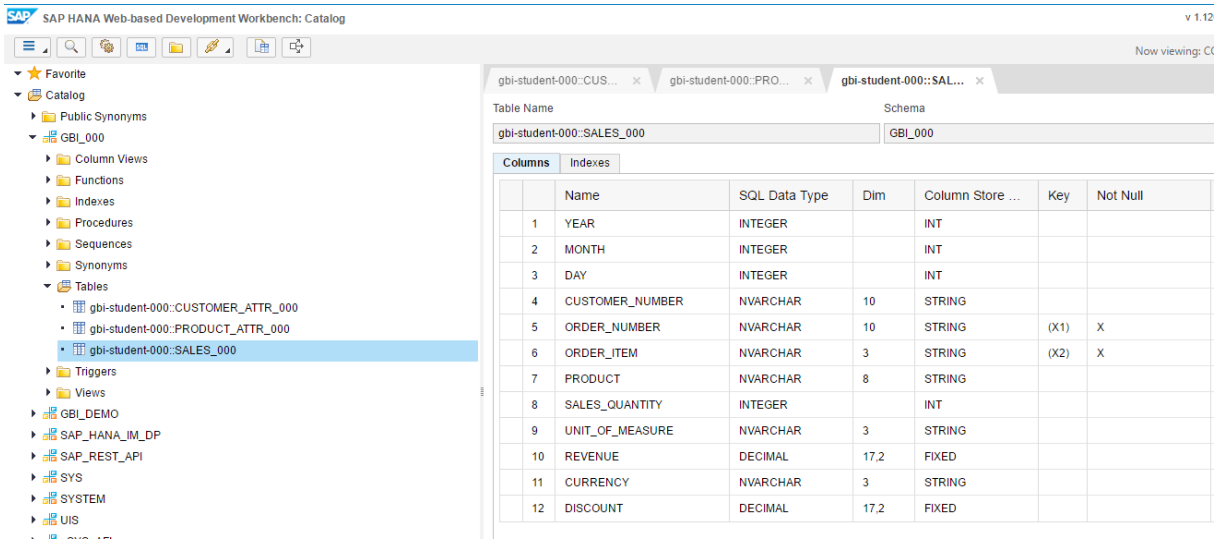
SAP University Alliances





Catalog perspective → Catalog → GBI_XXX → Tables → SALES_XXX

The created table should look as following:



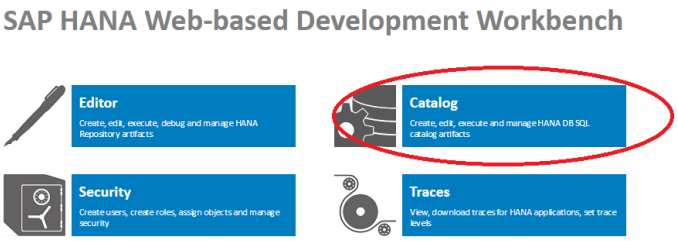
© SAP SE

SAP University Alliances

Step 2: Data Provisioning

1. Open the catalog perspective in the SAP HANA Web Workbench

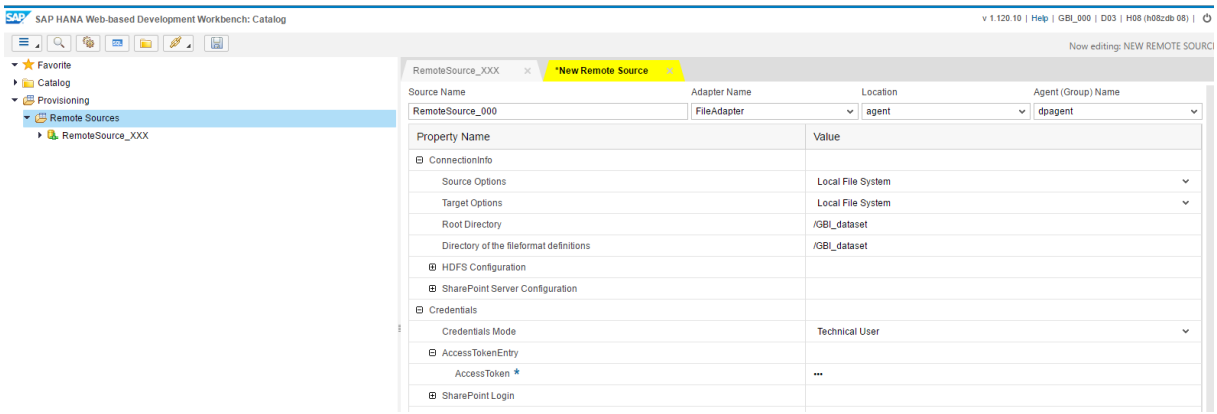




2. Add a new remote source to the SAP HANA system

Navigator View → Provisioning → Remote Sources

Right-click on *Remote Sources*, select "New Remote Source" and enter the data as shown in the screenshot. Please type in *gbi* as AccessToken and press ENTER.



Save the remote source using the *Save* button.

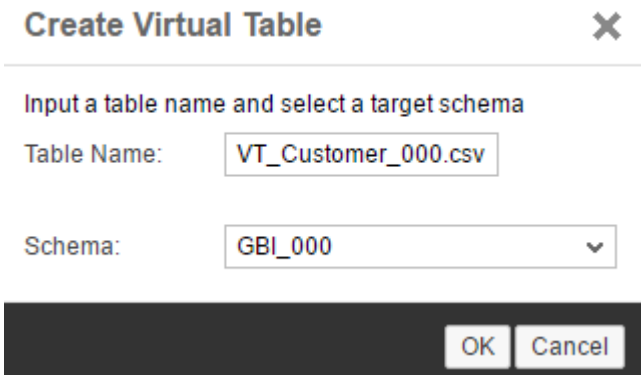
© SAP SE

SAP University Alliances

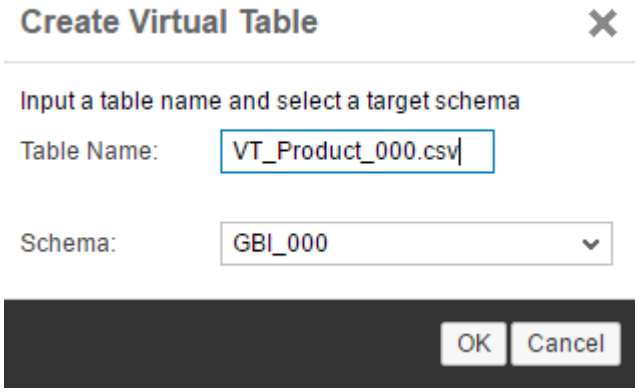
3. Add virtual data tables

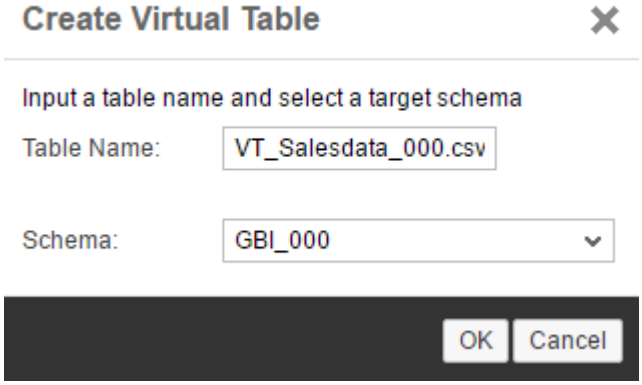
Navigator View → Provisioning → Remote Sources → <Source Name>

Right-click on the file *Customer.csv* and select "New virtual table". Name the table VT_Customer_XXX.csv, select your database schema and click on OK:



Repeat this step for the *Product.csv* and *Salesdata.csv* file:



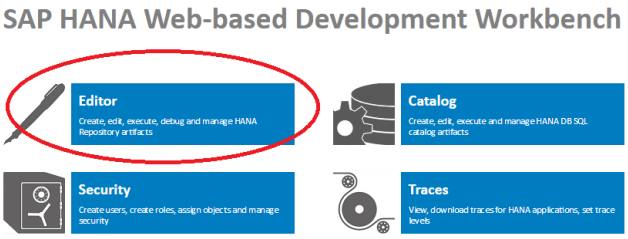


© SAP SE

SAP University Alliances

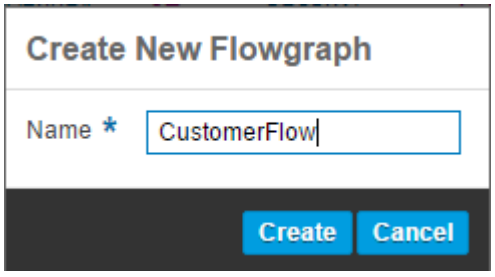
4. Create a new flowgraph model to load the customer data





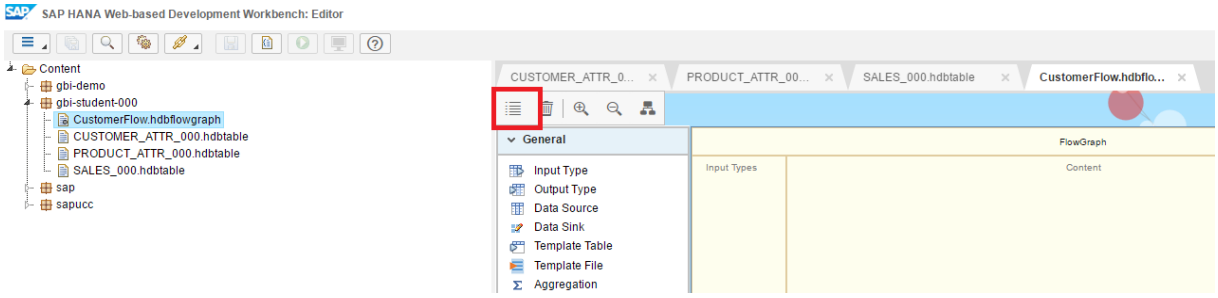
Right-click on the package *gbi-student-xxx* and select

New → Flow Graph

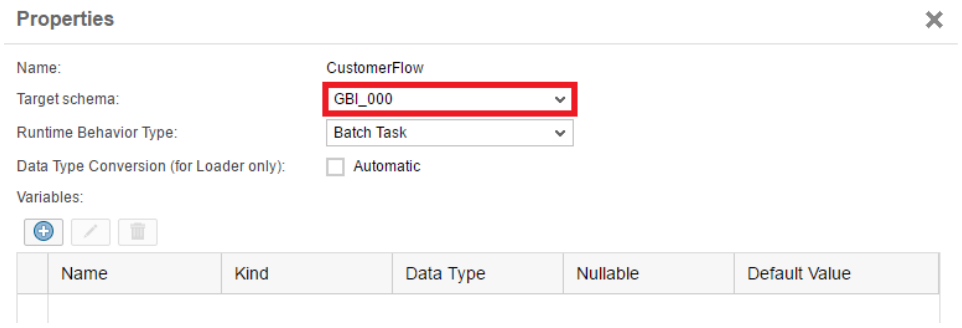


Click on *Create*.

Open the properties view on the top left corner of the empty flowgraph



There, change the target schema to your schema:

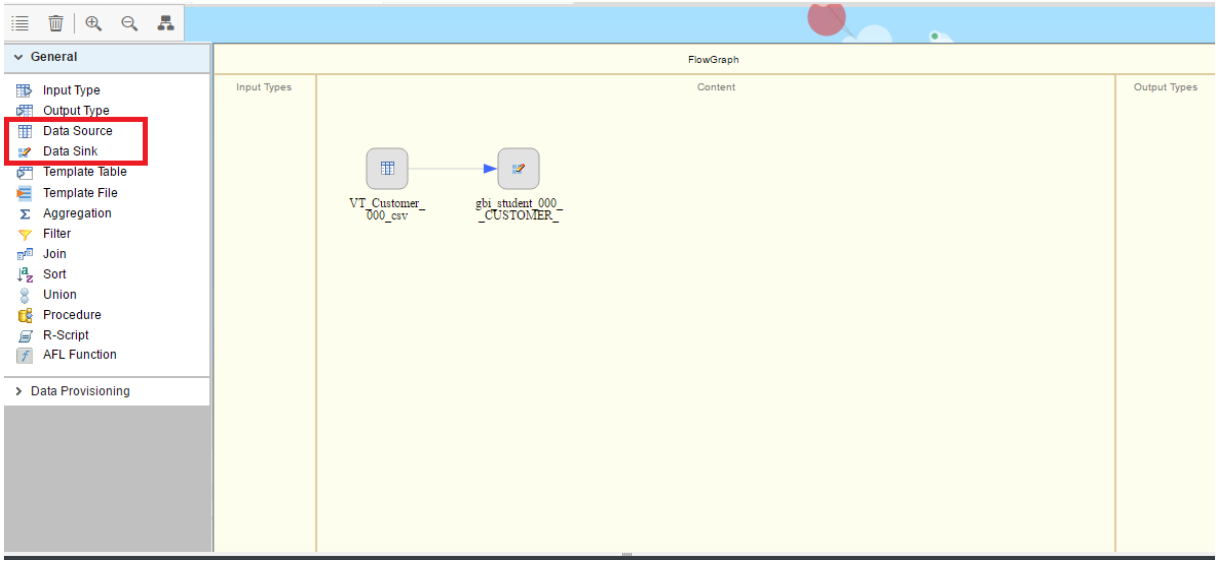


SAP University Alliances

5. Within the flowgraph model, establish a data connection between the virtual table and the data table

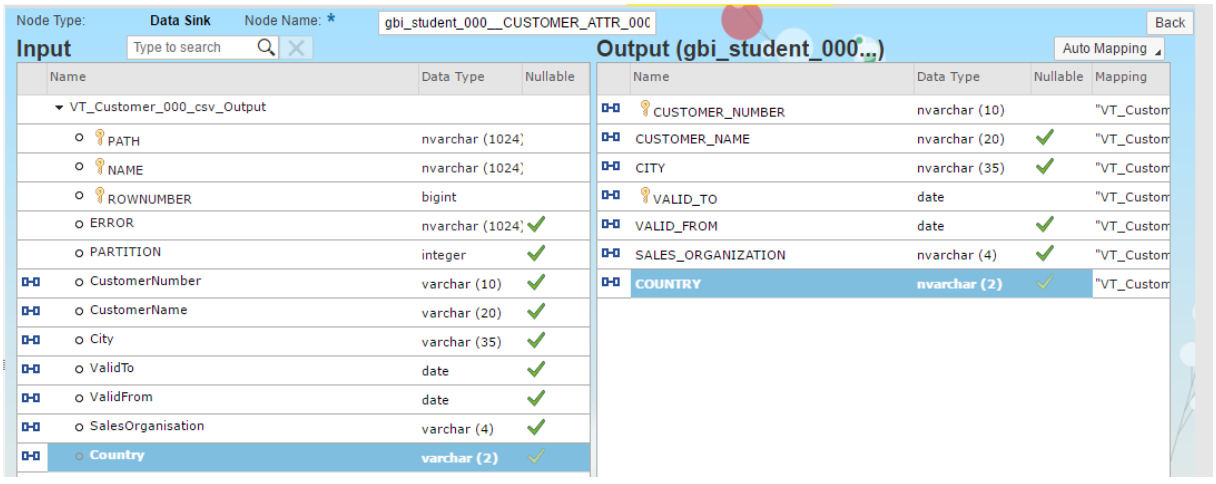
Add a Data Source to the workspace and select the virtual table "VT_Customer_**XXX**.csv" as table. Add a Data Sink to the workspace and select the table CUSTOMER_ATTR_**XXX.** Both tables are assigned to the schema GBI_XXX.

Create a connection between the two tables, by linking data source and data sink.



Double click on the data sink to open the mapping section.

There, perform the field mapping between the virtual table and the data table, by linking the corresponding fields:

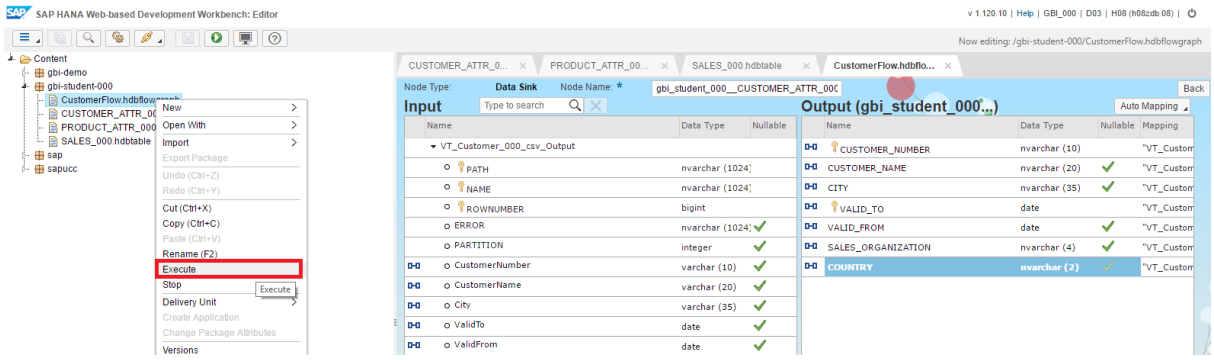


Save the flowgraph model, using the Save button.

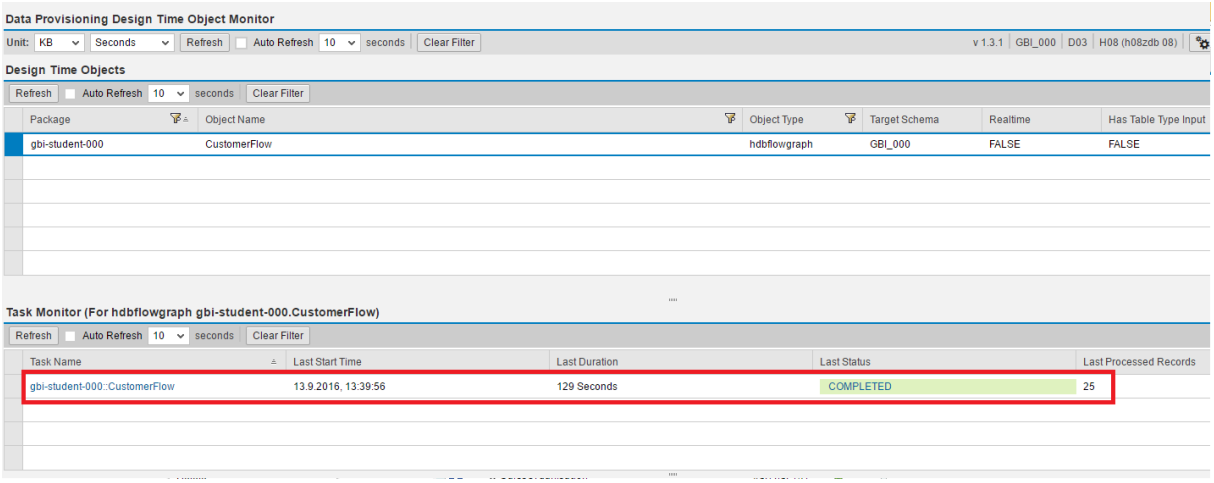
SAP University Alliances

6. Execute the flowgraph model to load the customer data

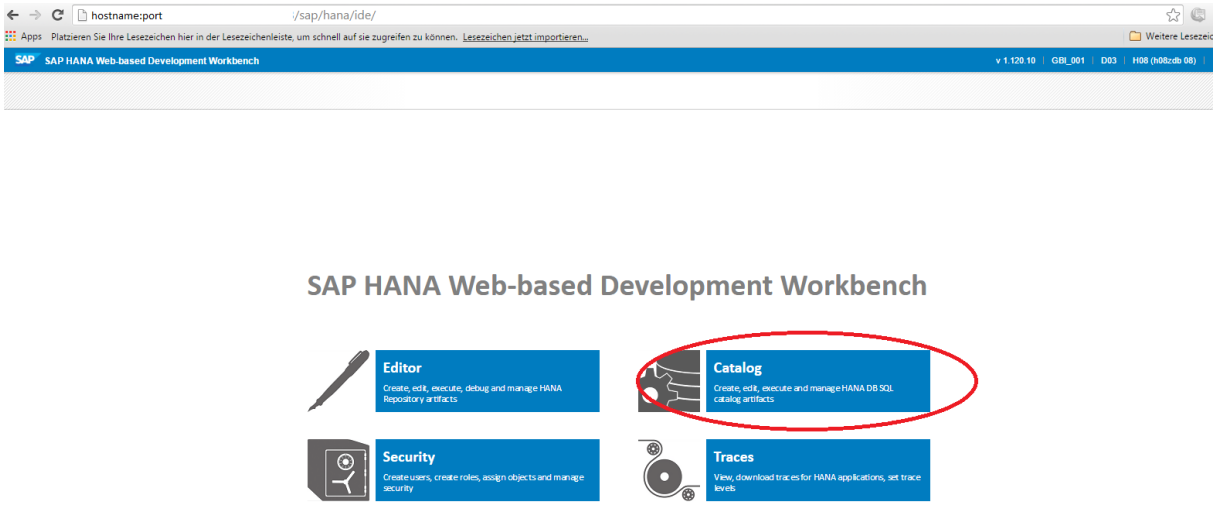
Execute the model using the context menu:



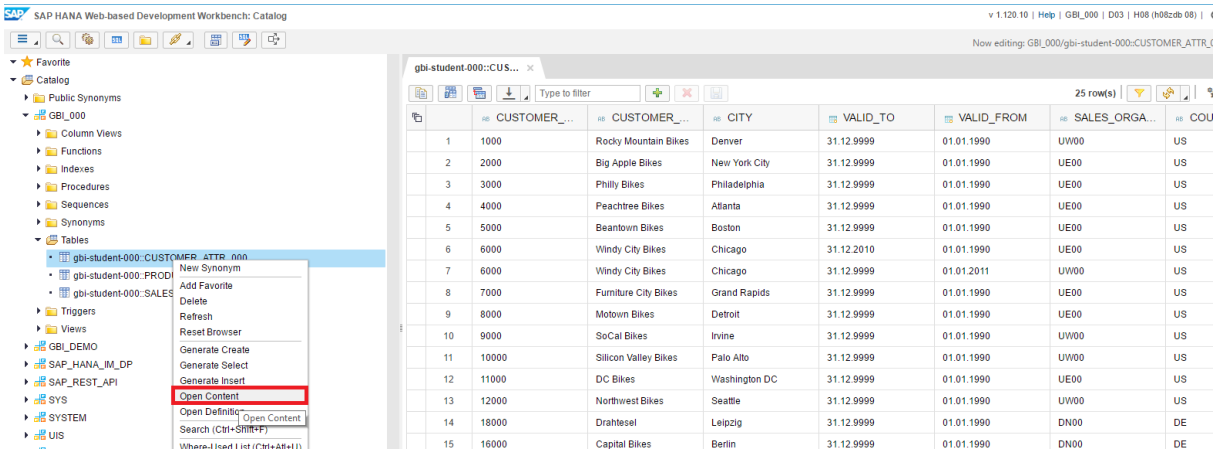
Check the status of the data load in the *monitoring console:*



Switch to the *Catalog* perspective and view the content of the table CUSTOMER_ATTR_**XXX**:



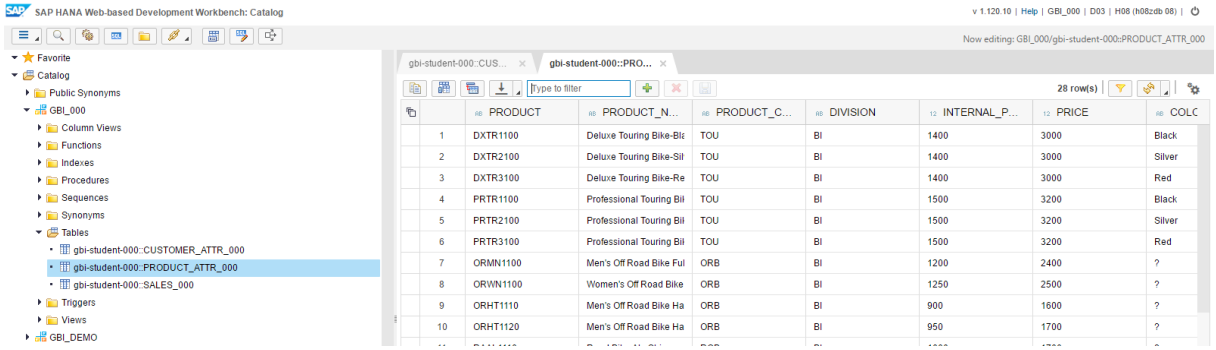
SAP University Alliances



7. Create a flowgraph model for the product data, and load the data. Perform the same steps as you did for the customer data.

Perform the steps 2.4 to 2.6 for the product data.

After performing the steps successfully, the data in table PRODUCT_ATTR_**XXX** should look as shown in the screenshot below.

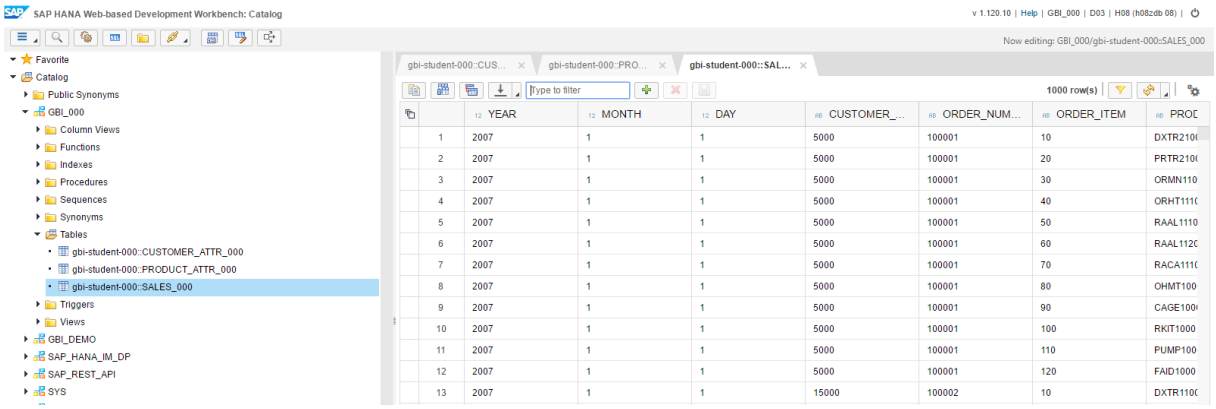


8. Create a flowgraph model for the sales data, and load the data. Perform the same steps as you did for the customer data.

Perform the steps 2.4 to 2.6 for the sales data.

After performing the steps successfully, the data in table SALES_***XXX*** should look as shown in the screenshot below.

SAP University Alliances



	YEAR	MONTH	DAY	CUSTOMER_...	ORDER_NUM...	ORDER_ITEM	PROD
1	2007	1	1	5000	100001	10	DXTR210(
2	2007	1	1	5000	100001	20	PRTR210(
3	2007	1	1	5000	100001	30	ORMN110
4	2007	1	1	5000	100001	40	ORHT111(
5	2007	1	1	5000	100001	50	RAAL1110
6	2007	1	1	5000	100001	60	RAAL112(
7	2007	1	1	5000	100001	70	RACA111(
8	2007	1	1	5000	100001	80	OHMT100
9	2007	1	1	5000	100001	90	CAGE100(
10	2007	1	1	5000	100001	100	RKIT1000
11	2007	1	1	5000	100001	110	PUMP100
12	2007	1	1	5000	100001	120	FAID1000
13	2007	1	1	15000	100002	10	DXTR110(

© SAP SE

EXERCISE

Exercise 2: Explore the demo graph

Exercise Explore the graph viewer. **Time** 15 Min

Task Get used to the graph explorer, visualize the graph and use the filter to get an overview.

Name (Position) Rick Sanchez (Business Analyst)

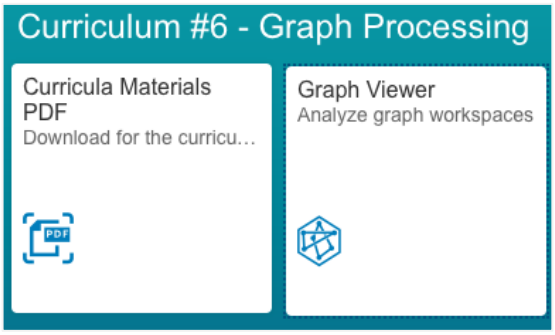
Open the Graph in a browser using the Fiori Launchpad of SAP HANA.

Open the Launchpad

`http://[fqdn]:[http port]/`

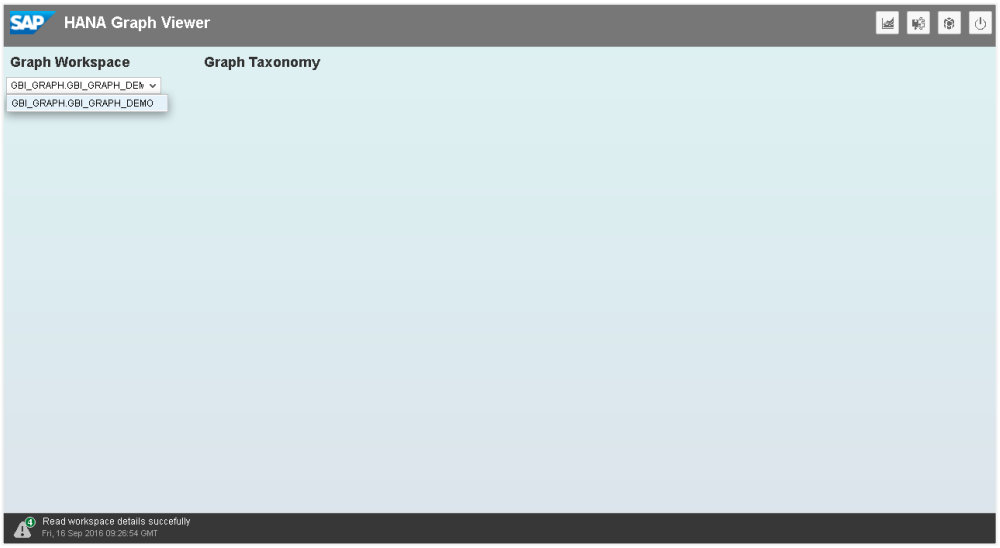
Logon on with your username and your password. All required information will be provided by your instructor.

Fiori Launchpad ► Curriculum #6 - Graph Processing ► Graph Viewer



Select the Graph Workspace *GBI_GRAPH.GBI_GRAPH_DEMO* in the drop-down menu.

GBI_GRAPH_DEMO

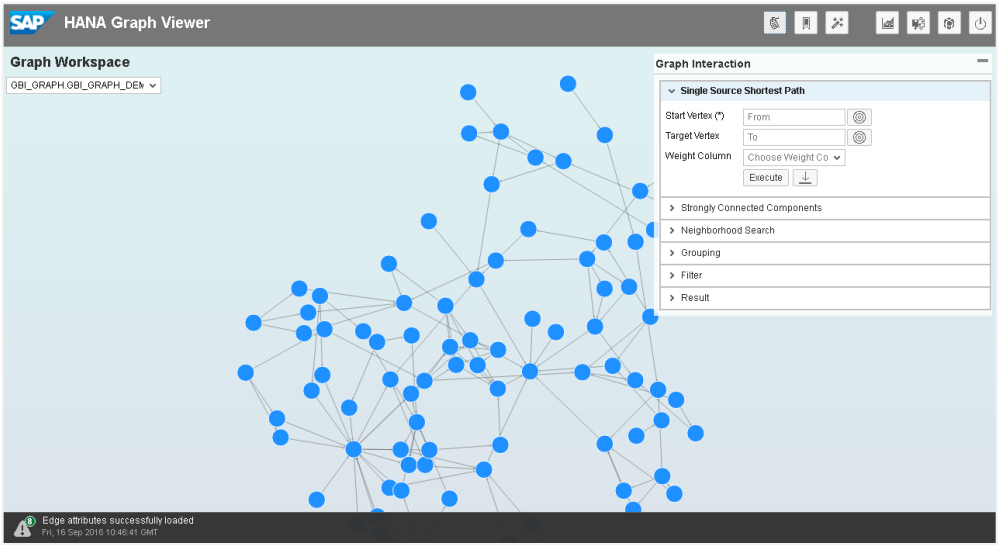


Visualize the graph by clicking on the *Visualize Graph* button in the

Visualize Graph

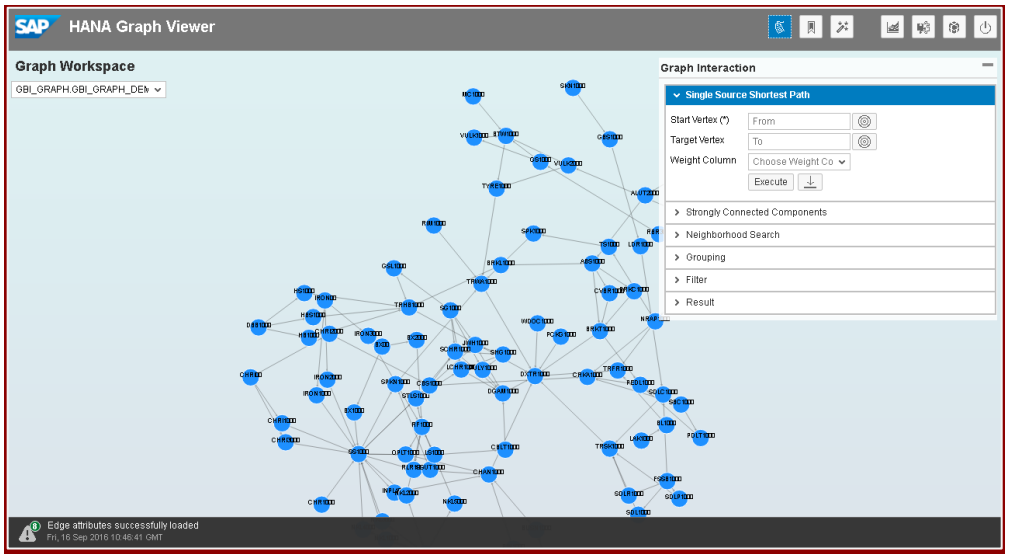
upper right corner.

The graph is loaded and visualized, including all edges and vertices.



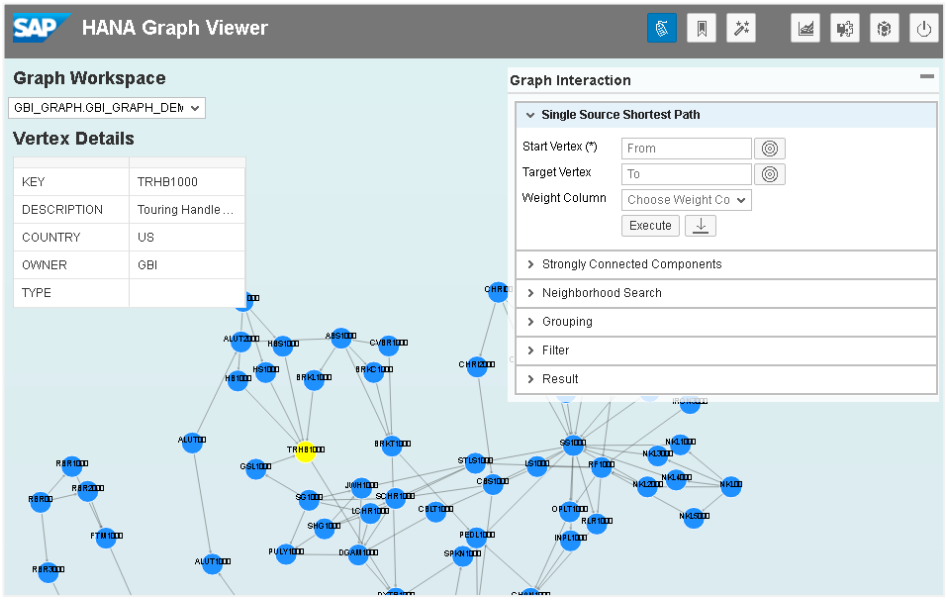
Click on the *Toggle Vertices Key* button  to show a label inside the vertices. The labels shown are the keys of the vertices.

Toggle Vertices Key



Additional data about the vertices can be shown by clicking on a vertex. This will not only mark the vertex but also load a pop-up on the left hand side that shows all information about it.

Show additional vertex information



You can also select an edge to get additional information:

Show additional edge information

Edge Details	
1. edge	
KEY	15
SOURCE	HS1000
TARGET	TRHB1000
DESCRIPTION	is part of

You can also move the graph by clicking and dragging it.

You can also zoom in and out of the graph by using the mouse wheel.

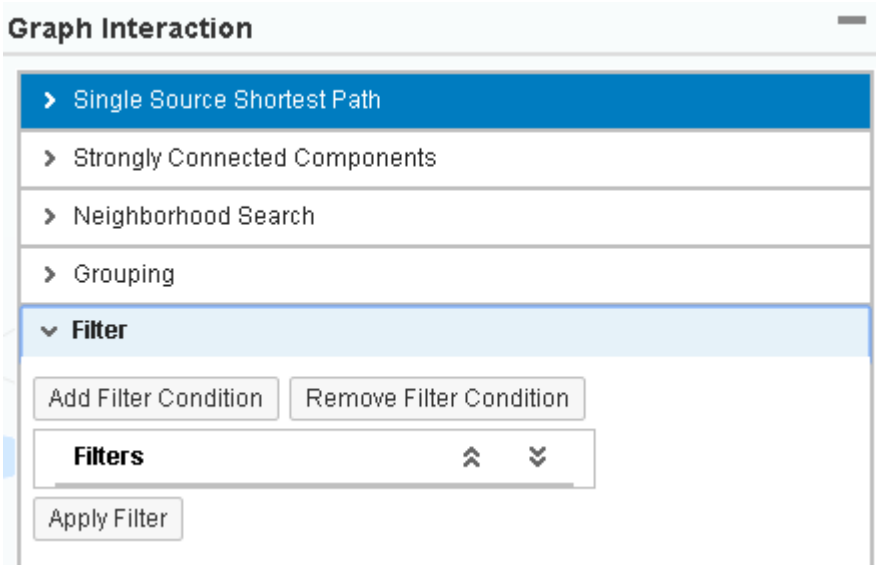
In order to reset the view, you can press the *Center Graph* button in the upper right corner to return to the default graph view.

Center Graph

Next, we want to apply filters to the graph. This may be required if a graph is very large or if only specific vertices / edges are of interest.

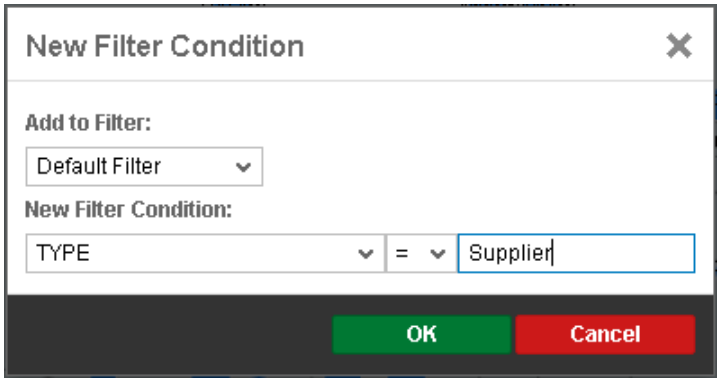
Open the *Filter* by clicking the Filter tab (see screenshot on the next page).

Filter



Click on the *Add Filter Condition* button Add Filter Condition.

Add Filter Condition

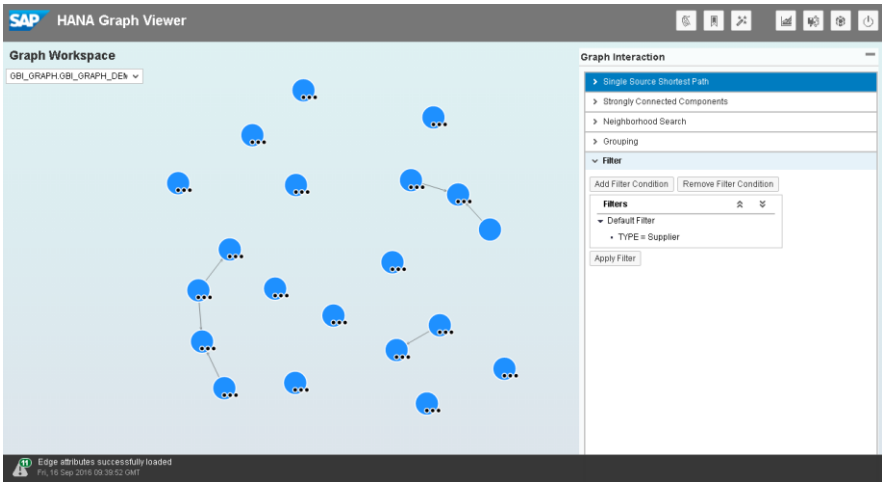


If not already selected, select *TYPE*, = (equals) and enter *Supplier*.

TYPE, =, Supplier

Click *OK* OK. This will add a filter condition. You can also stack more than one filter condition. Click the *Apply Filter* Apply Filter button to apply the filter to the graph.

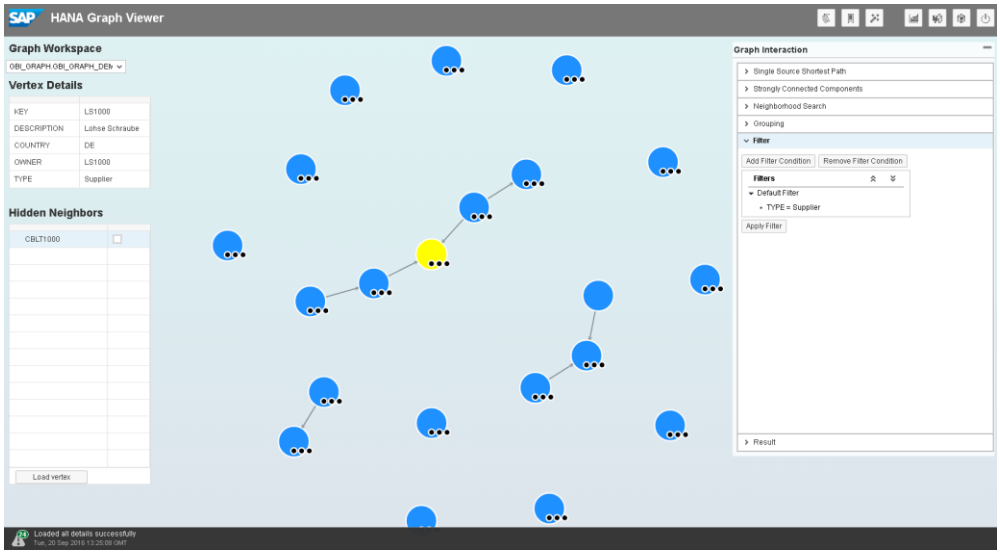
Apply Filter



Now, you will see all suppliers within the graph.

© SAP SE

EXERCISE



Click any vertex to expose details about it.

EXERCISE

Exercise 3: Single Shortest Path

Exercise Use the single shortest path algorithm. **Time** 10 Min

Task GBI wants to analyze their production pathway in order to check their sustainability. Use the single shortest path algorithm starting with the rubber and ending with the finished tire to analyze their sustainability.

Name (Position) Rick Sanchez (Business Analyst)

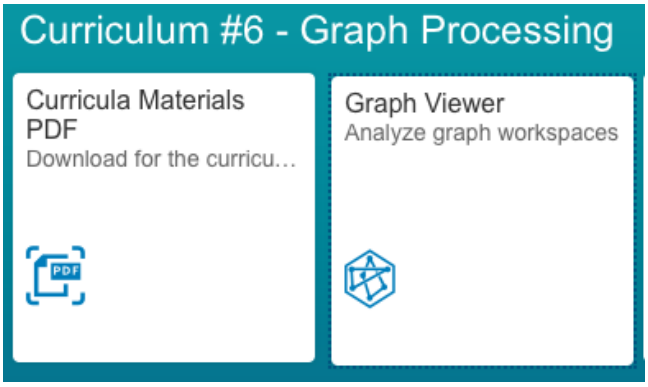
Open the Graph Taxonomy in a browser using the Fiori Launchpad of SAP HANA.

`http://[fqdn]:[http port]/`

Open the Launchpad

Logon on with your username and your password. All required information will be provided by your instructor.

Fiori Launchpad ► Curriculum #6 - Graph Processing ► Graph Viewer



Select the Graph Workspace *GBI_GRAPH.GBI_GRAPH_DEMO* in the drop-down menu.

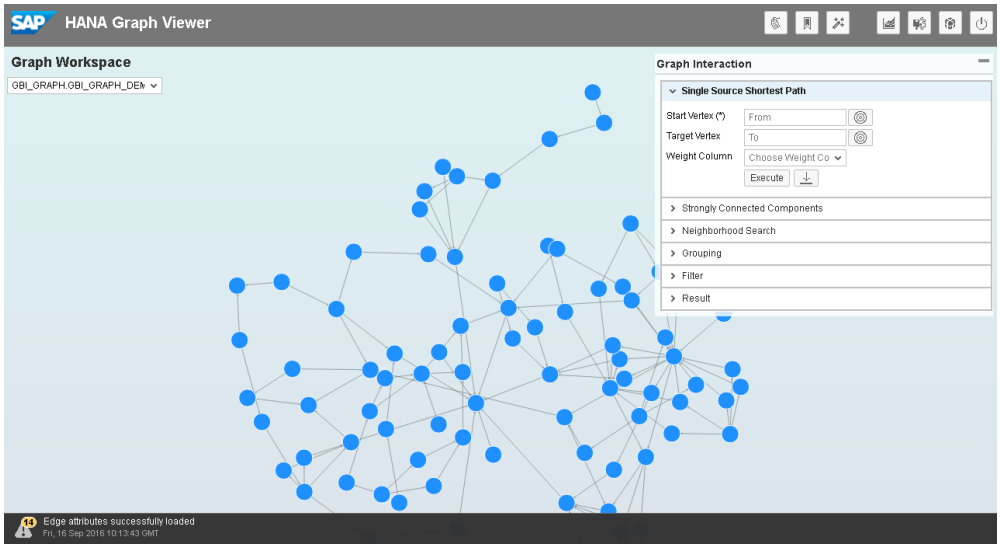
GBI_GRAPH_DEMO



© SAP SE

Open visualization

Open the graph visualization by clicking on the *Visualize Graph* button in the upper right corner.



If not opened, open the Graph Interaction tab on the right by clicking .

Single Source Shortest Path Algorithm

The Single Source Shortest Path algorithm finds the shortest path between one vertex and all other vertices of the graph. Distances between vertices are calculated using the number of vertices between source and target or using an optional edge weight (attribute).

In the tab Single Source Shortest Path enter the following data:

RB00

Start Vertex: *RBR00*

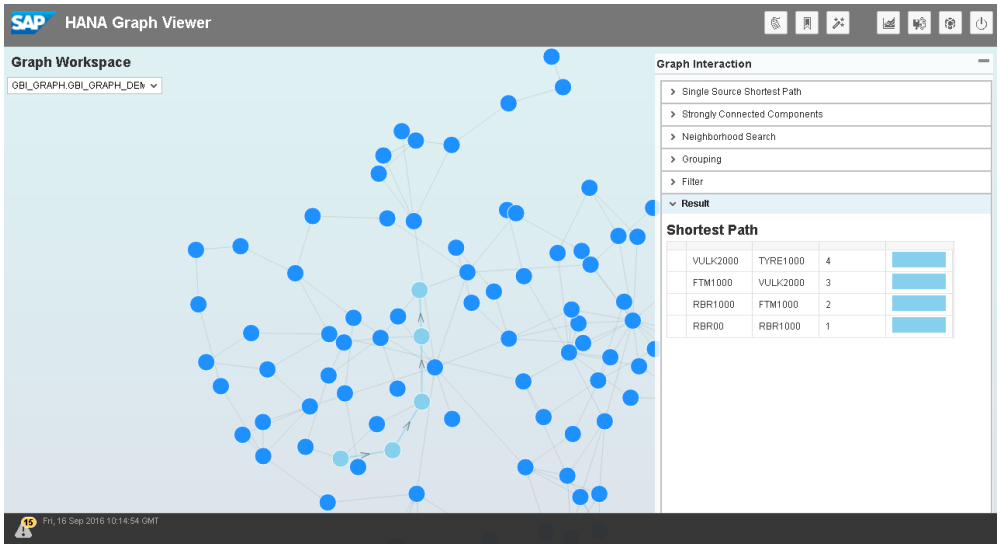
TYRE1000

Target Vertex: *TYRE1000*



You can also click on the target button once and then click on the correct vertex. This will automatically enter the vertex key into the field next to the button.

Click on the *Execute* button Execute to execute the algorithm.



The result of the algorithm, the shortest path, is displayed in the graph visualization as well as in the result tab on the right hand side of the viewer. In the result tab, you get a more detailed view of the resulting path.

Click on one of the colored vertices in the result tab to highlight them in the graph.

Click on the *Clear Color button*  to reset the Graph.

Exercise 4: Neighborhood Search

Exercise Find out which components constitute a Touring Seat Kit. **Time** 15 Min

Task GBI wants to check their sustainability of the Touring Seat Kit. Use the neighborhood search to see all components that constitute a Touring Seat Kit.

Name (Position) Rick Sanchez (Business Analyst)

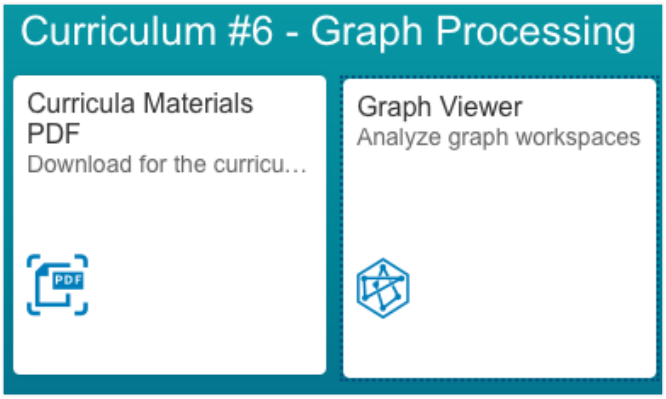
Open the the Graph Viewer in a browser using the Fiori Launchpad of SAP HANA.

Open the Launchpad

`http://[fqdn]:[http port]/`

Logon on with your username and your password. All required information will be provided by your instructor.

Fiori Launchpad ► Curriculum #6 - Graph Processing ► Graph Viewer



© SAP SE

Select the Graph Workspace *GBI_GRAPH.GBI_GRAPH_DEMO* in the drop-down menu.

GBI_GRAPH_DEMO



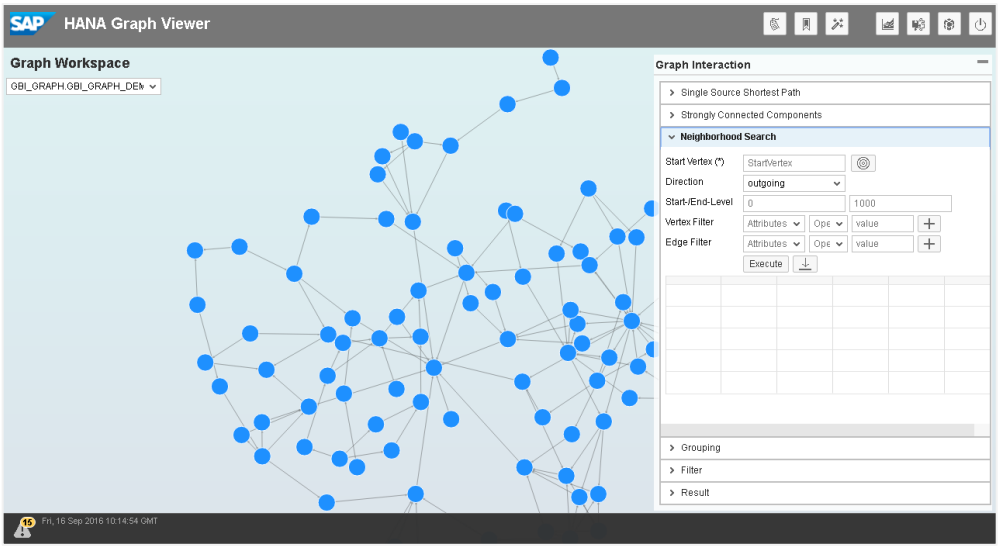
Open the graph visualization by clicking on the *Visualize Graph* button in the upper right corner.

Visualize Graph

If not opened, open the Graph Interaction tab on the right by clicking the expose button and select **Neighborhood Search**.

Expose

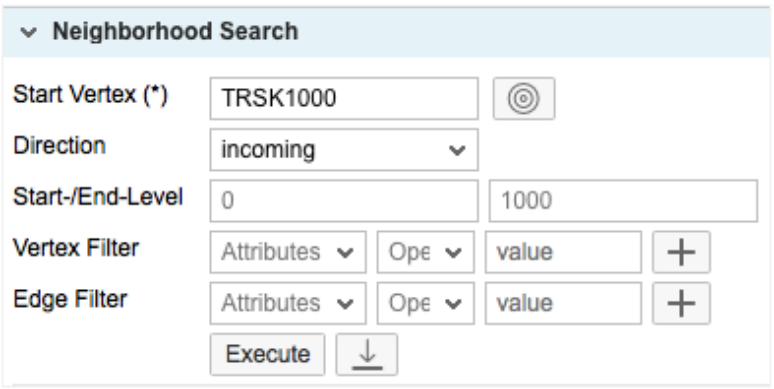
Neighborhood Search



In the Neighborhood Search tab select Start Vertex: *TRSK1000* and Direction *incoming* (see screenshot on the next page). This will search through the neighborhood of the vertex *TRSK1000* looking through all incoming edges recursively. As in the previous algorithm, you can also use the target button to locate the start vertex.

TRSK1000

incoming



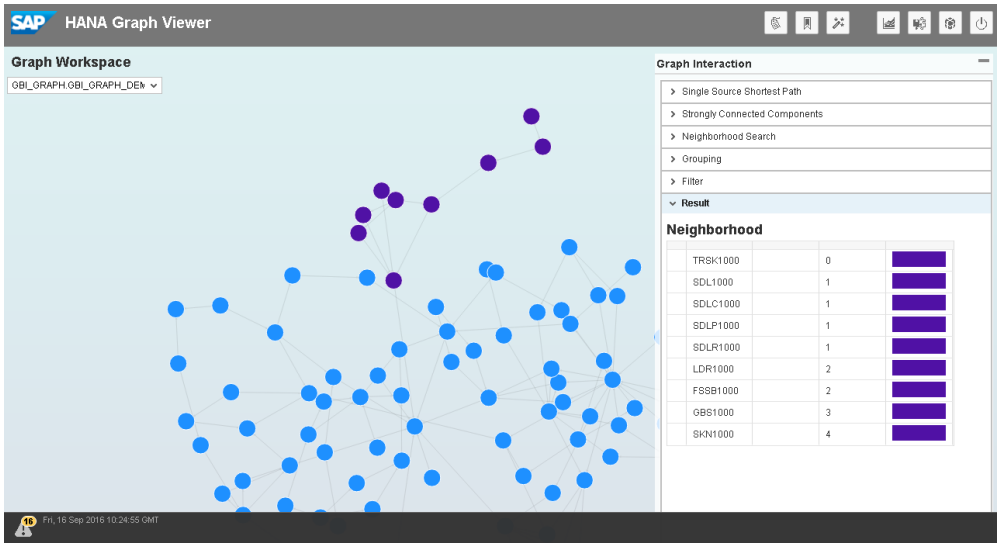
Keep in mind that, if no other parameters are specified, all incoming edges are recursively traversed up to a depth of 1000 (default deep).

Click on the *Execute* button Execute to run the algorithm with the specified parameters.

Execute

The Neighborhood Search algorithm finds all adjacent vertices of a specified vertex within a certain distance, which is called the neighborhood, by traversing the graph. As output, the algorithm delivers a list of vertices including their depth („distance“ from the start vertex).

Neigborhood Search Algorithm



The Graph Viewer will automatically show a result list of vertices including their distance from the start vertex on the right hand side in the result tab. The resulting vertices are also highlighted in color in the graph.

Exercise 5: Pattern Matching

Exercise Use the Pattern-Matching algorithm to determine which parts use resources by certain countries.

Time 10 Min.

Task A volcano eruption in China prevents certain resources from supplying. Find out which components are affected by using the pattern-matching algorithm so GBI can look for alternatives.

Name (Position) Rick Sanchez (Business Analyst)

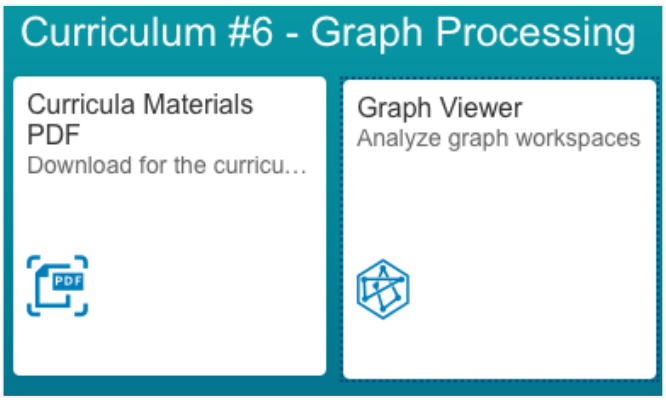
Open the Graph Viewer in a browser using the Fiori Launchpad of SAP HANA.

Open the Launchpad

`http://[fqdn]:[http port]/`

Logon on with your username and your password. All required information will be provided by your instructor.

Fiori Launchpad ► Curriculum #6 - Graph Processing ► Graph Viewer



Select the Graph Workspace *GBI_GRAPH.GBI_GRAPH_DEMO* in the drop-down menu.

GBI_GRAPH_DEMO



Press the *Pattern Editor* button in the top right corner to switch to the pattern-matching editor.

The Pattern-Matching algorithm extracts all sub graphs that match a certain pattern.

Pattern-Matching Algorithm

First, select the vertex *V1* in the middle of the screen. With the *V1* vertex selected add the following operations in the *Graph Pattern Editor* on the right hand side:

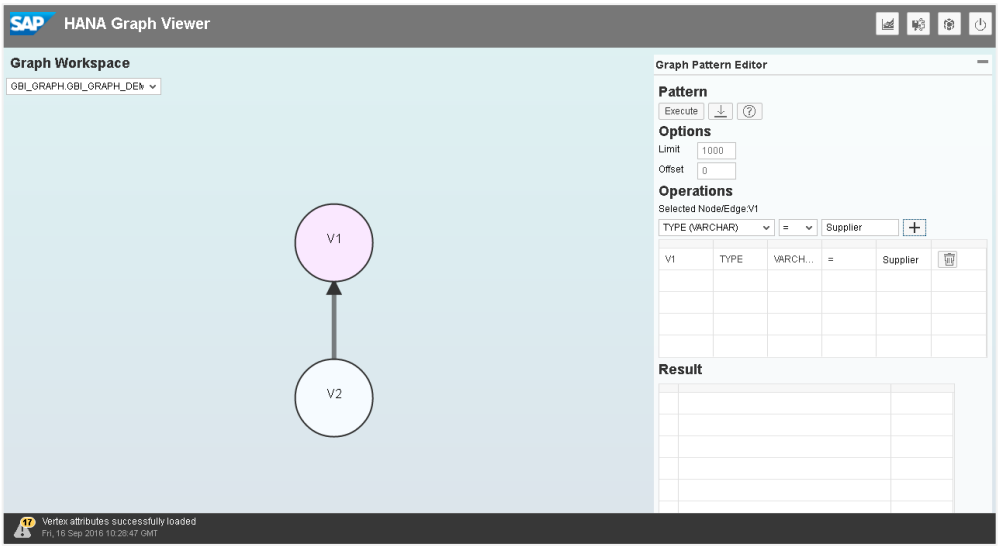
V1

Attributes: *TYPE (VARCHAR)*
Operator: = *(equals)*
Value: *Supplier*

TYPE
=
Supplier

To add the operation, click on the *Add* button +.

Add the first operation



Now select the vertex *V2* and add the following operations:

V2

Attributes: *COUNTRY (VARCHAR)*

COUNTRY

Operator: = *(equals)*

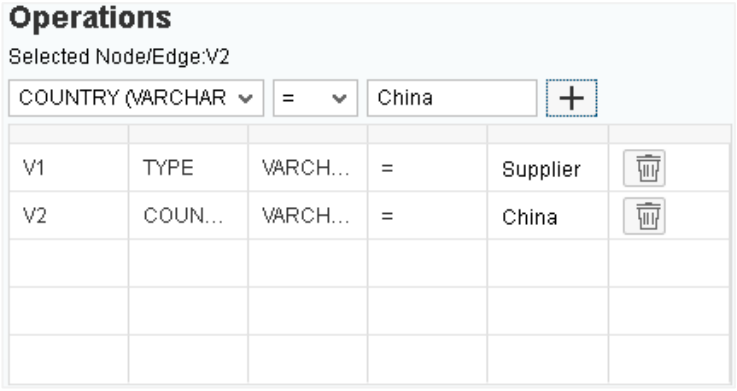
=

Value: *China*

China

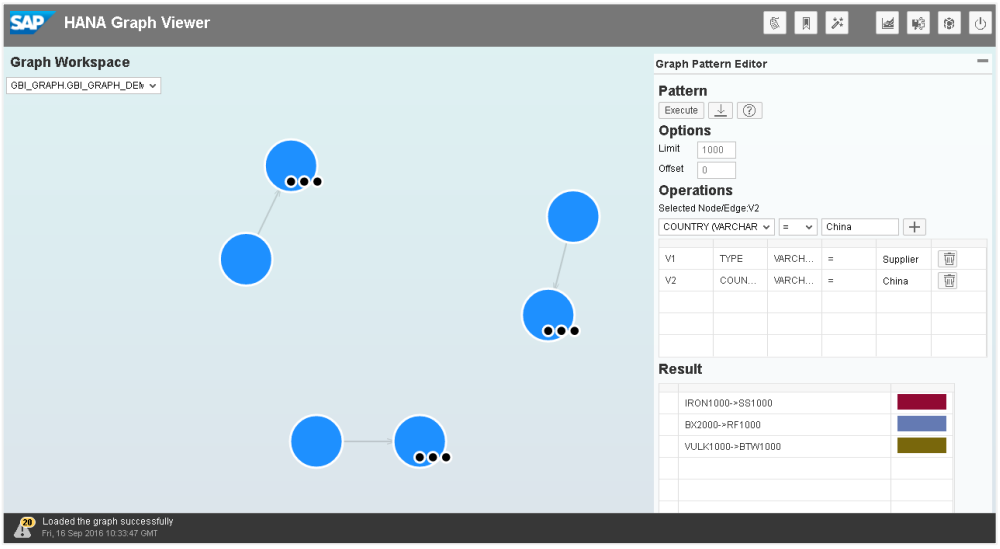
Add the second operation

To add the second operation, click on the *Add* button [+]. Now, your *Operations* window should look like this:



Execute

To run the pattern-matching algorithm, click on the *Execute* button [Execute].



The resulting subgraphs represent all components that are shipped from China. Note that you also get a detailed view of the subgraphs in the result table of the *Graph Pattern Editor* on the right hand side.

Create a simple OData service for a table

Task Before you create complex OData services, we're going to start with a simple ex-ample. In this step, you will create an OData service that exposes data from a database table as a web service to the internet. You're going to retrieve different in-formation from this service.

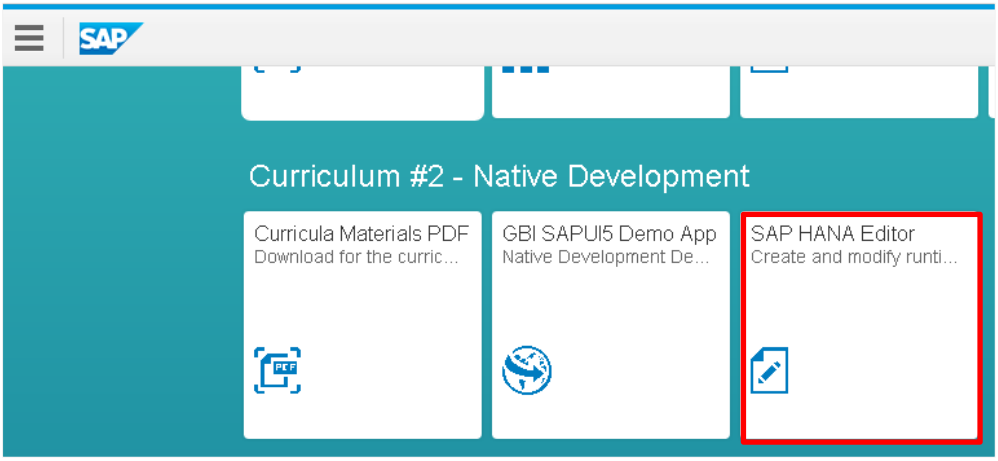
Time 45 min

Short Description Create and consume a simple OData service based on a table.

Name (Position) Mona Falco (Development Specialist)

To open the Web-based Development Workbench Editor, log on to the Fiori Launchpad of the SAP HANA system and follow the menu path:

Fiori Launchpad ► Curriculum #2 - Native Development ► SAP HANA Editor



Open the SAP Fiori Launchpad which is available under

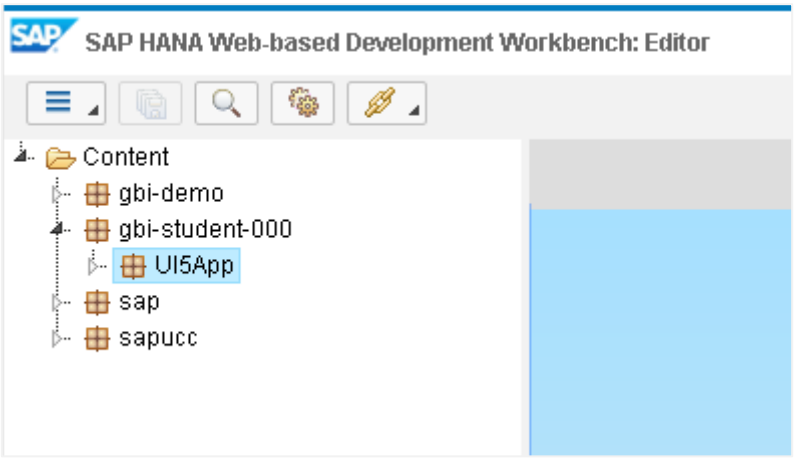
`http://[fqdn]:[http port]/`

Logon on with your username and your password. All required information will be provided by your instructor.

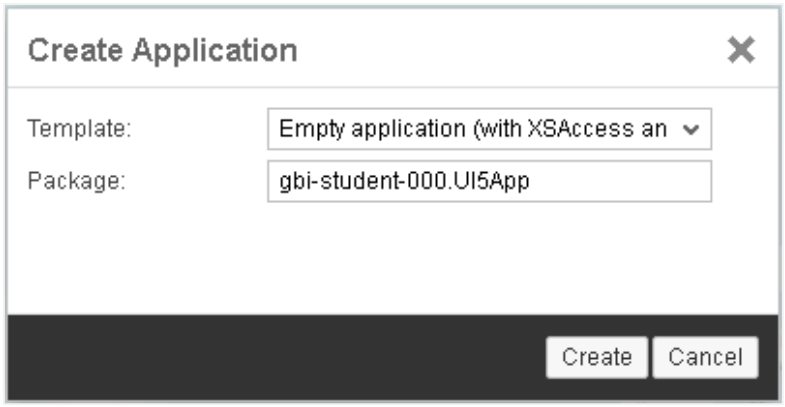
Before we start adding OData services, we need to create a new package where we can build our UI5 application and all of its components (such as the OData services).

Navigate to your package *gbi-student-###* and create a new package under it with the name *UI5App*.

© SAP SE



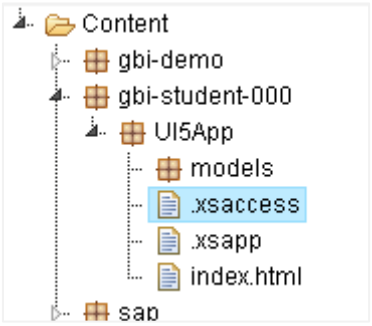
Now we can create the scaffold for our application. Right-click on the newly created package and select *Create Application* from the context menu.



As template, select *Empty application (with XSAccess and XSApp)* and click *Create*. This will create all the basic files required for you application.

Within this package, we need an additional package to store our OData services.

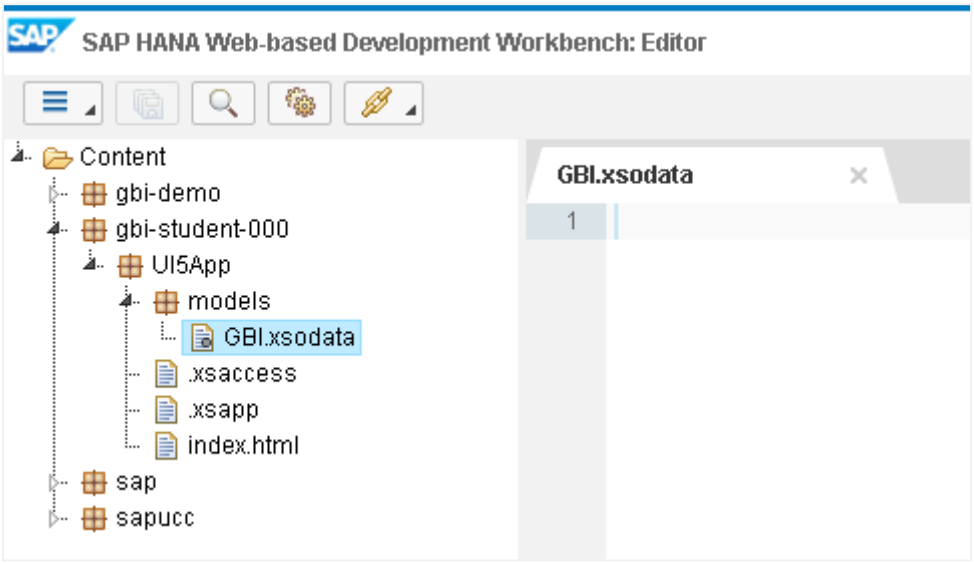
Create a new package *models* under the package *UI5App*.



Now we are ready to build our OData service. Right-click on the package *models* and click on *New* ▶ *File*. As filename, use *GBI.xsodata*. Files with the file extension **.xsodata* are automatically treated as OData services in SAP HANA and the SAP HANA Web-based Development Workbench.

The new file should automatically open and your application structure should look like this:

© SAP SE



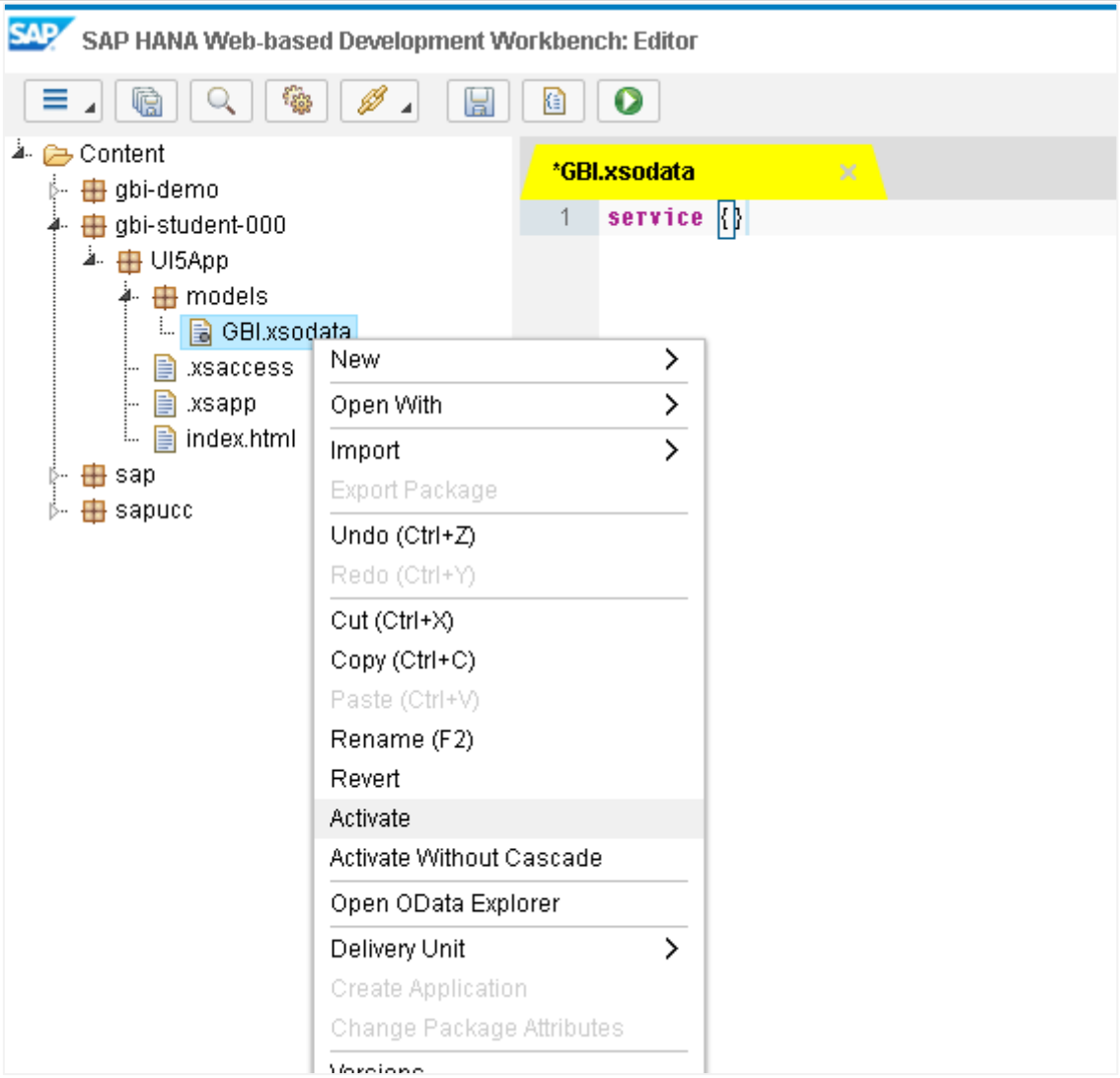
You can see that the new *file GBI.xsodata* has a small, grey circle in its icon. This means that the current version of the file is only available on your PC (respectively in your browser).

Before we push the file to the SAP HANA server, we need to add some content because the Editor will do a syntax check based on the syntax of **.xsodata* files. Files with erroneous syntax are not transferred to the SAP HANA Server.

Add the minimal definition of an XS OData service to the file:

```
service {}
```

To push the file to the SAP HANA server and make it available to everybody, you can either activate the file by right-clicking on it and selecting *Activate* (only available when the file is initially created):



or you can just save files after you have modified them by clicking on the save button in the toolbar or pressing *Ctrl+S* on your keyboard.

Activate the file.

The file is now active on the SAP HANA server but doesn't expose any data as a web service because a proper service definition is missing.

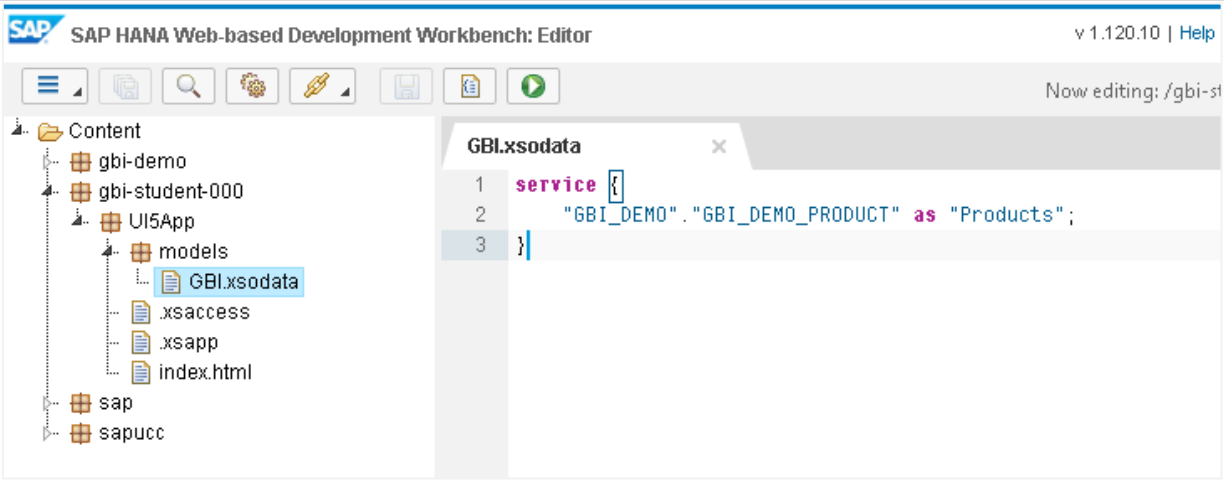
To add a first service, replace the content of *GBI.xsodata* with the following code:

```
service {
      "GBI_DEMO"."GBI_DEMO_PRODUCT" as "Products";
}
```

Save the file.

Basically, this tells the XS engine on the SAP HANA server to create an OData service that exposes the table *GBI_DEMO_PRODUCT* in the schema *GBI_DEMO* to external clients, which makes it possible to request this service over the Internet in a standardized manner to add, modify, change or delete data in that table.

© SAP SE

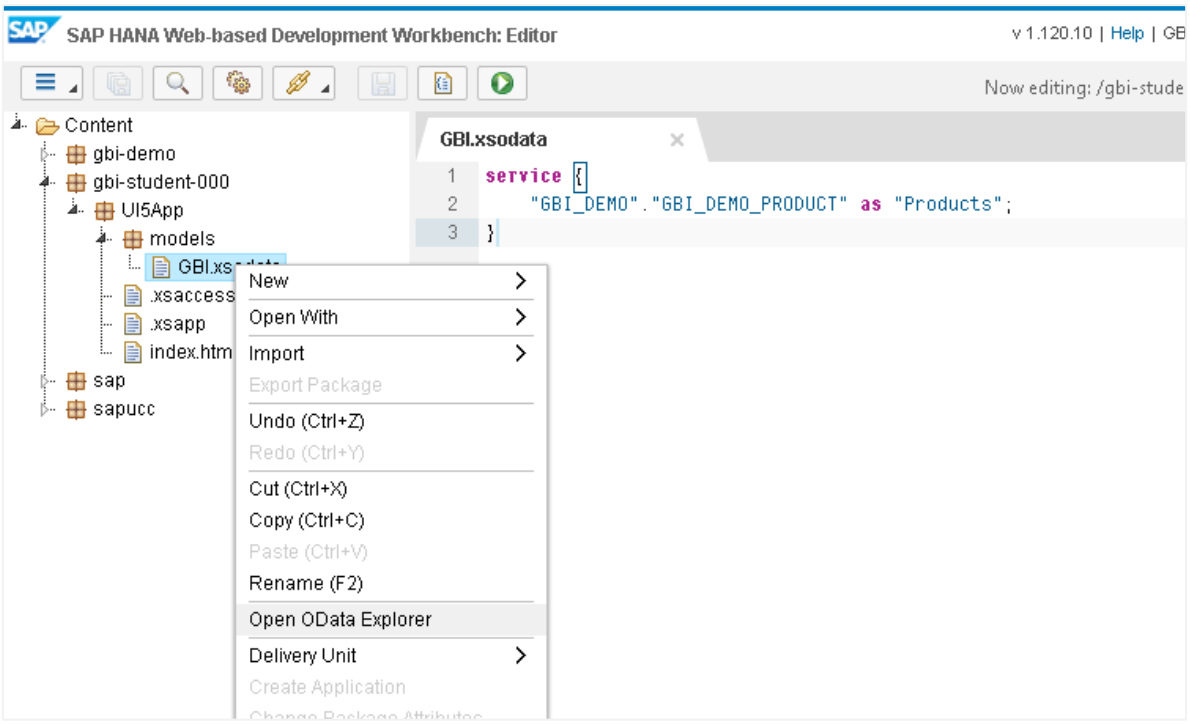


If you successfully finished the SAP HANA Introduction case study, you can use the data that you created yourself instead of the predefined demo content:

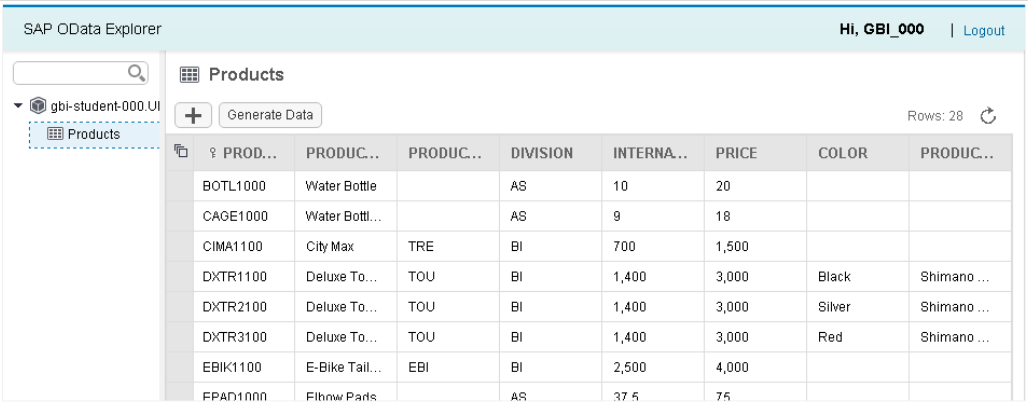
```
service {
      "GBI_###"."gbi-student-###::PRODUCT_ATTR_###" as "Products";
}
```

Remember to replace ### with your ID. Save the file to activate it.

To display the content of the OData service(s) defined in the file, you can open the OData Explorer by right-clicking on the *GBI.xsodata* file and selecting *Open OData Explorer*.



This will open a convenient interface for exploring the data of the OData services.



If you want to see the real, unfiltered output of the services, you can click on the run button in the toolbar, which opens the OData services in the browser just like applications see the services.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<service xmlns:atom="http://www.w3.org/2005/Atom" xmlns:app="http://www.w3.org/2007/app"
 xmlns="http://www.w3.org/2007/app" xml:base="http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-
 000/UI5App/models/GBI.xsodata/">
   <script id="tinyhippos-injected"/>
  ▼<workspace>
     <atom:title>Default</atom:title>
    ▼<collection href="Products">
       <atom:title>Products</atom:title>
     </collection>
   </workspace>
 </service>
```

Note: If you are using a different browser then Chrome, you might need an Add-on or Plug-in to format the XML output of the OData service.

This is the service document describing the OData data model in XML. It shows the service endpoint or item collections that are made available by this service.

To see this service's metadata, append the metadata tag to the URL (add */$metadata* to the URL as shown below):

`http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/$metadata`

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<edmx:Edmx xmlns:edmx="http://schemas.microsoft.com/ado/2007/06/edmx" Version="1.0">
   <script id="tinyhippos-injected"/>
  ▼<edmx:DataServices xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata"
   m:DataServiceVersion="2.0">
    ▼<Schema xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices"
     xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata"
     xmlns="http://schemas.microsoft.com/ado/2008/09/edm" Namespace="gbi-student-000.UI5App.models.GBI">
      ▼<EntityType Name="ProductsType">
        ▼<Key>
           <PropertyRef Name="PRODUCT"/>
         </Key>
         <Property Name="PRODUCT" Type="Edm.String" Nullable="false" MaxLength="8"/>
         <Property Name="PRODUCT_NAME" Type="Edm.String" MaxLength="40"/>
         <Property Name="PRODUCT_CATEGORY" Type="Edm.String" MaxLength="3"/>
         <Property Name="DIVISION" Type="Edm.String" MaxLength="2"/>
         <Property Name="INTERNAL_PRICE" Type="Edm.Decimal" Precision="17" Scale="2"/>
         <Property Name="PRICE" Type="Edm.Decimal" Precision="17" Scale="2"/>
         <Property Name="COLOR" Type="Edm.String" MaxLength="10"/>
         <Property Name="PRODUCT_GROUP" Type="Edm.String" MaxLength="20"/>
       </EntityType>
      ▼<EntityContainer Name="GBI" m:IsDefaultEntityContainer="true">
         <EntitySet Name="Products" EntityType="gbi-student-000.UI5App.models.GBI.ProductsType"/>
       </EntityContainer>
     </Schema>
   </edmx:DataServices>
 </edmx:Edmx>
```

To access the data in the Products collection, add the endpoint name */Products* to the end of the service URL:

`http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/Products`

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<feed xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices"
 xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata" xmlns="http://www.w3.org/2005/Atom"
 xml:base="http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-000/UI5App/models/GBI.xsodata/">
  <script id="tinyhippos-injected"/>
  <title type="text">Products</title>
  <id>
    http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-000/UI5App/models/GBI.xsodata/Products
  </id>
  <author>
    <name/>
  </author>
  <link rel="self" title="Products" href="Products"/>
  <entry>
    <id>
      http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-000/UI5App/models/GBI.xsodata/Products('BOTL1000')
    </id>
    <title type="text"/>
    <author>
      <name/>
    </author>
    <link rel="edit" title="Products" href="Products('BOTL1000')"/>
    <category term="gbi-student-000.UI5App.models.GBI.ProductsType"
    scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
    <content type="application/xml">
      <m:properties>
        <d:PRODUCT m:type="Edm.String">BOTL1000</d:PRODUCT>
        <d:PRODUCT_NAME m:type="Edm.String">Water Bottle</d:PRODUCT_NAME>
        <d:PRODUCT_CATEGORY m:type="Edm.String" m:null="true"/>
        <d:DIVISION m:type="Edm.String">AS</d:DIVISION>
        <d:INTERNAL_PRICE m:type="Edm.Decimal">10</d:INTERNAL_PRICE>
        <d:PRICE m:type="Edm.Decimal">20</d:PRICE>
        <d:COLOR m:type="Edm.String" m:null="true"/>
        <d:PRODUCT_GROUP m:type="Edm.String" m:null="true"/>
      </m:properties>
    </content>
  </entry>
  <entry>...</entry>
  <entry>...</entry>
```


Review the OData specification and write a URL query that returns the following records from the server:

1. Get the full record of the Product “BOTL1000”

   Solution:

   `http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/Products('BOTL1000')`

2. Get the top 5 records starting from the 10th record

   Solution:

   `http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/Products/?$top=5&$skip=10`

3. Return all records in JSON format instead of XML.

   Solution:

   `http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/Products/?$format=json`

EXERCISE

MVC Hello World App

Exercise Create an MVC Hello World App to learn and understand how SAPUI5 MVC applications are built.

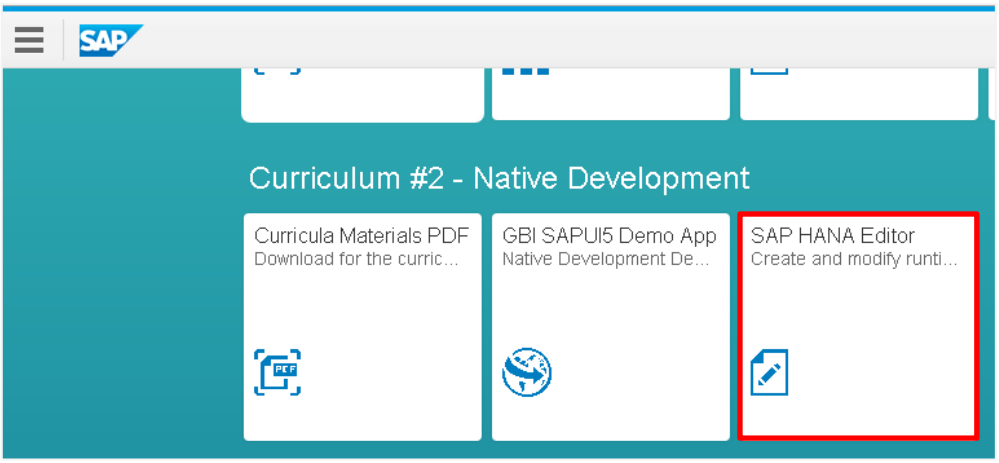
Time 30 Min.

Task In order to learn how MVC application in SAPUI5 work, you will build an MVC version of the Hello World App that you've built in the previous exercise. This includes creating the basic building blocks of the SAPUI5 Apps well as view and controller.

Name (Position) Mona Falco (Development Specialist)

To open the Web-based Development Workbench Editor, logon to the Fiori Launchpad of the SAP HANA system and follow the menu path:

Fiori Launchpad ► Curriculum #2 - Native Development ► SAP HANA Editor



Open the SAP Fiori Launchpad which is available under

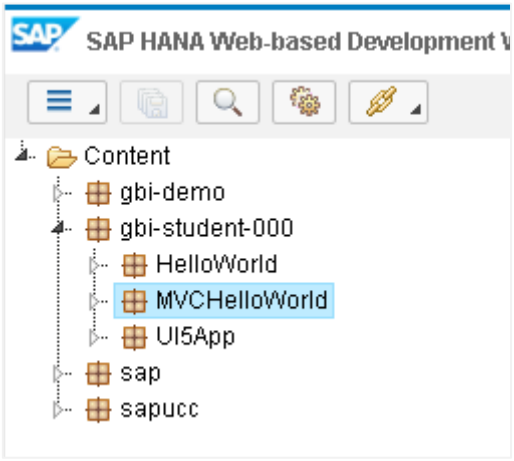
`http://[fqdn]:[http port]/`

Logon with your username and your password. All required information will be provided by your instructor.

Note: Carefully read all comment sections that are included into the source code of this document. These section explain the different controls and functions that are used throughout this document.

First, right-click on you package *gbi-student-###*, select *New* ► *Package* and create a new package with the name *MVCHelloWorld*.

© SAP SE

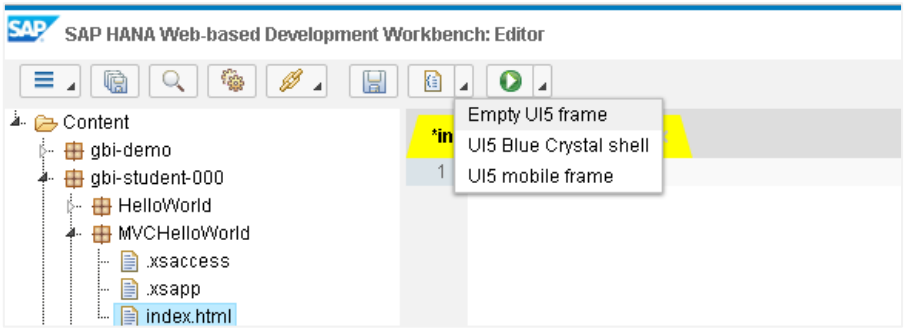


As for the *SAPUI5 Hello World* App or the OData services, we need some basic files for the App. Right-click on the package *MVCHelloWorld* and select *Create Application*.
Select *Empty application (with XSAccess and XSApp)* and click *Create*. This will create the *.xsaccess*, the *.xsapp* and the *index.html* file. The index.html file should open automatically. If not, open it.

index.html

```
1  <!DOCTYPE HTML>
2
3  <html lang="en">
4
5  <head>
6
7  <meta http-equiv="content-type" content="text/html; charset=utf-8">
8
9  <title>Title Goes Here</title>
10
11 </head>
12
13
14 <body>
15 <p>This is my web page</p>
16 </body>
17
18 </html>
```

The *index.html* only contains some basic HTML code, but we need the SAPUI5 bootstrapping and some other functions within it. Select all of the content of the *index.html* and remove it so that the file is empty.
Now click on the *Insert Snippet* button and select *Empty UI5 frame*.



This will create a scaffold that we can easily modify.

First, we would like to change the title of the App. Change the <*title*> tag from *Hello World* to *MVC Hello World*.
Next, we would like to use the *sap.m* library instead of the *sap.ui.commons* library. Change the value of *data-sap-ui-libs* accordingly.
After this, we want to create a MVC application, so the content of the second <*script*> tag is not required. Remove it so that the second <*script*> tag is empty.
Last, we want to modify the id of the <*div*> tag that will hold all of our views later on. Change the id from *content* to *uiArea*. Save the file.
Your file *index.html* should now look like this:

index.html

```
<!DOCTYPE html>
<html><head>
    <meta http-equiv='X-UA-Compatible' content='IE=edge' />
    <title>MVC Hello World</title>

    <script id='sap-ui-bootstrap' type='text/javascript'
        src='/sap/ui5/1/resources/sap-ui-core.js'
        data-sap-ui-theme='sap_bluecrystal'
        data-sap-ui-libs='sap.m'></script>

    <script>

    </script>

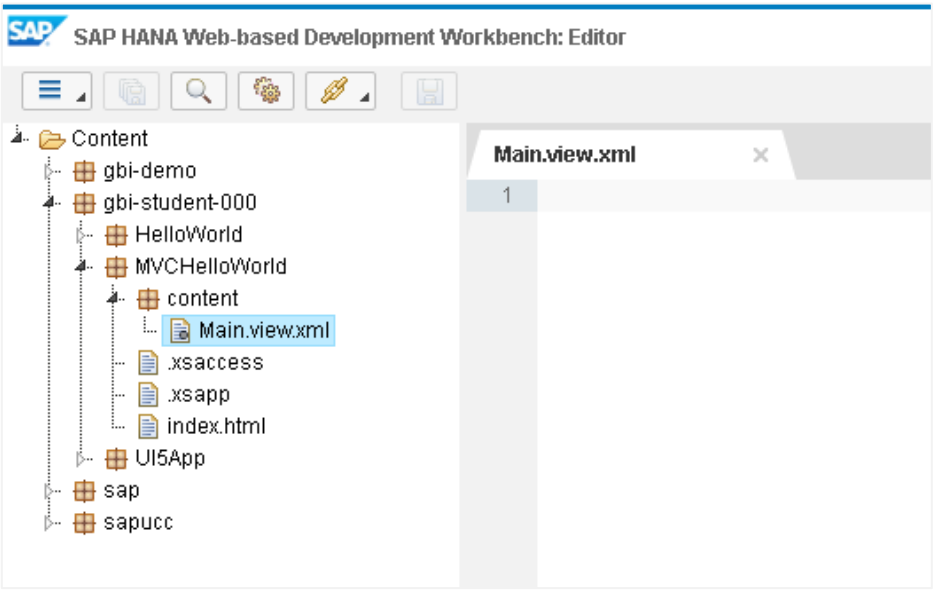
</head>
<body class='sapUiBody'>
    <div id='uiArea'></div>
</body>
</html>
```

Next, we need a new package in *MVCHelloWorld* that will hold all of the controllers and views that we're going to create. Right-click on *MVCHelloWorld*, select *New* ► *Package* and create a new package *content*.
Within this package, we're going to create the view for our landing page of our application.

In the package *content*, create a new file *Main.view.xml* by right-clicking on the package and selecting *New* ► *File*.

The new file should open in the Editor automatically:

© SAP SE



In this case study, we're using XML based views. If you build your own application, you can also use one of the other languages that are supported by SAPUI5 for views, e.g. JavaScript, JSON or HTML.
Now, we're adding the basic structure, which is required by SAPUI5, to the view.

Add the following code to the *Main.view.xml*:

```
<!-- this is a comment. in the next tag, the xmlns tags define namespaces, the
name after of the equality signs being the name of the namespace. In the code
below, we can use this name instead of the full path -->
<mvc:View controllerName="content.Main"
      xmlns:mvc="sap.ui.core.mvc"
      xmlns="sap.m">

      <!-- view content goes here -->

</mvc:View>
```

Save the file.The file is now a valid view, but empty. We're adding a small label so that we're going to see something when we've built our first version of the landing page.

Add the following code below the `<!-- view content goes here -->` comment tag:

```
<!-- this control creates a button that calls the function
      changeText when pressed -->
<Button id="mybutton" text="Hello World" press="changeText"/>
```

Save the file.
Your file *Main.view.xml* should now look like this:

© SAP SE

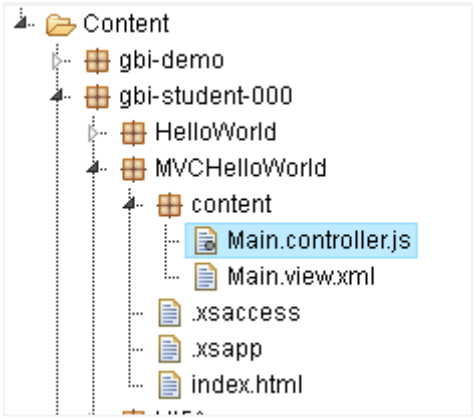
Main.view.xml

```
<!-- this is a comment. in the next tag, the xmlns tags define namespaces,
    the name after of the equality signs being the name of the namespace.
    In the code below, we can use this name instead of the full path -->
<mvc:View controllerName="content.Main"
    xmlns:mvc="sap.ui.core.mvc"
    xmlns="sap.m">

    <!-- view content goes here -->
    <!-- this control creates a button that calls the function
        changeText when pressed -->
    <Button id="mybutton" text="Hello World" press="changeText"/>

</mvc:View>
```

Now we have a simple view that shows us a button. For our application to work we're still missing two things: a controller for the view that we've created and we need to modify the index.html to route the application to our landing page.
First, we are creating a controller. Create the file *Main.controller.js* in the package *content*.



Add the following code to the empty file:

```
// the controller for the view Main
sap.ui.controller("content.Main", {

      // controller logic goes here

});
```

This defines a minimal controller for the view *Main*. In order to manipulate content later on, we have to add the standard functions to it.
Add the following code after the `// controller logic goes here` comment tag:

```
onInit: function() {
      // this function is called when the view is instantiated.
      // Used to modify the view before displaying
},

onExit: function() {
      // this function is called when the view is destroyed.
      // Used for clean-up activities
},

onAfterRendering: function() {
      // this function is called when the view has been rendered.
      // Used for post-rendering manipulation of the HTML code
},
```

```
onBeforeRendering: function() {
	// this function is called before the view is re-rendered
	// (not before first rendering)
}
```

Save the file. Your *Main.controller.js* should now look like this:

Main.view.xml | **Main.controller.js**

```
1  // the controller for the view Main
2  sap.ui.controller("content.Main", {
3
4      // controller logic goes here
5
6      onInit: function() {
7          // this function is called when the view is instantiated.
8          // Used to modify the view before displaying
9      },
10
11     onExit: function() {
12         // this function is called when the view is destroyed.
13         // Used for clean-up activities
14     },
15
16     onAfterRendering: function() {
17         // this function is called when the view has been rendered.
18         // Used for post-rendering manipulation of the HTML code
19     },
20
21     onBeforeRendering: function() {
22         // this function is called before the view is re-rendered
23         // (not before first rendering)
24     }
25 });
26
```

At this point, we have a fully functioning view/controller pair, but the function *changeText* that is called when the button in the view is pressed, is not yet implemented. In addition to the four standard function, add another function to the controller:

```
// custom function that is called when the button is clicked
// changes the text of the button that fires the event
changeText : function(oEvent) {
	// get the button control from the event
	var myButton = oEvent.getSource();
	// change the button's text
	myButton.setText("Clicked!");
}
```

Do not forget to add a comma after the function *onBeforeRendering*!
Save the file. Your controller should now look like this:

Main.controller.js

```
1  //To use a javascript controller its name must end with .controller.js
2  sap.ui.controller("##controler name here", {
3
4      onInit: function() {},
8
9      onExit: function() {},
13
14     onAfterRendering: function() {},
18
19     onBeforeRendering: function() {
20         // this function is called before the view is re-rendered
21         // (not before first rendering)
22     },
23
24     // custom function that is called when the button is clicked
25     // changes the text of the button that fires the event
26     changeText : function(oEvent) {
27         // get the button control from the event
28         var myButton = oEvent.getSource();
29         // change the button's text
30         myButton.setText("Clicked!");
31     }
32
33 });
```

Note: You can collapse functions and code sections by clicking on the little triangle on the left-hand side:



Now, we have to call our view from the *index.html*, which is the entry point for our application.

Open the *index.html* in the package *MVCHelloWorld* and add the following code to the second, empty <*script*> tag:

```
// tells SAPUI5 where to look for resources
sap.ui.localResources("content");
// creates an instance of our view
content_view = sap.ui.view({
      // unique identifier for this instance
      id:"idmain1",
      // location and name of the view
      viewName:"content.Main",
      // type of the view, i.e. XML, JavaScript, JSON...
      type:sap.ui.core.mvc.ViewType.XML
});
// places the output of our view on this page
content_view.placeAt("uiArea");
```

Save the file. Your *index.html* should now look like this:

index.html

```
<!DOCTYPE html>
<html><head>
    <meta http-equiv='X-UA-Compatible' content='IE=edge' />
    <title>MVC Hello World</title>

    <script id='sap-ui-bootstrap' type='text/javascript'
        src='/sap/ui5/1/resources/sap-ui-core.js'
        data-sap-ui-theme='sap_bluecrystal'
        data-sap-ui-libs='sap.m'></script>

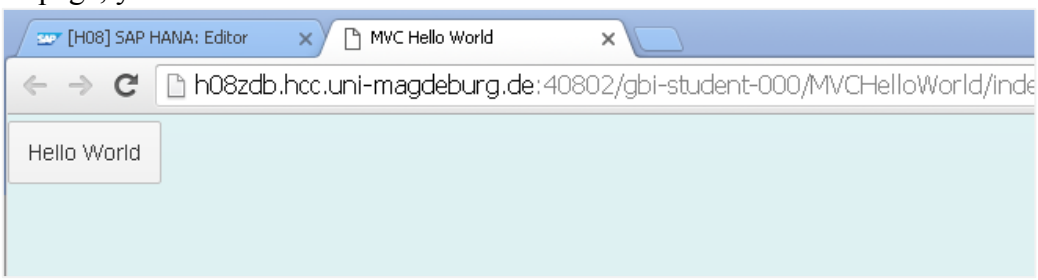
    <script>
        // tells SAPUI5 where to look for resources
        sap.ui.localResources("content");
        // creates an instance of our view
        content_view = sap.ui.view({
            // unique identifier for this instance
            id:"idmain1",
            // location and name of the view
            viewName:"content.Main",
            // type of the view, i.e. XML, JavaScript, JSON...
            type:sap.ui.core.mvc.ViewType.XML
        });
        // places the output of our view on this page
        content_view.placeAt("uiArea");
    </script>

</head>
<body class='sapUiBody'>
    <div id='uiArea'></div>
</body>
</html>
```

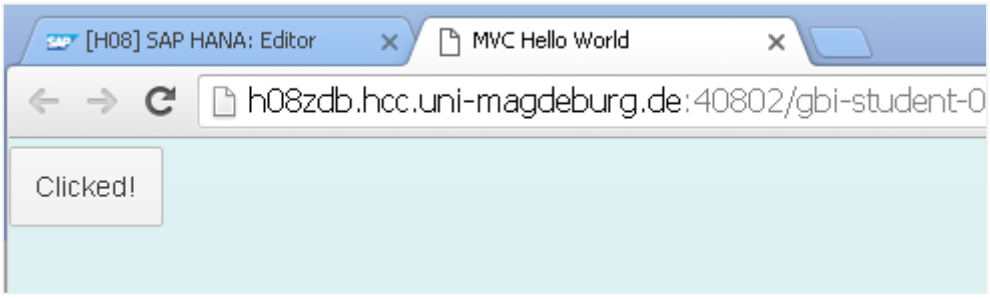
Now click on the run button in the toolbar or open the following URL:

`http://[fqdn]:[http port]/gbi-student-###/MVCHelloWorld/index.html`

On the webpage, you can see the label that we've defined in our view:



If you now click on the button, the function changeText() in the controller will be called and the text will change:



EXERCISE

You've successfully created an SAPUI5 MVC application!



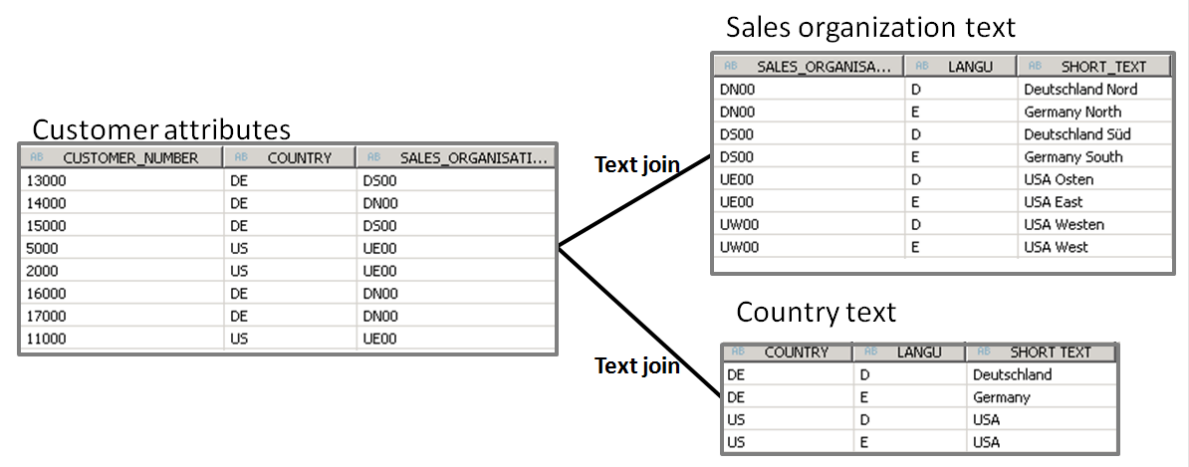
© SAP SE

SAP University Alliances

Step 3: Create a calculation view for customer data

Your previously created customer table includes various attributes that are related to customers. For some of these attributes, e.g. country and sales organization, the table includes only keys, but no textual descriptions. Descriptions for these fields are kept in separate text tables.

Example: In the *Customer attributes* table, the field COUNTRY includes only country keys, like "DE" for Germany. The description *Germany* is stored in text table *Country text*.



To merge this data, you have to create a calculation view. This view joins the *customer attributes* table with the *text tables* for country and sales organization, using a text join.

Furthermore, the customer table includes not only current but also historical customer data. Historical customer data has a valid-to-date before 9999-12-31. To suppress any historical data (valid-to-date < 9999-12-31) you set a filter in this view.

This customer data view will serve as customer dimension within the star schema like data model you are going to create in this case study. Therefore you classify the view as *dimension*.

1. **Open the editor perspective in the SAP HANA Web Workbench**

2. **In your package gbi-student-xxx, create a calculation view for customer data**

   Use following details to create the initial view:

   Package name: gbi-student-xxx

   View name: CUSTOMER_DIM_CV_**XXX**

   View lable: Customer **XXX**

   Data category: Dimension

© SAP SE

SAP University Alliances

3. **The view should show only current customers. For current customers, the valid-to date is equal to 9999-12-31. Add the customer table CUSTOMER_ATTR_XXX and a corresponding filter to your view.**

   Add an additional projection node to the workspace.

   Click on the plus sign of the projection to add your customer table CUSTOMER_ATTR_XXX.

   In the mapping section, add all fields as output columns except VALID_FROM.

   In the projection, add a filter for field VALID_TO with following expression: “VALID_TO”= 99991231.

4. **Join the table GBI_DEMO_COUNTRY with your customer data using a text join**

   To join the tables, add a join  to the workspace.

   Use the result of PROJECTION_1, created in the previous step, as input for this node. To do so, drag-and-drop a connection line between the nodes.

   Add the table GBI_DEMO_COUNTRY as data source to the join.

   In the JOIN DEFINITION, join both tables using the field COUNTRY.

   Select TEXT JOIN as join type and GBI_DEMO_COUNTRY.LANGU as language column.

   Add all fields of PROJECTION_1 as output columns.

   Add the field SHORT_TEXT of table GBI_DEMO_COUNTRY as output column and rename it to SHORT_TEXT_COUNTRY.

5. **Add an additional text join for the sales organization text**

   Add a new join  node to the workspace.

   Use the result of JOIN_1 created previously as input for this node.

   Add the table GBI_DEMO_SALESORG as data source.

   Join both tables using the field SALES_ORGANISATION.

   Select TEXT JOIN as join type and GBI_DEMO_SALESORG.LANGU as language column.

   Add all fields of JOIN_1 as output columns except VALID_TO.

   Add the field SHORT_TEXT of table GBI_DEMO_SALESORG as output column and name it SHORT_TEXT_SALESORG.

© SAP SE

SAP University Alliances

6. **Perform the description mapping for the fields COUNTRY and SALES_ORGANISATION**

   Objective of the description mapping is to link a key and text fields for an attribute. To perform the description mapping you need to set a value for the Label column in the *Semantics* node of your view. Proceed as follows:

   Link the node JOIN_1 with the node PROJECTION and add all fields as output columns.

   Click on the node Semantics.

   For field COUNTRY, set to label column to SHORT_TEXT_COUNTRY.

   For field SALES_ORGANISATION, set the label column to SHORT_TEXT_SALESORG.

   Finally, hide the fields SHORT_TEXT_COUNTRY and SHORT_TEXT_SALESORG.

7. **Define the customer number as key field for your calculation view**

   Hint: Key fields are defined in the semantics node of the calculation view.

8. **Save the calculation view**

9. **Run the view to check the contents**

© SAP SE

SAP University Alliances

Step 4: Create a calculation view for product data

You will create a calculation view that includes product data.

The view should show all product attributes except the price.

1. Create a new calculation view for the product data in your package

User following details to create the view:

Package name: gbi-student-xxx

View name: PRODUCT_DIM_CV_*XXX*

View label: Products *XXX*

Data category: Dimension

2. Add the product table as data source

Add the product table PRODUCT_ATTR_XXX as data source to the existing projection node.

Add all fields listed below as output columns.

Configure the field product as key field.

Table	Field	Key
PRODUCT_ATTR_XXX	PRODUCT	X
PRODUCT_ATTR_XXX	PRODUCT_NAME	
PRODUCT_ATTR_XXX	PRODUCT_CATEGORY	
PRODUCT_ATTR_XXX	DIVISION	
PRODUCT_ATTR_XXX	COLOR	
PRODUCT_ATTR_XXX	PRODUCT_GROUP	
PRODUCT_ATTR_XXX	INTERNAL_PRICE	

© SAP SE

SAP University Alliances

3. **Save the calculation view**

4. **Run the view to check the data**

© SAP SE

SAP University Alliances

Step 5: Create a calculation view for your sales data

You want to analyze the sales data by customer and product.

Therefore you create a simple star schema that includes facts from your sales table. The customer data view and the product data view will represent the dimensions in the star schema.

The star schema is shown in the figure below.

To implement such a star schema, you create a calculation view with star join.



1. Create a new calculation view for sales data

Use following details to create the view:

Package name: gbi-student-*XXX*

View name: SALES_CUBE_CV_*XXX*

View lable: Sales *XXX*

Data category: CUBE

With start join: **x**

2. Add the table SALES_XXX as input for the star join, using a projection node

Add all fields to the output structure of the projection except the field DAY. You do not need this detail information.

Link the projection node with the star join node.

3. Add the net revenue as calculated column to the output structure of the projection

Add a new calculated column to the projection (table SALES_XXX), using following details:

Name: NET_REVENUE

SAP University Alliances

Data type : Decimal (Length 17, Scale 2)

Formula: "REVENUE"-"DISCOUNT"

4. Configure the start join node

Join the sales table with your customer view CUSTOMER_DIM_CV_XXX using the field CUSTOMER_NUMBER and a referential join.

Join the sales table with your product view PRODUCT_DIM_CV_XXX using the field PRODUCT_NUMBER and a referential join.

Add all fields as output columns except the fields CUSTOMER_NUMBER and PRODUCT. They are included in the dimensions.

5. Check the data types of the output structure

Check in the *Semantics* node, if all fields of the fact table are assigned to the correct type (attribute or measure). If not, adjust the type.

Name	Type
YEAR	attribute
MONTH	attribute
ORDER_NUMBER	attribute
ORDER_ITEM	attribute
SALES_QUANTITY	measure
UNIT_OF_MEASURE	attribute
REVENUE	measure
CURRENCY	attribute
DISCOUNT	measure
NET_REVENUE	measure

© SAP SE

SAP University Alliances

6. **Save the calculation view**

7. **Run the calculation view and check the data**

8. **Adapt the calculation view in the text editor**

   Remark: This is a workaround to expose the data correctly in SAP Lumira. The step might be obsolete in future releases.

   Close the graphical version of the calculation view.

   Open the calculation view in the text editor.

   Go do the second line and delete the expression *dimensionType="STANDARD"*.

   Save the view.

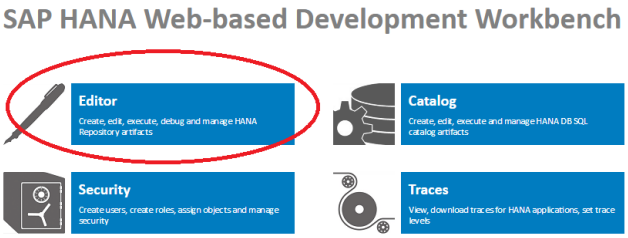
© SAP SE

SAP University Alliances

Step 3: Create a calculation view for customer data

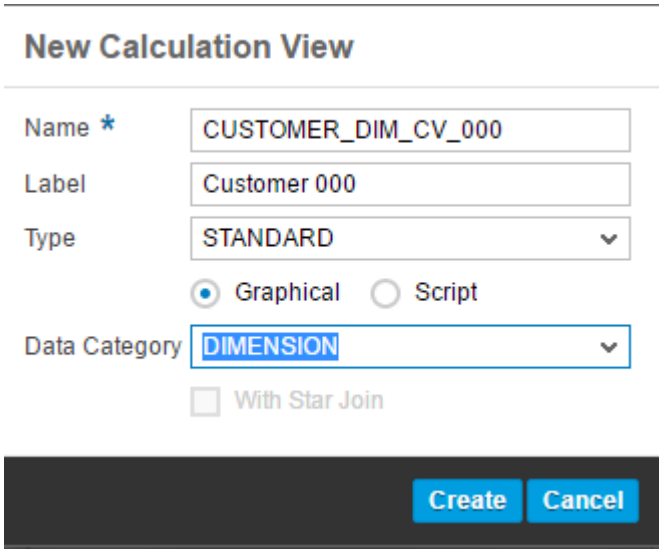
1. Open the editor perspective in the SAP HANA Web Workbench





2. In your package gbi-student-xxx, create a calculation view for customer data

Navigator View → Content → gbi-student-xxx→ Right click → New → Calculation View

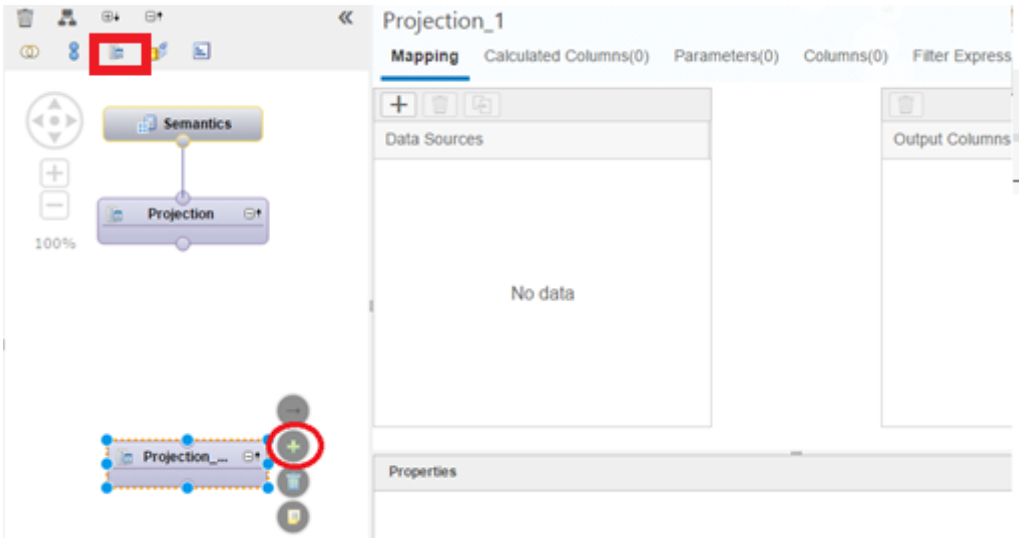


SAP University Alliances

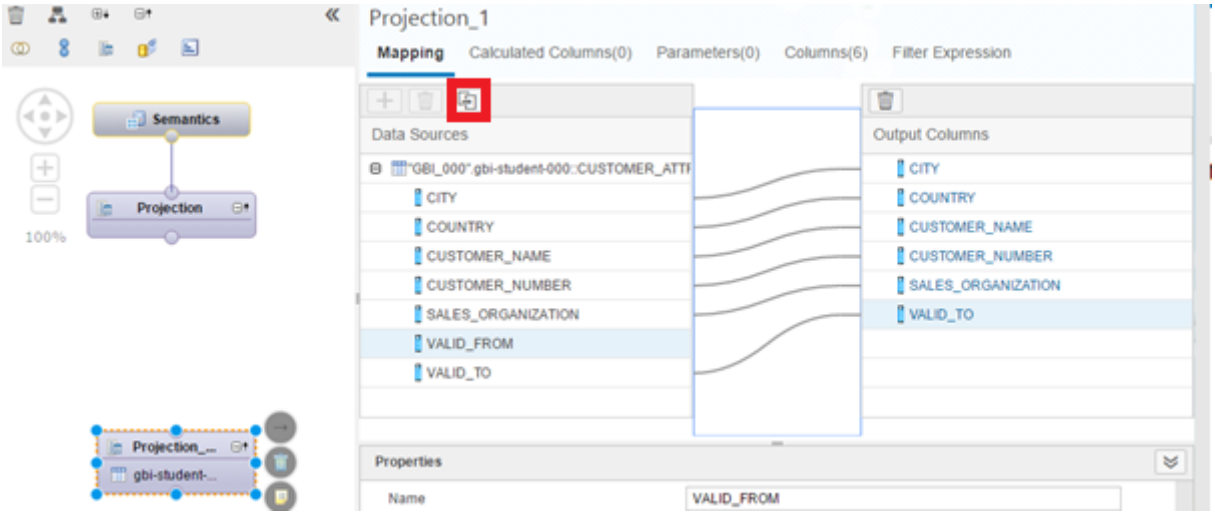
3. The view should show only current customers. For current customers, the valid-to date is equal to 9999-12-31. Add the customer table CUSTOMER_ATTR_XXX and a corresponding filter to your view.

Add an additional projection node to the workspace.

Click on the plus sign of the projection to add your customer table CUSTOMER_ATTR_XXX.

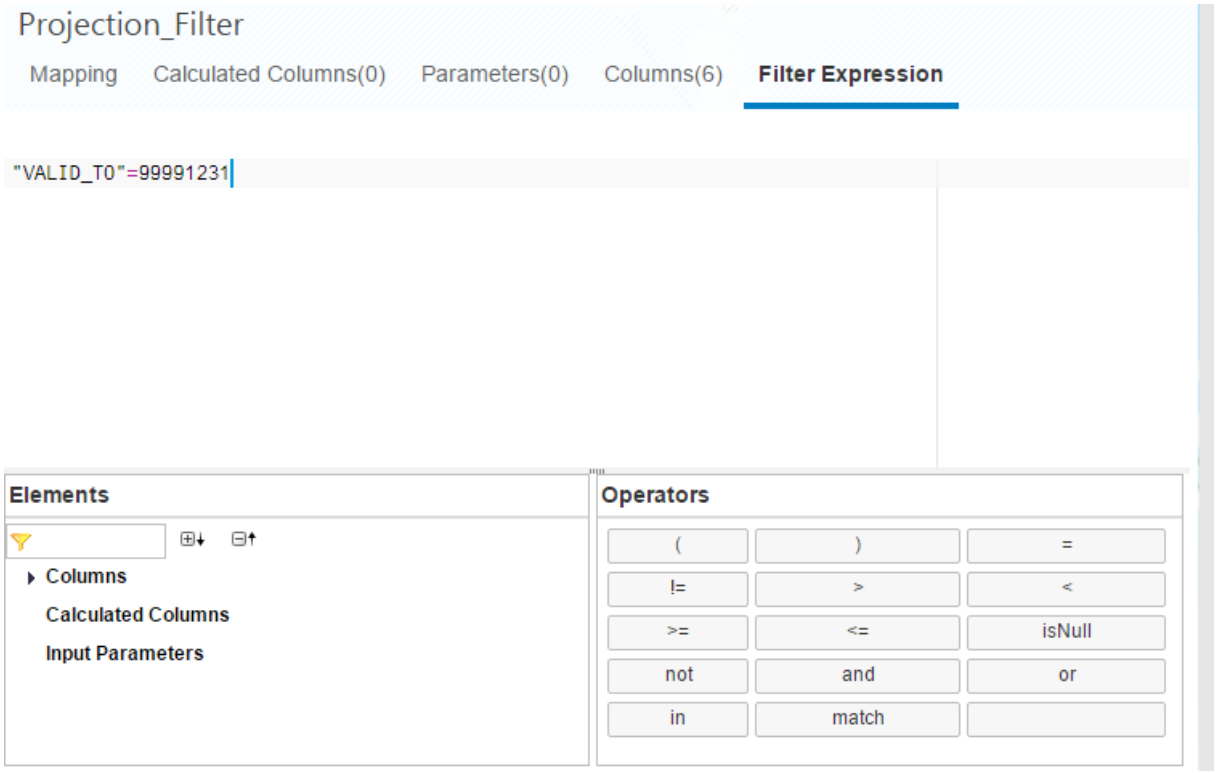


In the mapping section, add all fields as output columns except VALID_FROM.



In the projection, add a filter for field VALID_TO with following expression: "VALID_TO"= 99991231.

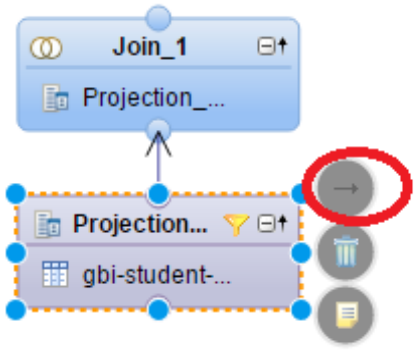
SAP University Alliances



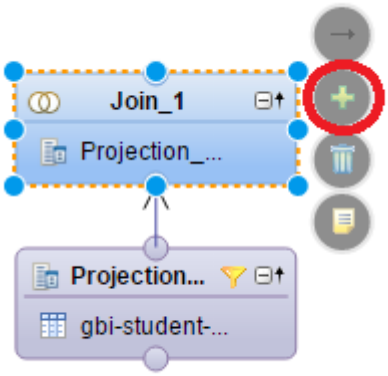
4. Join the table GBI_DEMO_COUNTRY with your customer data using a text join

To join the tables, add a join to the workspace.

Use the result of PROJECTION_1, created in the previous step, as input for this node. To do so, drag-and-drop a connection line between the nodes.

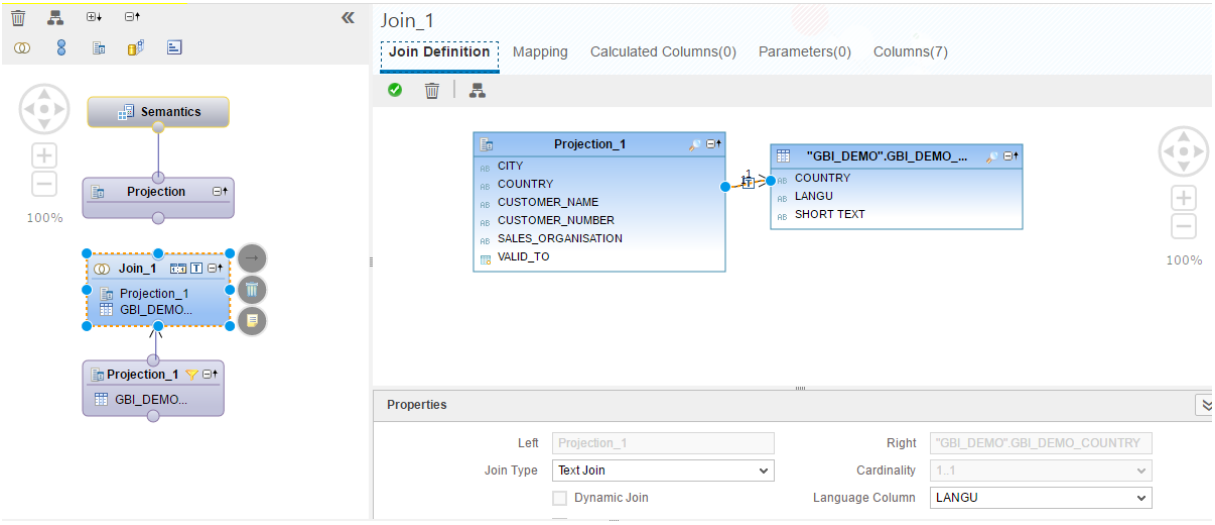


Add the table GBI_DEMO_COUNTRY as data source to the join.



In the JOIN DEFINITION, join both tables using the field COUNTRY.

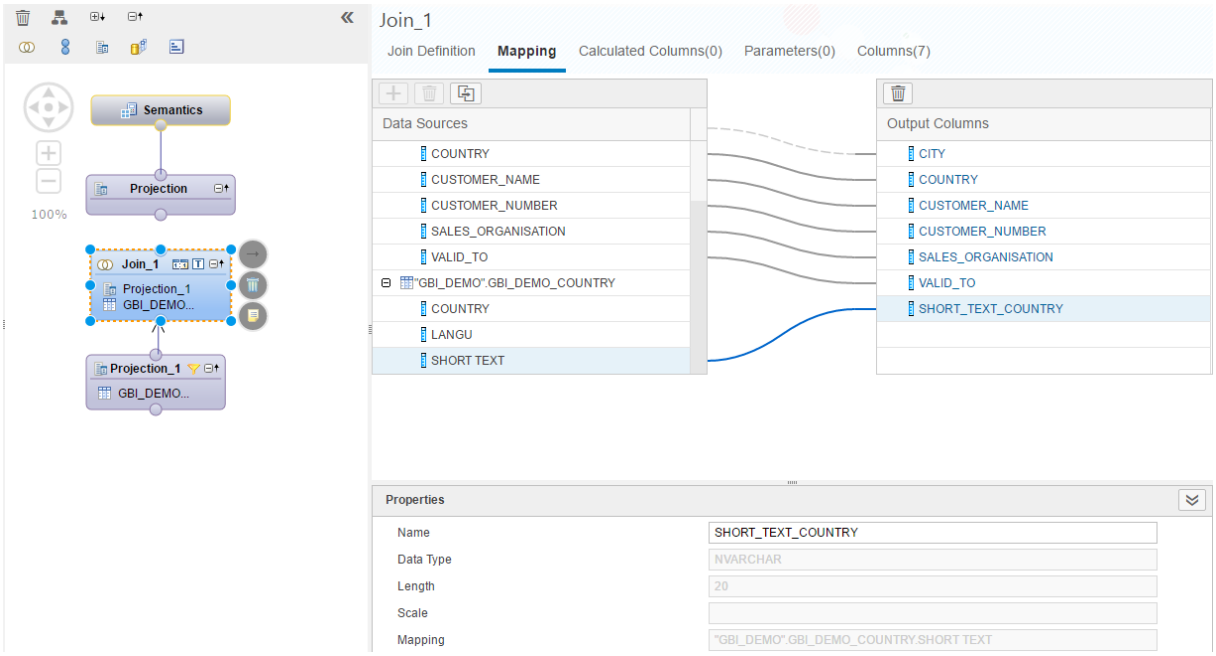
Select TEXT JOIN as join type and GBI_DEMO_COUNTRY.LANGU as language column.



Add all fields of Projection_1 as output columns.

Add the field SHORT_TEXT of table GBI_DEMO_COUNTRY as output column and rename it to SHORT_TEXT_COUNTRY.

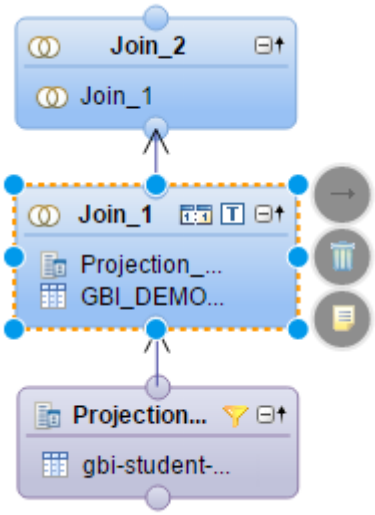
SAP University Alliances



5. Add an additional text join for the sales organization text

Add a new join node to the workspace.

Use the result of JOIN_1 created previously as input for this node.

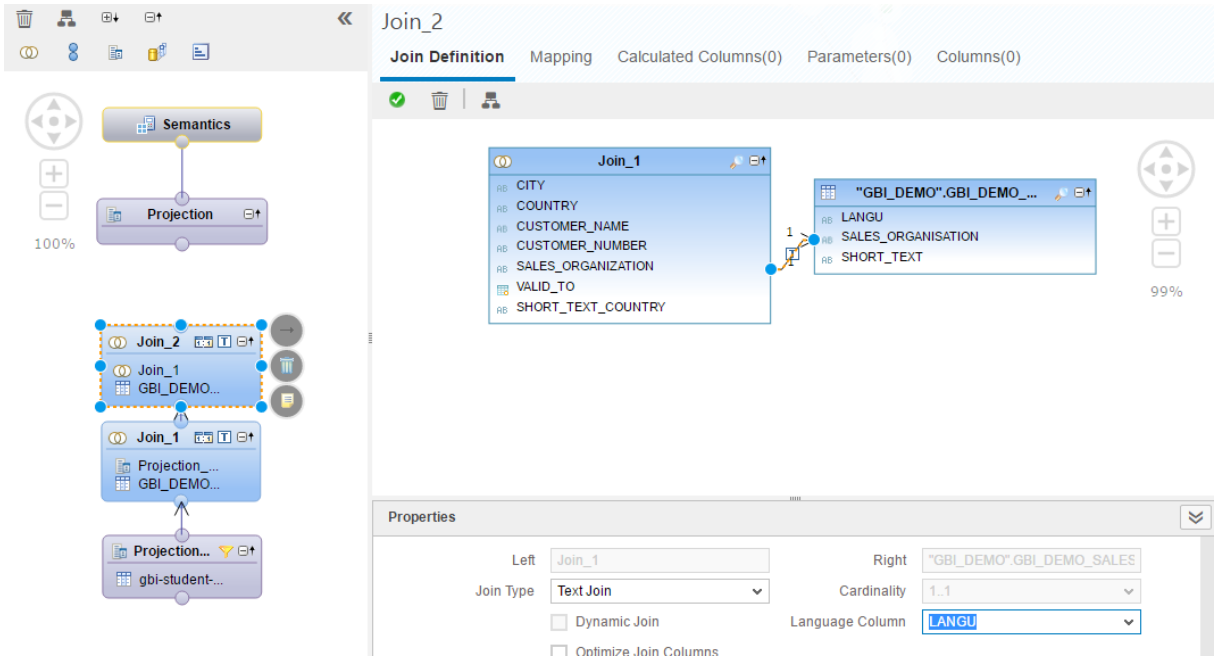


Add the table GBI_DEMO_SALESORG as data source.

Join both tables using the field SALES_ORGANISATION.

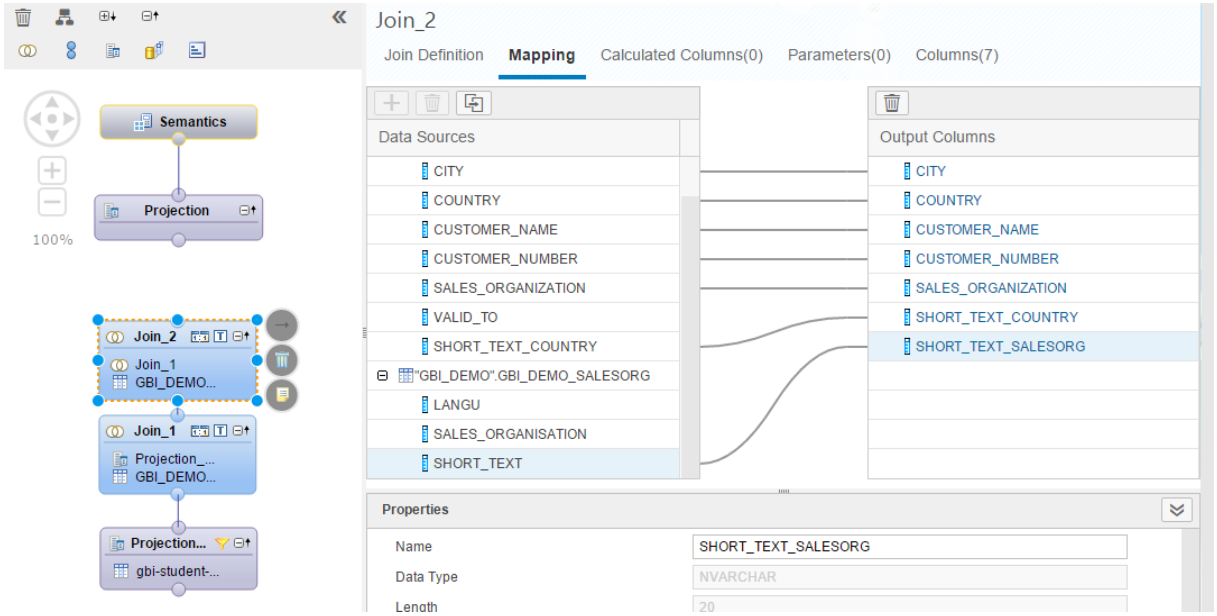
Select TEXT JOIN as join type and GBI_DEMO_SALESORG.LANGU as language column.

SAP University Alliances



Add all fields of JOIN_1 as output columns except VALID_TO.

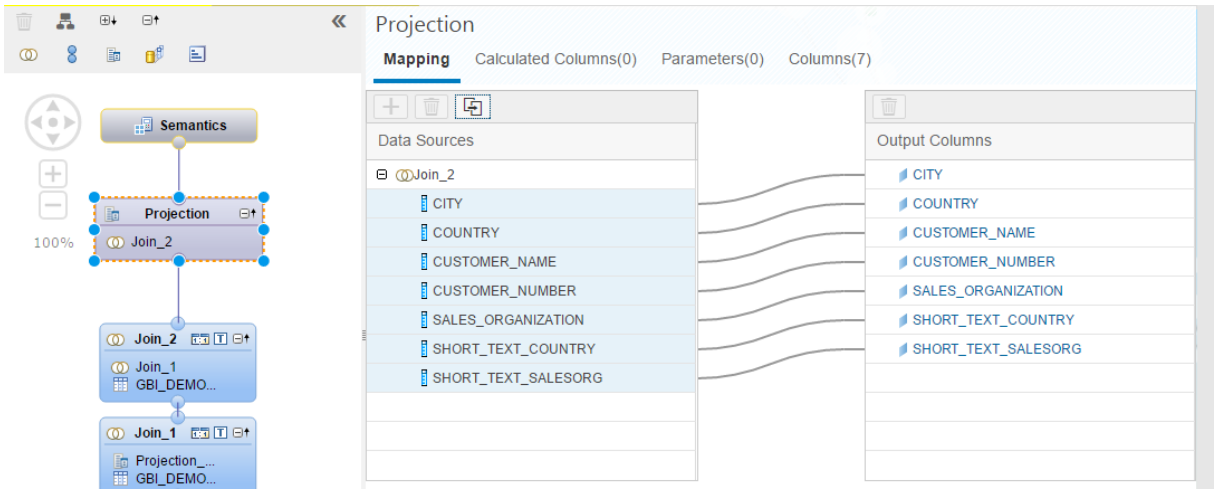
Add the field SHORT_TEXT of table GBI_DEMO_SALESORG as output column and name it SHORT_TEXT_SALESORG.



SAP University Alliances

6. Perform the description mapping for the fields COUNTRY and SALES_ORGANISATION

Link the node JOIN_1 with the node PROJECTION and add all fields as output columns.

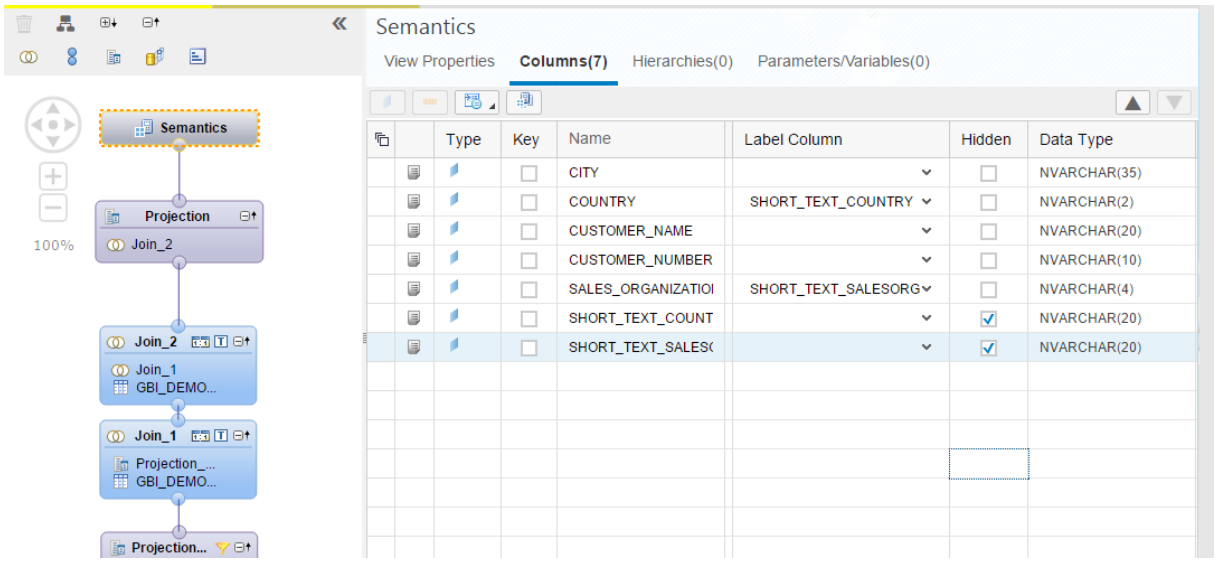


Click on the node Semantics.

For field COUNTRY, set to label column to SHORT_TEXT_COUNTRY.

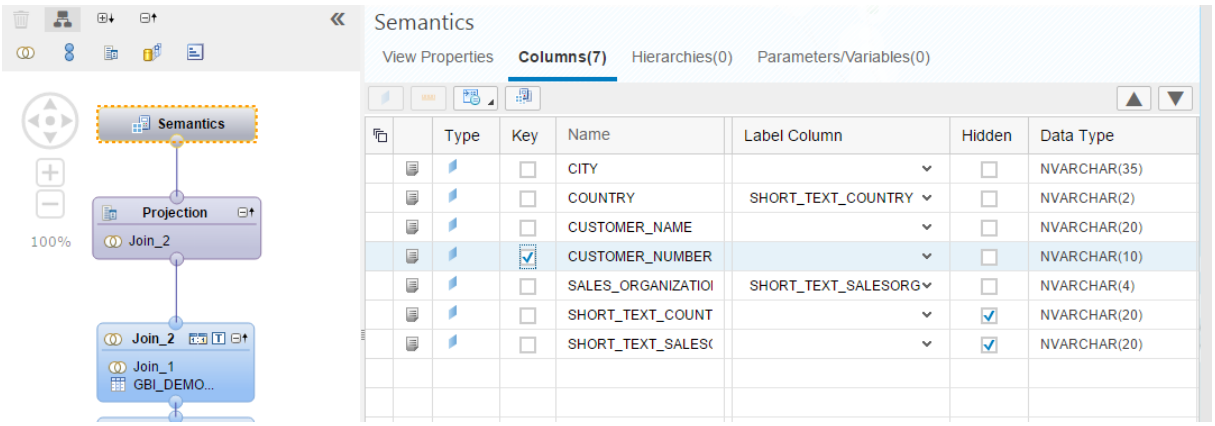
For field SALES_ORGANISATION, set the label column to SHORT_TEXT_SALESORG.

Finally, hide the fields SHORT_TEXT_COUNTRY and SHORT_TEXT_SALESORG.

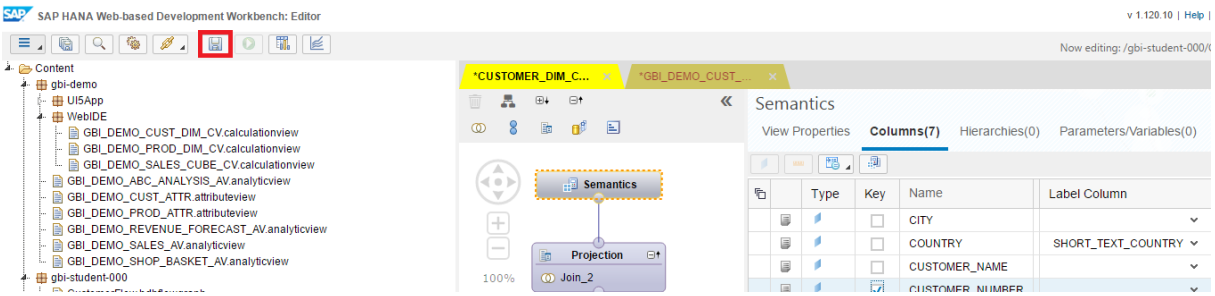


SAP University Alliances

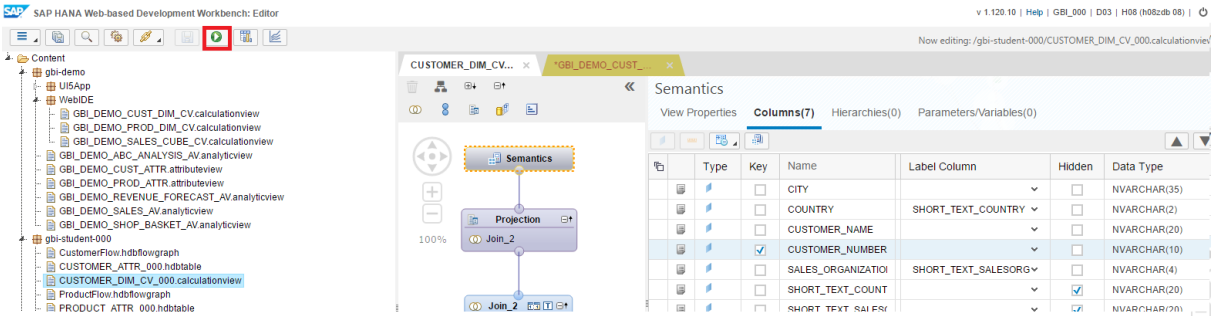
7. Define the customer number as key field for your calculation view



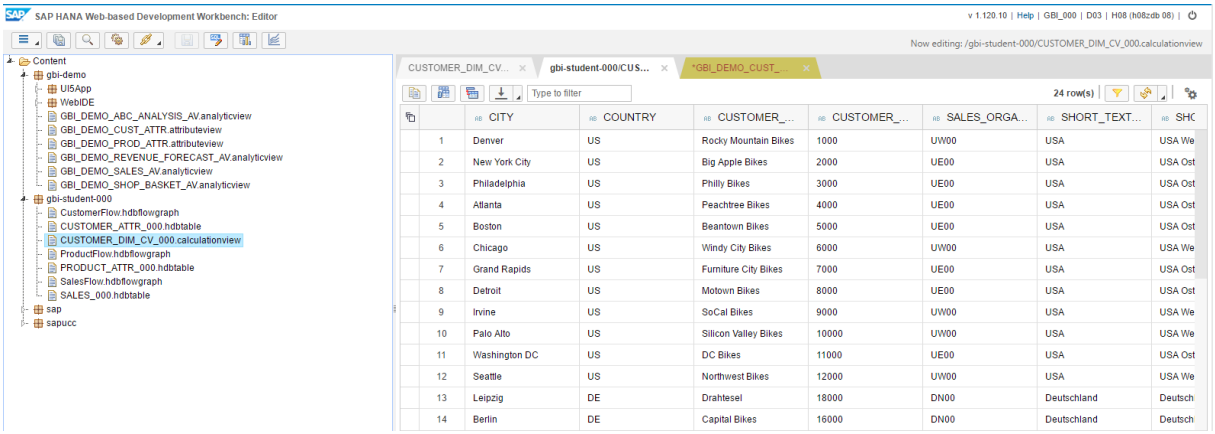
8. Save the calculation view



9. Run the view to check the contents



SAP University Alliances



	CITY	COUNTRY	CUSTOMER_...	CUSTOMER_...	SALES_ORGA...	SHORT_TEXT...	SHO
1	Denver	US	Rocky Mountain Bikes	1000	UW00	USA	USA We
2	New York City	US	Big Apple Bikes	2000	UE00	USA	USA Ost
3	Philadelphia	US	Philly Bikes	3000	UE00	USA	USA Ost
4	Atlanta	US	Peachtree Bikes	4000	UE00	USA	USA Ost
5	Boston	US	Beantown Bikes	5000	UE00	USA	USA Ost
6	Chicago	US	Windy City Bikes	6000	UW00	USA	USA We
7	Grand Rapids	US	Furniture City Bikes	7000	UE00	USA	USA Ost
8	Detroit	US	Motown Bikes	8000	UE00	USA	USA Ost
9	Irvine	US	SoCal Bikes	9000	UW00	USA	USA We
10	Palo Alto	US	Silicon Valley Bikes	10000	UW00	USA	USA We
11	Washington DC	US	DC Bikes	11000	UE00	USA	USA Ost
12	Seattle	US	Northwest Bikes	12000	UW00	USA	USA We
13	Leipzig	DE	Drahtesel	18000	DN00	Deutschland	Deutsch
14	Berlin	DE	Capital Bikes	16000	DN00	Deutschland	Deutsch

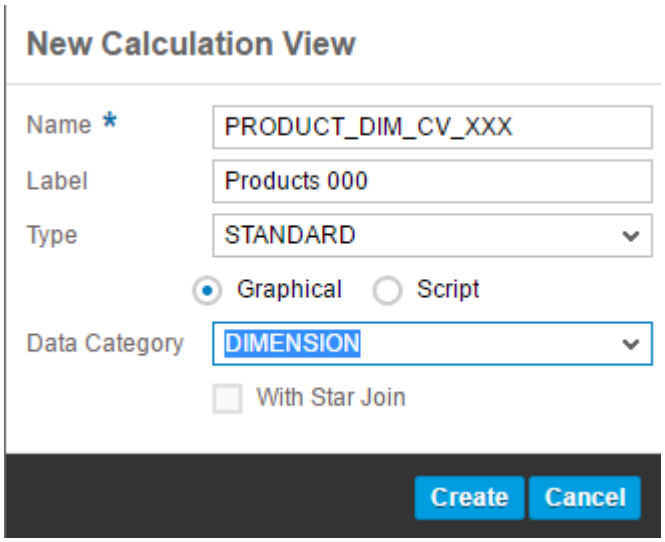
© SAP SE

SAP University Alliances

Step 4: Create a calculation view for product data

1. Create a new calculation view for the product data in your package

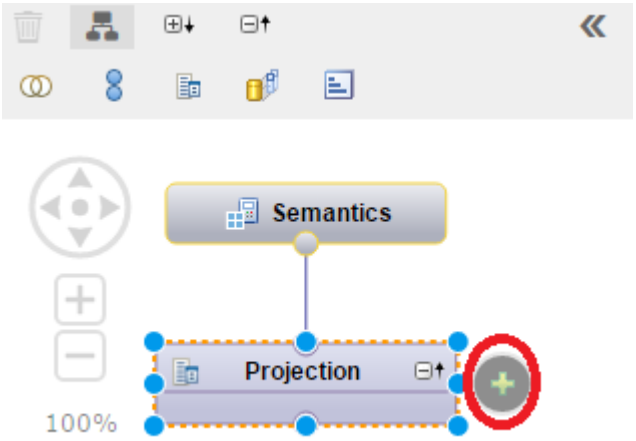
Navigator View → Content → Package gbi-student-xxx → Right click → New → Calculation View



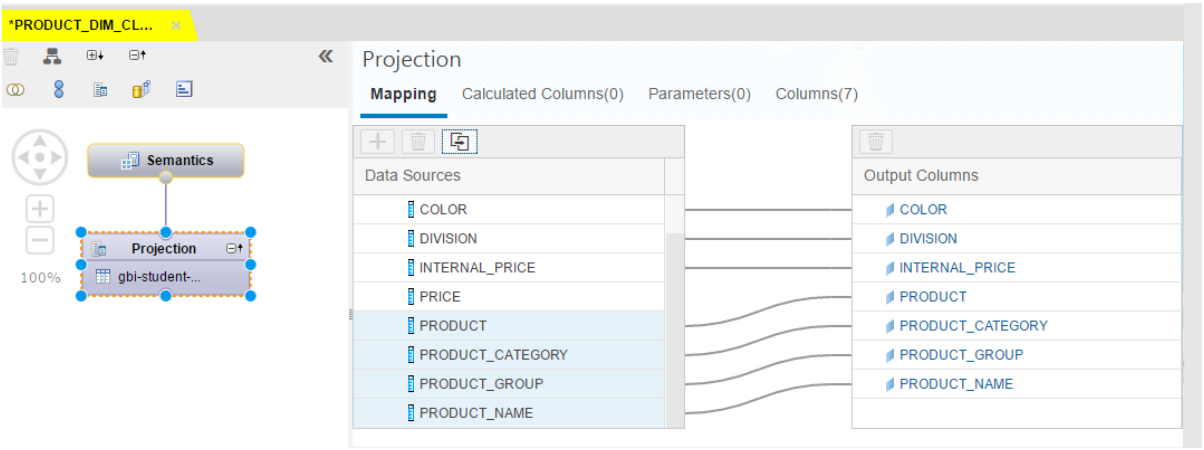
SAP University Alliances

2. Add the product table as data source

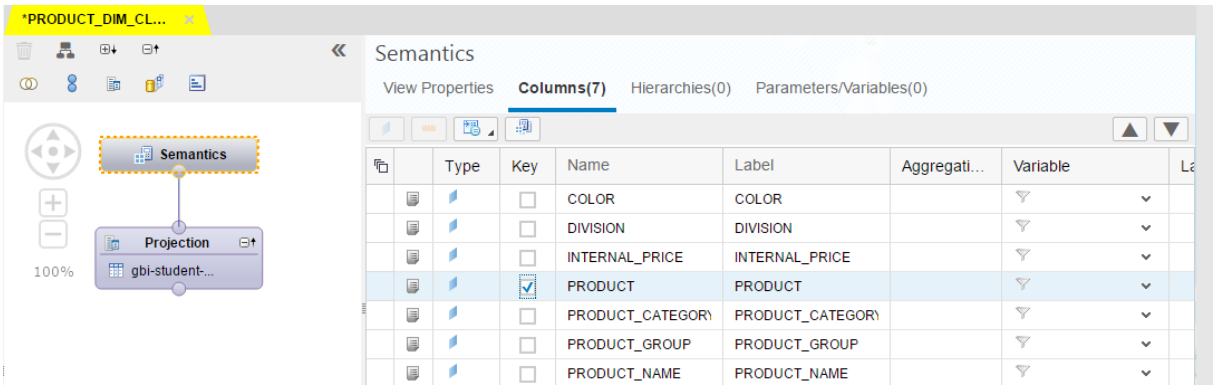
Add the product table PRODUCT_ATTR_XXX as data source to the existing projection node.



Add all fields listed below as output columns.

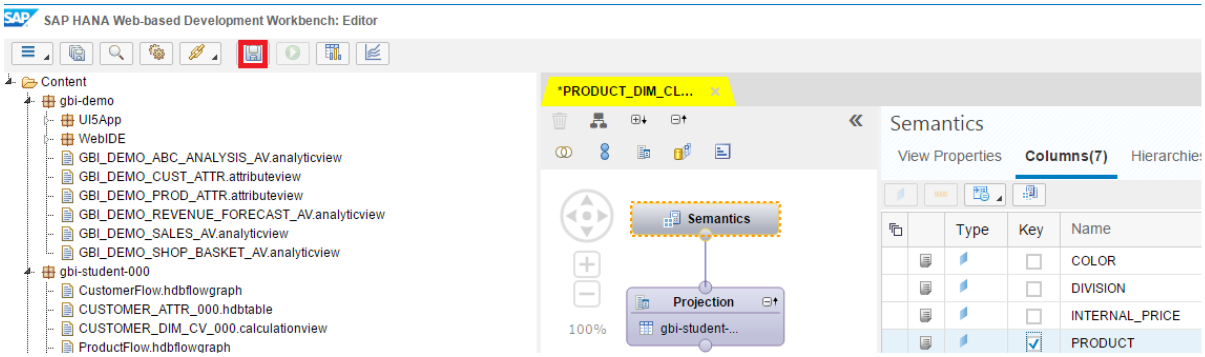


Configure the field product as key field.

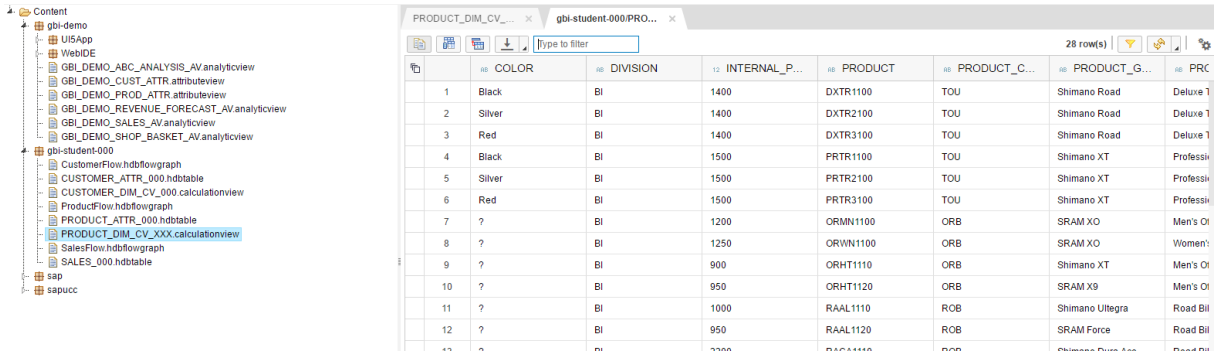


SAP University Alliances

3. Save the calculation view



4. Run the view to check the data

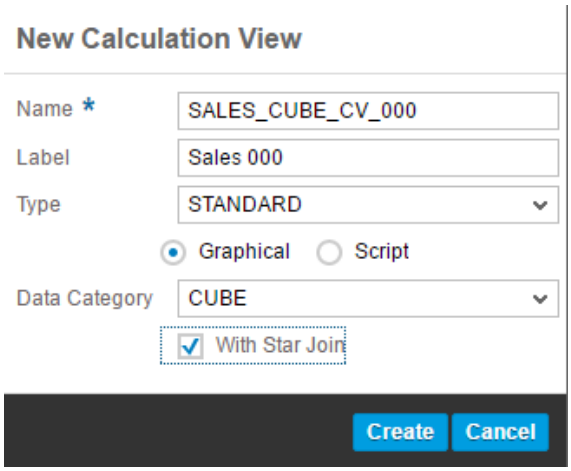


SAP University Alliances

Step 5: Create a calculation view for your sales data

1. Create a new calculation view for sales data

Navigator View → Content → Package gbi-student-xxx → Right click → New → Calculation View

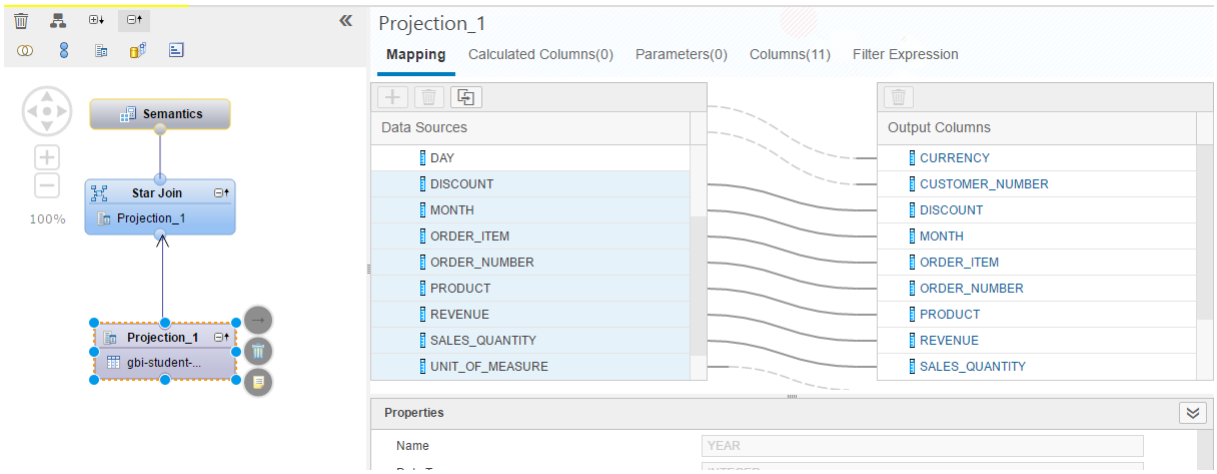


SAP University Alliances

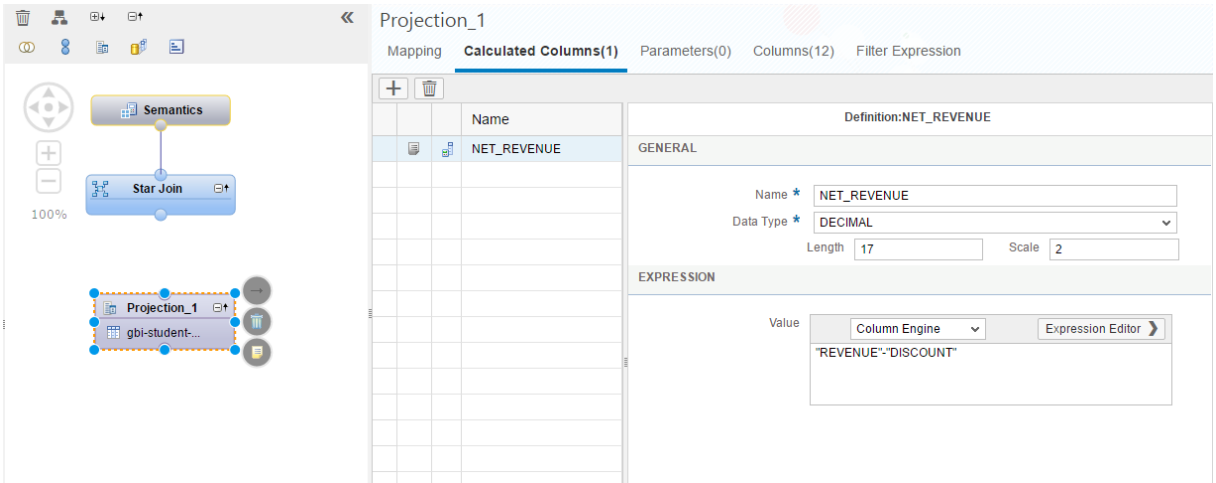
2. Add the table SALES_XXX as input for the star join, using a projection node

Add all fields to the output structure of the projection except the field DAY. You do not need this detail information.

Link the projection node with the star join node.

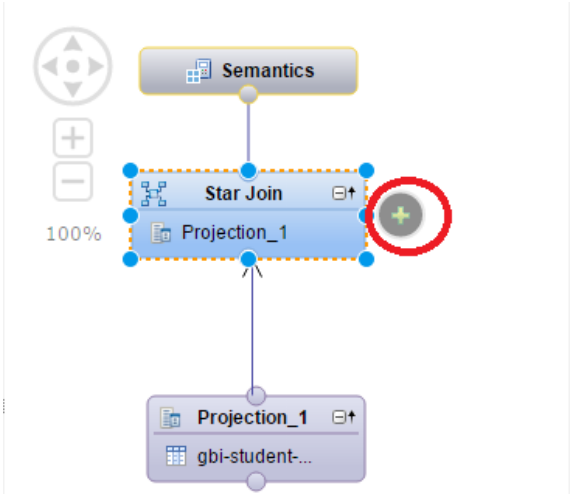


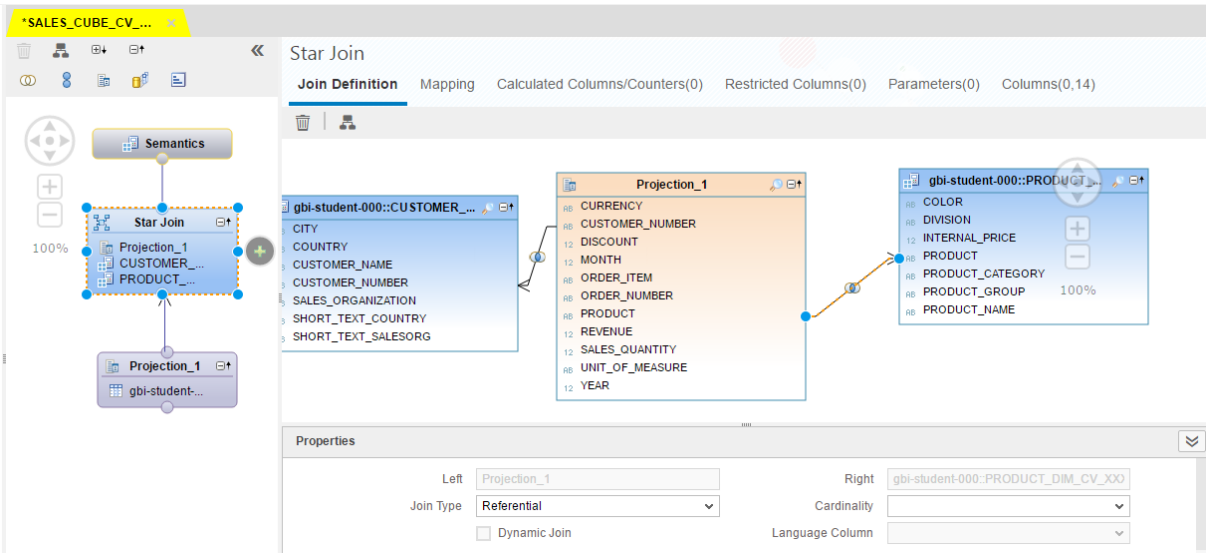
3. Add the net revenue as calculated column to the output structure of the projection

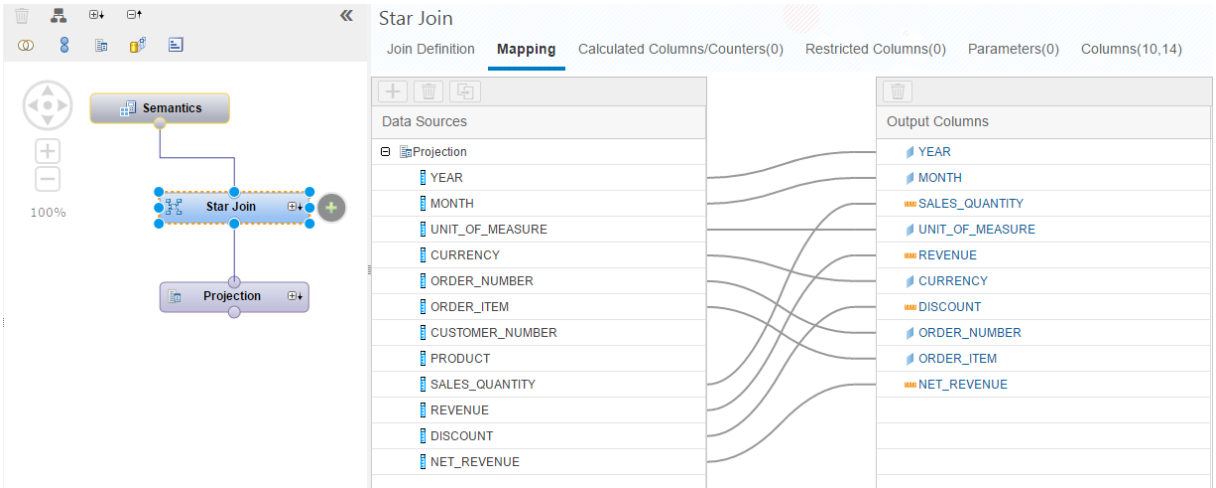


4. Configure the start join node

SAP University Alliances

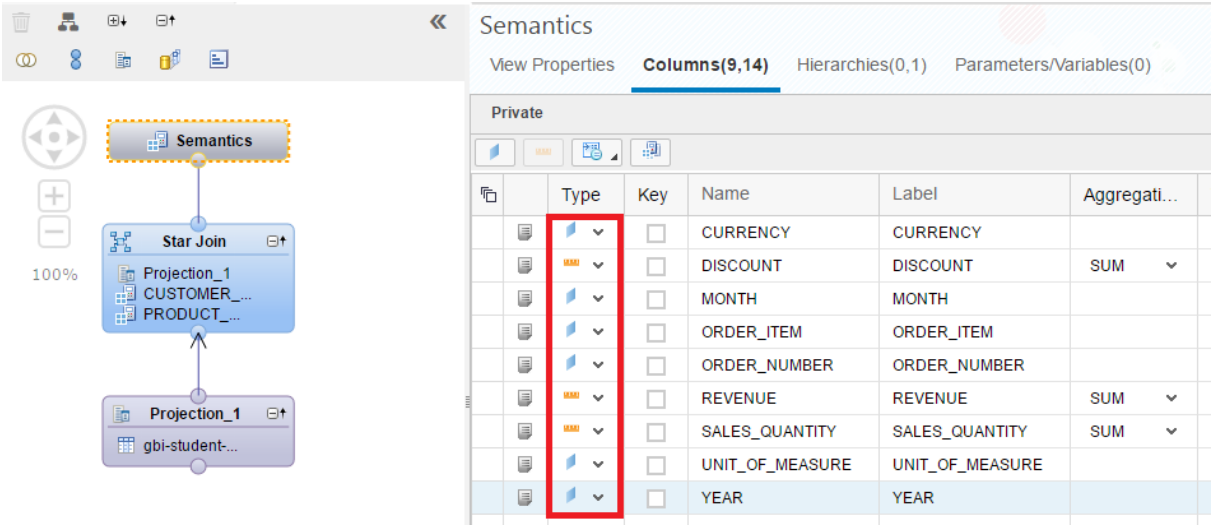




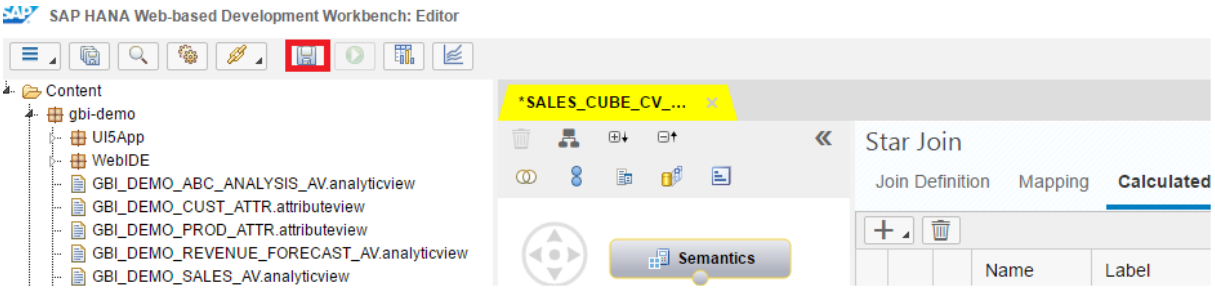


SAP University Alliances

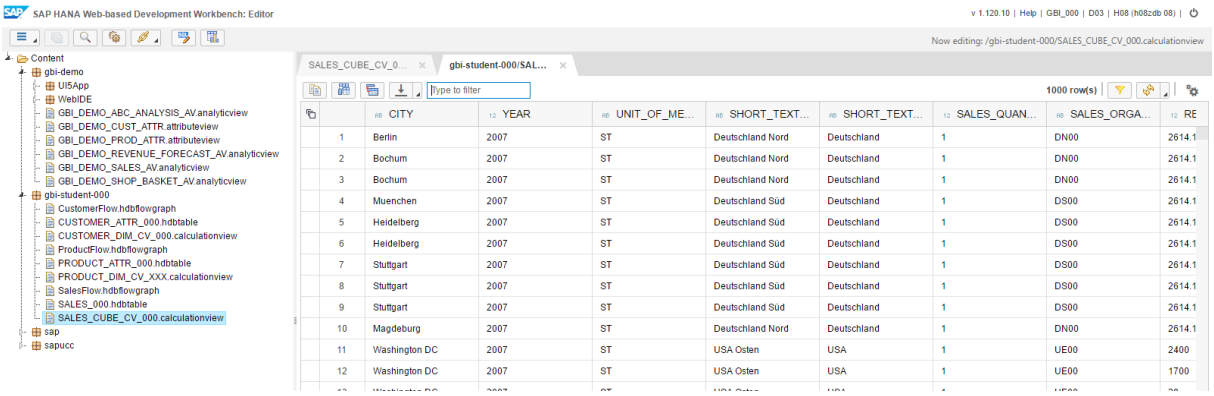
5. Check the data types of the output structure



6. Save the calculation view



7. Run the calculation view and check the data

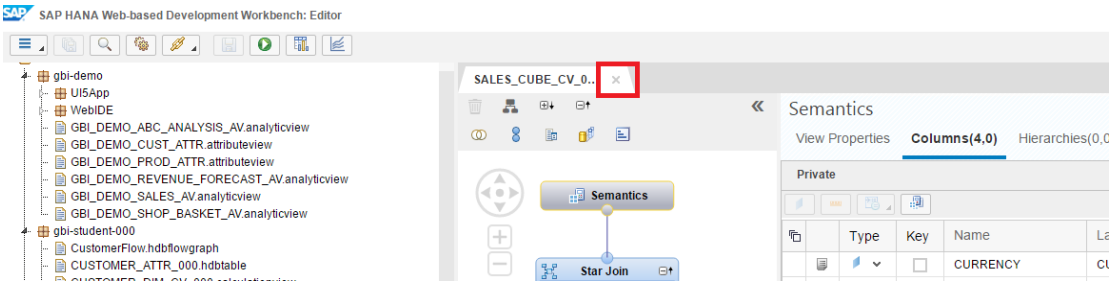


8. Adapt the calculation view in the text editor

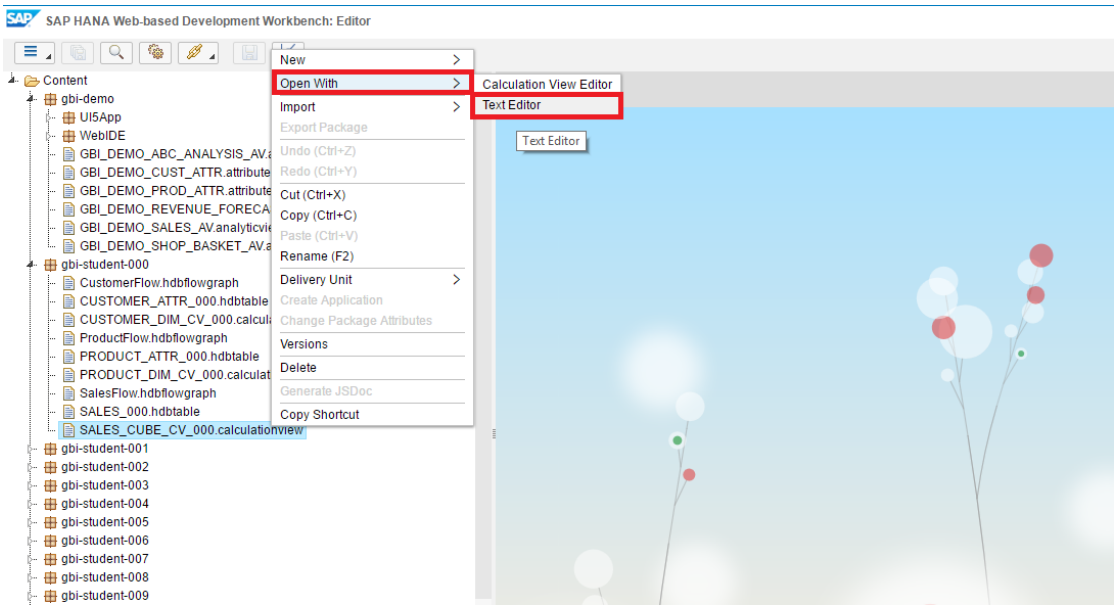
Remark: This is a workaround to expose the data correctly in SAP Lumira. The step might be obsolete in future releases.

SAP University Alliances

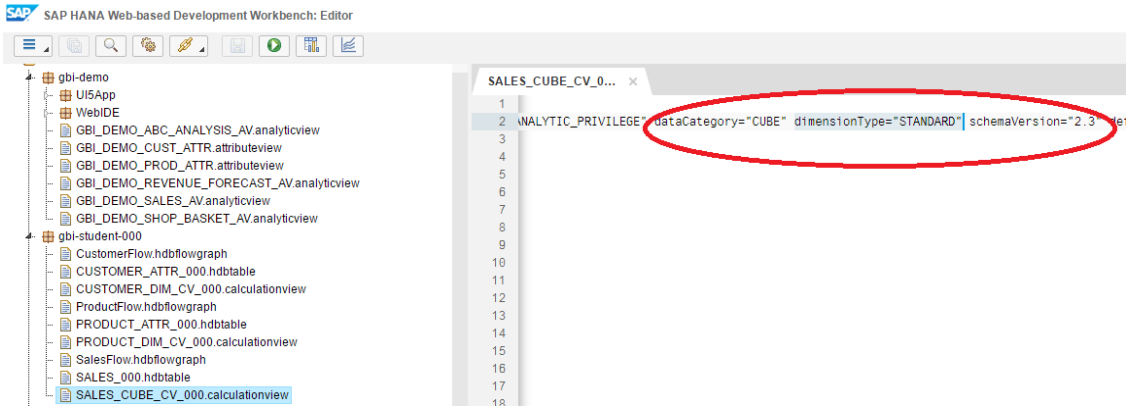
Close the graphical version of the calculation view.



Open the calculation view in the text editor.

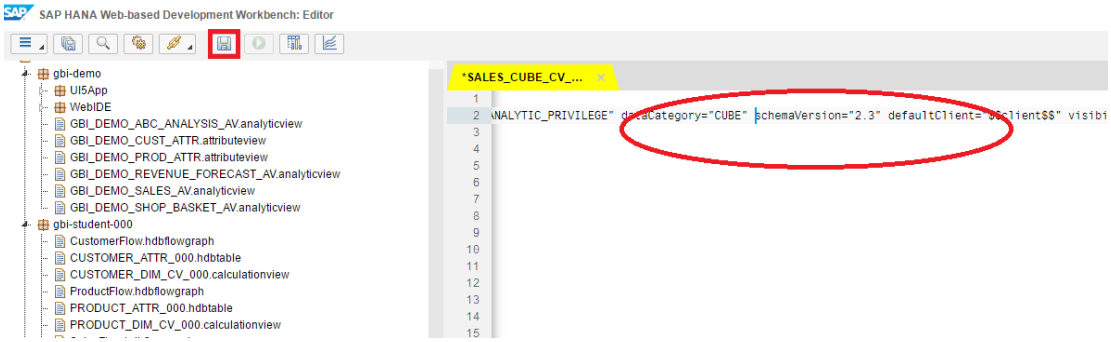


Go do the second line and delete the expression *dimensionType="STANDARD"*



Save the view.

SAP University Alliances



Implement more complex OData services

Task After you've seen how to define an OData service, you will learn how to implement more complex web services and how to use several services at the same time.

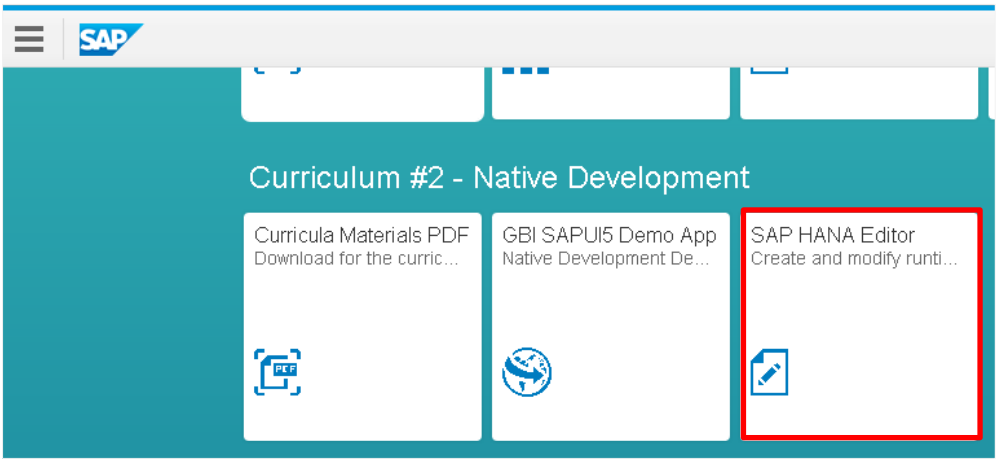
Time 30 min

Short Description Learn how to create multiple entity set endpoints in one service file and how to create complex OData services based on views.

Name (Position) Mona Falco (Development Specialist)

To open the Web-based Development Workbench Editor, log on to the Fiori Launchpad of the SAP HANA system and follow the menu path:

Fiori Launchpad ► Curriculum #2 - Native Development ► SAP HANA Editor



Open the SAP Fiori Launchpad which is available under

`http://[fqdn]:[http port]/`

Logon on with your username and your password. All required information will be provided by your instructor.

You can add multiple endpoints to a service definition.

Open the *GBI.xsodata* file by clicking on it and edit the code so that it looks like this

```
service {
      "GBI_DEMO"."GBI_DEMO_PRODUCT" as "Products";
      "GBI_DEMO"."GBI_DEMO_CUSTOMER" as "Customers";
}
```

Save the file. As stated before, you can also use your own tables instead of the predefined demo content.

After saving the file, the service document lists two endpoints (or item collections): *Products* and *Customers*.

`http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata`



Until now, we've only created OData services based on tables. You can also create an OData service based on Analytic Views.

Add the following code to the service definition in the *GBI.xsodata* file:

```
"gbi-demo.WebIDE::GBI_DEMO_SALES_CUBE_CV" as "Sales"
      keys generate local "ID"
      aggregates always;
```

This adds a third endpoint to the service document. This service will generate an ID for the rows as identification and always aggregates selected measures.

Your service definition should now look like this:

```
service {
      "GBI_DEMO"."GBI_DEMO_PRODUCT" as "Products";
      "GBI_DEMO"."GBI_DEMO_CUSTOMER" as "Customers";

      "gbi-demo.WebIDE::GBI_DEMO_SALES_CUBE_CV" as "Sales"
      keys generate local "ID"
      aggregates always;
}
```

If you like, you can also add your own Calculation View from the SAP HANA Introduction Curriculum.

Your service definition should then look similar to this definition:

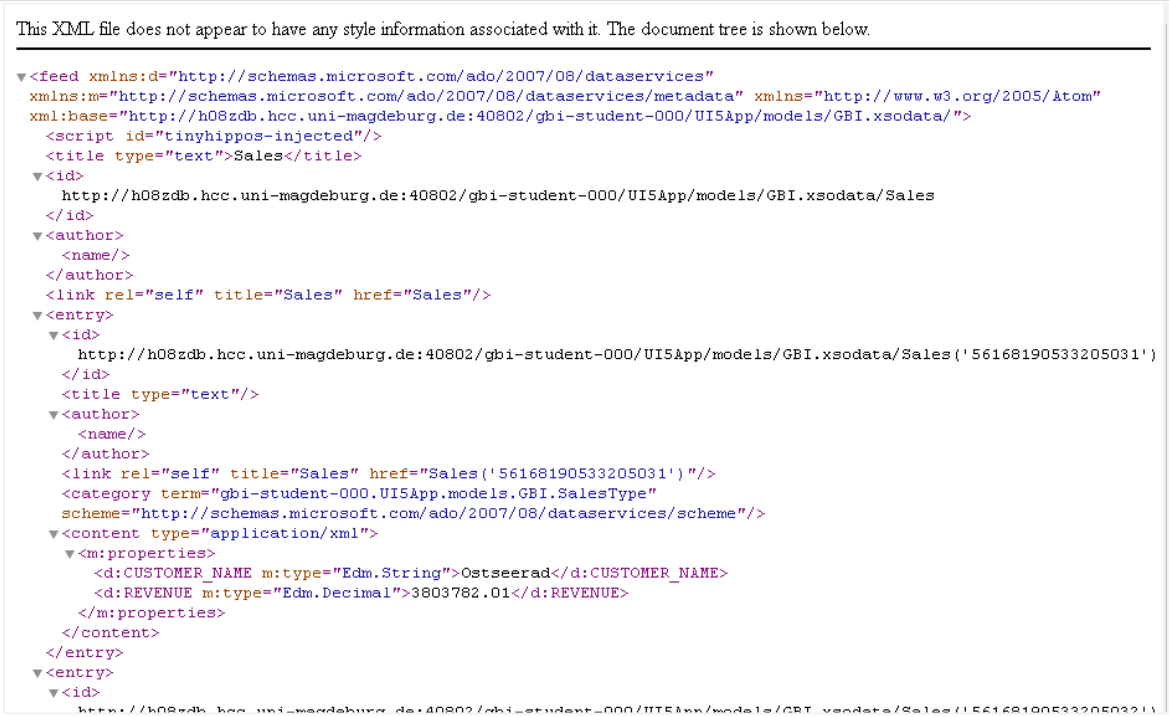
GBI.xsodata

```
1  service {
2      "GBI_DEMO"."GBI_DEMO_PRODUCT" as "Products";
3      "GBI_DEMO"."GBI_DEMO_CUSTOMER" as "Customers";
4
5      "gbi-demo.WebIDE::GBI_DEMO_SALES_CUBE_CV" as "Sales"
6          keys generate local "ID"
7          aggregates always;
8
9  }
```

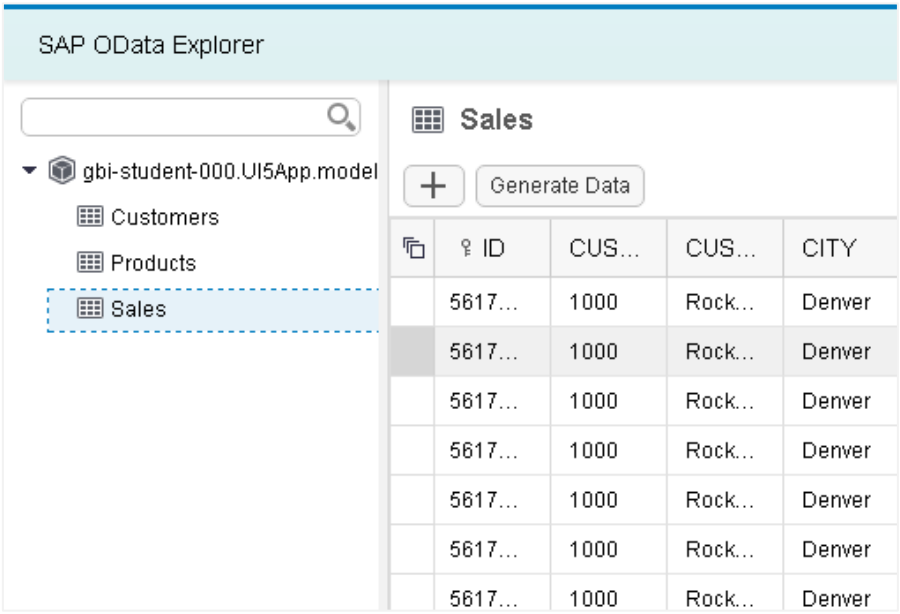
Open the following URL to retrieve the top 5 customers sorted by revenue:

```
http://[fqdn]:[http port]/gbi-student-
###/UI5App/models/GBI.xsodata/Sales?$select=CUSTOMER_NAME,REVENUE&$orderby=

REVENUE&$desc&$top=5
```



Now open the OData Explorer for the file *GBI.xsodata* and select the Service Endpoint *Sales* to take a look at the raw data (you can also see the generated ID as first column):



For the visualization in UI5 App that you're going to build, you need an additional end point which only exposes fewer elements. Open the file *GBI.xsodata* and add the following service

```
"gbi-demo.WebIDE::GBI_DEMO_SALES_CUBE_CV" as "SalesOrders"
        with ("MONTH", "YEAR", "CITY", "NET_REVENUE", "REVENUE")
        keys generate local "ID"
        aggregates always (SUM of "NET_REVENUE", SUM of "REVENUE");
```

with the *with* keyword, only the stated entity properties are exposed. In addition, the aggregation is explicitly specified for the properties *NET_REVENUE* and *REVENUE*.

Open the following URL to see how the filter effects the aggregation:

`http://[fqdn]:[http port]/gbi-student-###/UI5App/models/GBI.xsodata/SalesOrders?$filter=YEAR eq 2007`

You should get the following result:

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<feed xmlns:d="http://schemas.microsoft.com/ado/2007/08/dataservices"
 xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata" xmlns="http://www.w3.org/2005/Atom"
 xml:base="http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-000/UI5App/models/GBI.xsodata/">
  <script id="tinyhippos-injected"/>
  <title type="text">SalesOrders</title>
  <id>
    http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-000/UI5App/models/GBI.xsodata/SalesOrders
  </id>
  <author>
    <name/>
  </author>
  <link rel="self" title="SalesOrders" href="SalesOrders"/>
  <entry>
    <id>
      http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-
      000/UI5App/models/GBI.xsodata/SalesOrders('56204380820000291')
    </id>
    <title type="text"/>
    <author>
      <name/>
    </author>
    <link rel="self" title="SalesOrders" href="SalesOrders('56204380820000291')"/>
    <category term="gbi-student-000.UI5App.models.GBI.SalesOrdersType"
    scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
    <content type="application/xml">
      <m:properties>
        <d:ID m:type="Edm.String">56204380820000291</d:ID>
        <d:CITY m:type="Edm.String">Anklam</d:CITY>
        <d:YEAR m:type="Edm.Int32">2007</d:YEAR>
        <d:MONTH m:type="Edm.Int32">1</d:MONTH>
        <d:REVENUE m:type="Edm.Decimal">2960.57</d:REVENUE>
        <d:NET_REVENUE m:type="Edm.Decimal">2960.57</d:NET_REVENUE>
      </m:properties>
    </content>
  </entry>
  <entry>
    <id>
      http://h08zdb.hcc.uni-magdeburg.de:40802/gbi-student-
      000/UI5App/models/GBI.xsodata/SalesOrders('56204380820000292')
```